



Decision Manager (DM) Installation and Configuration Guide

Document Version : 7.10.2

Software Version: 7.10.2

July 2025



Partnering for Success

Notice

This document contains information that is proprietary to Sapiens International Corporation N.V. or any of its affiliates. No part of this publication may be reproduced, modified, or distributed without prior written authorization of Sapiens International Corporation N.V. or any of its affiliates.

This document is provided as is, without warranty of any kind.

Trademarks

Sapiens International Corporation N.V.® is a registered trademark.

Sapiens Decision™ is a trademark of Sapiens International Corporation N.V. Other names mentioned in this publication are owned by their respective holders.

Statement of Conditions

The information contained in this document is subject to change without notice. Sapiens International Corporation N.V. and/or any of its affiliates shall not be liable for errors contained herein or for any damage or losses arising out of or in connection with the furnishing, performance, or use of this document or the software supplied with it.

Caution

Any changes or modifications of the software not expressly approved in writing by the manufacturer could void the user's authority to use the application and its warranty.

Document Revision History

Revision	Date	Author	Description	Approved by
7.9.2	May 2025	Decision Documentation	Describes the DM 7.9.2 Release	Decision Product Management
7.9.2a	May 2025	Decision Documentation	Updated the Component-Specific Prerequisites table in Section 4.5.4 to reflect the latest configuration requirements	Decision Product Management
7.10.2	July 2025	Decision Documentation	Describes the DM 7.10.2 Release	Decision Product Management

Document Authorization

This document is digitally signed and authorized by the Sapiens Decision documentation team. The steps to authenticate the digital certificate are documented in the Index file present in the documentation package. In case of any queries or feedback on the document, send an e-mail to Decision_documentation@sapiens.com.



Copyright © 2025 by Sapiens International Corporation N.V. All rights reserved.

Table Of Contents

1. About this Guide	11
2. Version Compatibility	12
3. Decision Components	13
3.1 Decision Manager (DM)	13
3.2 Decision Execution (DE)	13
3.3 Decision Testing (DT)	14
3.3.1 DT Repository	14
3.3.2 DT Executor	14
3.3.3 DT Generate	14
3.3.4 DT Repair	14
3.3.5 DT Orchestrator	15
3.4 Decision Gateway	15
3.5 Parameter Management (PaM)	15
3.6 Health Service	16
3.7 Configuration Service	16
4. Decision Deployment & Installation	17
4.1 Overview	17
4.2 Database Implementation	17
4.2.1 Database – DM	17
Oracle	17
PostgreSQL	17
4.2.2 Database – DT	18
4.2.3 Database – DE	18
4.3 Containerized Image Deployment	18
4.3.1 Overview	18
4.3.1.1 Accessing the README.md File for Deployment Instructions	19
4.3.2 Package Content	20
4.3.3 Recommended Use Cases	20
4.3.4 Prerequisites	20
4.3.5 Installation	22
4.4 “All-in-One” Deployment	22
4.4.1 Overview	22
4.4.2 Package Content	23

4.4.3 Recommended Use Cases	23
4.4.4 Prerequisites	23
4.4.5 Installation	25
Step 1: Set Up Global Class-Path and Configuration Folders	25
Step 2: Configure Application Contexts in Tomcat	26
Step 3: Configure Tomcat HTTP and Shutdown Ports	27
Step 4: Create and Configure Tomcat setenv.sh Script	27
Step 5: Prepare Database Schemas	28
Step 6: Deploy Application WAR Files	28
Step 7: Configure Application Properties	29
4.5 Standalone Deployment	29
4.5.1 Overview	29
4.5.2 Package Content	30
4.5.3 Recommended Use Cases	30
4.5.4 Prerequisites	31
Java Installation (Global Requirement)	31
Component-Specific Prerequisites	31
4.5.5 Installation	32
5. Configuration	34
5.1 Configuration location	34
5.1.1 Containerized Deployment	34
5.1.2 All-in-One Deployment	34
5.1.3 Standalone Deployment	34
5.2 Property Value encryption	35
5.2.1 Overview	35
5.2.2 How Property Encryption Works	35
5.2.3 Encryption Tool Usage	35
5.2.4 Generating Encrypted Property Values	35
5.3 Authentication Types	37
5.4 DM	37
5.4.1 Containerized Deployment	37
5.4.2 All-in-One Deployment	37
5.4.3 Standalone Deployment	37
5.4.4 Basic Configuration	37
5.4.5 Authentication	38
Username-Password Login Form (default)	38

Header-Based Authentication	39
OAuth 2.0 Authentication	39
SAML Single Sign-On Authentication	40
LDAP Authentication	40
Integration with other Decision Components Configuration	40
5.5 DE	42
5.5.1 Containerized Deployment	43
5.5.2 All-in-One Deployment	43
5.5.3 Standalone Deployment	43
5.5.4 Basic Configuration	43
5.5.5 Authentication	44
Basic Authentication	44
OAuth 2.0 Authentication	44
5.6 DT	44
5.6.1 Authentication	45
Restricted Origin (default)	45
OAuth 2.0 Authentication	46
5.6.2 DT Repository	46
Containerized Deployment	46
All-in-One Deployment	46
Standalone Deployment	47
Basic Configurations	47
OAuth 2.0 Authentication	48
Integration with Decision Manager (DM)	48
5.6.3 DT Orchestrator Configuration	48
Containerized Deployment	49
All-in-One Deployment	49
Standalone Deployment	49
Basic Configurations	49
OAuth 2.0 Authentication	49
Integration with other Decision Components	50
5.6.4 DT Executor	51
Containerized Deployment	51
All-in-One Deployment	51
Standalone Deployment	52
Basic Configurations	52

OAuth 2.0 Authentication	52
Integration with other Decision Components	52
5.6.5 DT Generate	54
Containerized Deployment	54
All-in-One Deployment	54
Standalone Deployment	54
Basic Configurations	55
OAuth 2.0 Authentication	55
Integration with DT Repository	55
5.6.6 DT Repair Configuration	56
Containerized Deployment	56
All-in-One Deployment	56
Standalone Deployment	57
Basic Configurations	57
OAuth 2.0 Authentication	57
Integration with DT Repository	57
5.7 Decision Gateway	58
5.7.1 Containerized Deployment	58
5.7.2 Standalone Deployment	59
5.7.3 Basic Configuration	59
5.7.4 OAuth 2.0 Authentication	59
5.7.5 Integration with Other Decision Components	59
5.7.6 Session Management	60
Redis Session Store Configuration	60
In-Memory Session Store Configuration	61
5.8 Config Server	61
5.8.1 Containerized Deployment	61
5.8.2 Standalone Deployment	62
5.8.3 Basic Configurations	62
5.8.4 Authentication (Basic Authentication)	62
5.9 RabbitMQ	62
5.9.1 Standalone Deployment	62
5.9.2 Containerized Deployment	63
5.10 Parameter Management (PaM)	63
5.10.1 Containerized Deployment	63
5.10.2 Standalone Deployment	63

5.11 Health Service	63
5.11.1 All-in-One Deployment	63
5.11.2 Basic Configurations	64
5.11.3 Authentication	66
5.12 Logging	66
5.12.1 Log Files Location	66
5.12.2 Log Level Configuration	66
5.12.3 External Log Configuration File	67
6. Startup	68
6.1 Containerized	68
6.2 All-in-one Deployment	68
6.3 Standalone	68
7. Post Installation Validation	70
7.1 Health Check Validation	70
7.2 Health Check and Heartbeat API Endpoints	71
8. Advanced Configuration Topics	72
8.1 Customized Reports	72
8.1.1 Prerequisites	72
8.1.2 Adding a New Report to Decision	72
8.1.3 About Reports	72
8.1.4 XML File Attributes	74
8.1.5 Important Issues	79
8.1.6 Entity Type Names and Classes	80
8.2 Customized Configuration Properties	83
8.2.1 Configuration Files and Location	83
8.2.1.1 Containerized Deployment	84
8.2.1.2 All-in-One Deployment	84
8.2.1.3 Standalone Deployment	84
8.2.2 Configuration Files	85
8.2.3 Advanced DT related properties on DM	85
8.2.4 Optional Configuration	85
8.3 Custom Functions in Decision Modeling	85
8.3.1 Defining a Custom Function Prototype	86
8.3.2 Adding a Custom Implementation in Decision	89
8.3.2.1 Custom Functions Jars Deployment	89
Containerized Deployment	89

All-in-one Deployment	89
Standalone Deployment	90
8.3.3 Appendix: Java Code Examples	90
8.4 Customizing External Rule IDs	91
8.4.1 External Rule IDs	91
8.4.2 Configuring the External Rule ID	91
8.4.3 Resetting the External Rule IDs for Existing Logic	92
8.4.4 Managing External Rule IDs in Decision	93
8.4.5 Validating the External Rule ID	97
8.4.6 Locating Assets by their External Rule IDs	98
8.5 Clustering Configuration	99
8.5.1 Configuring a DM Cluster	99
8.5.1.1 Standard Environment Configuration	99
8.5.1.2 AWS Environment Configuration	104
8.5.1.3 Hazelcast Configuration Properties for Clustering	104
8.6 JNDI Configuration (Tomcat Data Source)	110
8.6.1 JNDI Configuration – DM	111
8.6.2 JNDI Configuration – DT	111
8.6.3 Configuring Internal Thread Pool Mechanism	112
8.7. Authorization and Authentication	112
8.7.1 Configuring LDAP for DM	112
8.7.2 LDAP enhancements for private Communities	115
8.7.3 SAML Single sign-on (SSO) Authentication	116
8.7.3.1 Configuring DM for SSO authentication	117
DM Configuration	117
Configuring Basic SSO	117
Setting Single IDP with IDP Metadata file	118
Setting Single IDP with Metadata URL	118
Setting Multiple IDPs - IDP Metadata	119
Multiple IDPs - IDP Selection Prompt Page	120
DM SAML KeyStore configuration	120
KeyStore Generation	120
KeyStore DM Configuration	121
8.7.3.2 Customize DM SSO configurations	121
8.7.3.3 IDP Configuration	123
ADFS	123

Okta	124
8.7.4 Integrating OAuth with DM	124
8.7.4.1 OAuth 2.0 Configuration	125
OAuth 2.0 Authentication – OAuth IDP is accessible by DM	125
OAuth 2.0 Authentication - OAuth IDP JWKS URI	126
OAuth 2.0 Authentication – OAuth IDP is not accessible by DM	126
OAuth 2.0 Authentication – Scope Validation	127
8.7.4.2 Auto Synchronizing User Display name and email (in OAuth environment)	127
8.7.4.3 Configuring DM User Identifier from OAuth Token	128
8.7.5 Activate Users Auto-Creation	128
8.7.6 Configuring auto synchronization of Roles with external IDP	129
8.7.6.1 Using SAML	129
8.7.6.2 Using OAuth	130
8.7.6.3 Property Valid Values	130
8.7.6.4 Additional Properties to read community Roles from IDP	131
8.8 Configure CAPTCHA support	133
8.9 Configure Parameter Management Link on DM Header	134
8.10 Test with a Java Double Implementation	134
8.11 Test with Java Date or Timestamp	135
8.12 Appendix	135
8.12.1 Configuration Files	135
8.12.1.1 DM application.properties file	135
8.12.1.2 DT apps application.properties files	153
8.12.1.3 DT Repository application.properties	165
8.12.1.4 DT Generate application.properties	174
8.12.1.5 DT Repair application.properties	177
8.12.1.6 DT Executor application.properties	178
8.12.1.7 DT Orchestrator application.properties	181
8.12.1.8 Health Service application.properties	188
8.12.2 Advanced DT-related properties on DM	189
8.12.3 Glossary	190
8.12.4 Troubleshooting	191
8.12.4.1 Align IDP users with DM users	191
8.12.4.2 DM metadata generation failure	192

1. About this Guide

This guide provides installation and configuration instructions for the Decision Components in any of multiple deployment types – all components under one Tomcat, each component in its own embedded Tomcat, or each component as a container image.

The instructions in this guide are mainly applicable for the PostgreSQL database, although additional information is provided, where applicable, for working with Oracle DB server / MS-SQL DB.

Prerequisites and Notes

- This guide assumes technical knowledge in the relevant operating system, RDBMS, Tomcat setup, and security requirements.
- For containerized deployment, this guide assumes knowledge of Kubernetes and the Azure cloud implementation of Kubernetes (AKS).
- Installation setup assumes access to RDBMS is provided - Oracle, MS-SQL Server, and PostgreSQL, with the required DBA rights.
- All firewall definitions and access rights to the different services are assumed ready.

Related Documents

For information about the latest release of each of the Decision applications, refer to its latest relevant Release Notes document.

2. Version Compatibility

For version compatibility information for the Decision Manager (DM) application, refer to the **Version Compatibility** section in the *Decision Manager (DM) Release Notes*.

3. Decision Components

Decision has been componentized to allow for a container-based deployment that enables optimal usage of container-based platforms like Kubernetes, OpenShift, AKS, and EKS. This deployment is referred to as the 'container-based' deployment.

Traditional WAR deployment is still supported and is referred to as the 'all-in-one' deployment.

When using standalone or containerized deployment, the dependencies described for each component below are especially important for connectivity between the applications and for performance considerations (deploying multiple applications on the same machine or close-by machines).

3.1 Decision Manager (DM)

Decision Manager is the core component of Decision, responsible for the architecture and design of the Decision management system

DM hosts the Decision Angular client and supplies services for Decision user administration and security, user communities, Decision administration, Decision asset modeling, code generation, governance and version control, deployment to DE, and many other related functionalities.

Dependencies:

- **DM server dependencies:**
 - A database schema (PostgreSQL or Oracle) with all permissions - **mandatory**.
 - DT Orchestrator component and DT Repository component – **required** to include test group export on asset deployment to a DE environment.
- **DM Angular client (hosted by DM, requires no installation) dependencies:**
 - All DT components and the DE component – required for all testing and test group/case-related operations.

3.2 Decision Execution (DE)

The Decision Execution component provides the runtime capabilities of the Decision management system and acts as the repository of deployed Decision executable assets.

DE supplies services for the Java execution of any Decision or Flow asset deployed to it, given the execution inputs, and responds with the execution results. It is designed for maximum performance during execution and offers fine-tuning for sophisticated caching capabilities and memory efficiency based on artifact execution frequency and execution purpose.

Important Note:

Whenever DE components are separately licensed, there will usually be **two or more DE component instances** installed:

- A dedicated instance for test executions of assets within DM during their development or test phases.
- One or more instances where approved assets are deployed to provide decision services to calling applications.

Although possible, from a performance perspective, it is strongly **advised not to use the same DE instance for both asset testing and runtime execution environments**.

Dependencies:

- A database schema (PostgreSQL, Oracle, or MS-SQL) with all permissions - **mandatory**.

3.3 Decision Testing (DT)

Decision Testing is a bundle of five microservice applications responsible for asset test group and test case management and execution with DE. The DT components collectively support testing and test group/case-related operations within the DM component.

3.3.1 DT Repository

DT Repository provides persistence services to save, edit, delete, import/export, and manage Decision asset test groups and test cases

Dependencies:

- **DM** – required for test group/case purge, orphan auto-delete, and test group copy.
- A database schema (PostgreSQL or Oracle) with all permissions.

3.3.2 DT Executor

DT Executor provides services for carrying out test case execution requests to DE and handling their results.

Dependencies:

- **DE** – The internal DE instance (not to be confused with runtime execution DE instances configured under Environment Management) is required for the execution of tests.
- **DT Repository** – required to persist execution results and other related execution information.

3.3.3 DT Generate

DT Generate provides services for generating test cases consisting of combinations of the different possible input values for the Decision asset.

Dependencies:

- **DT Repository** – required to save the generated test cases.

3.3.4 DT Repair

DT Repair provides services for the automatic repair of inputs of existing asset test cases when the asset input structure has changed, invalidating the previously saved tests.

Dependencies:

- **DT Repository** – required to save the test case repair changes.

3.3.5 DT Orchestrator

DT Orchestrator orchestrates the various complex DT operations that require a combination of services from multiple DT microservice components.

Dependencies:

- **All other DT microservice components** – required to orchestrate from the UI most of their operations.
- **DM** – mandatory for all operations related to new or changed assets to display asset and test details, test/test group execution, and the deployment of new or changed assets to the DE component testing instance.

3.4 Decision Gateway

The Decision Gateway component acts as a channel through which all the Decision Angular client requests pass and is responsible for performing user OAuth 2.0 authentication.

It decorates each request with the relevant OAuth 2.0 token, which is obtained through the IDP's login page presented to the user and refreshes the token if required.

The Decision Gateway can be used only if **OAuth2 authentication** is utilized across all the installed Decision components configured with it. It is only applicable if the components are installed as **standalone or containerized deployment types** (see Decision deployment).

Dependencies:

- OAuth IDP (e.g., Keycloak).

3.5 Parameter Management (PaM)

The Parameter Management (PaM) component provides a User Interface and services for managing Parameter tables, which consist of rows of data with criteria and result fields. These tables can be dynamically retrieved using Decision's built-in lookup function at test and execution time.

Important Note:

Whenever PaM is separately licensed, there will usually be two PaM component instances installed:

- A dedicated instance for test executions of assets within DM during their modeling or test phases. It has the following configuration property value:
 - `pam.target.env.type=DEVELOPMENT`
- One instance where approved assets are deployed in DE to provide decision services to calling applications.

Dependencies:

- **DM** – mandatory for startup and any operations within PaM because PaM requires access to DM's community-based permissions to enforce community parameter table authorizations.
- A database schema (PostgreSQL, Oracle, or MS-SQL) with all permissions.

Please refer to *Parameter Management Installation and Configuration Guide* for actual steps for installation of application within both environments.

3.6 Health Service

The Health Service provides a single endpoint to monitor the overall health status of the system, based on the aggregate health status of all components of the application.

This component is especially useful in **all-in-one deployment**, where a Load Balancer requires a single endpoint to determine whether the system is ready to receive traffic.

Note: The Health Service is usually not used in **containerized deployment**, which already has built-in health checks for the applications.

3.7 Configuration Service

The Configuration Service allows the dynamic updating of other component configurations without needing to restart the component being updated. It is applicable only when the components are installed as **standalone** or **containerized deployments**.

It is particularly useful for changing log level configurations or enabling/disabling features.

Note: Except for logging and feature flags, not every property can be changed dynamically without restarting the component. Only a subset of properties are supported.

The Configuration Service should be deployed alongside **RabbitMQ** to notify other components of configuration changes.

Dependencies:

- **RabbitMQ** – mandatory for communicating configuration changes.

4. Decision Deployment & Installation

4.1 Overview

Decision supports multiple deployment styles, each with a specific package provided. Although these packages are logically identical, they differ in layout, installation, configuration, platform dependencies, and startup mechanisms/commands.

4.2 Database Implementation

The following implementation is for unencrypted access. It is possible to increase the database's maximum pool connections.

For encrypted access details and Database pool settings, refer to the Decision Manager (DM) Configuration Guide.

4.2.1 Database – DM

Oracle

1. Restore DM schema from the Azure storage location `Decision-Oracle-DB-7.9.zip`, the file `DECISION.DMP`.

The original tablespace is `USERS`.

Oracle database users should have permissions only for their respective schemas.

2. Copy the `DECISION.DMP` file to the data dump directory.
3. Execute `impdp` command like the following:

```
impdp system/{sys_dba_pass} remap_schema=decision:{target_schema_name} remap_
tablespace=USERS:{target_table_space} directory={dp_directory}
dumpfile=DECISION.DMP
```

where:

`{sys_dba_pass}` is the database system user password.

`{target_schema_name}` is the target schema name.

`{target_table_space}` is the target tablespace name.

`{dp_directory}` is the data dump directory containing `DECISION.DMP`.

Note: After you execute `impdp` command, set the schema password.

PostgreSQL

1. Restore DM schema from the Azure storage location `Decision-Postgres-DB-7.9.tar`. The schema name is `bdms`.
2. Restore the command in the following manner:

```
pg_restore --username={pg user} --host={target_server} --dbname={target_db} --
verbose --no-owner --no-data-for-failed-tables --role={role_name} --format=tar {tar file
name}
```

4.2.2 Database – DT

Create an empty DT schema in the **Oracle** or **PostgreSQL** database – the schema will be auto created in DT's first startup

4.2.3 Database – DE

Create an empty DE schema in the **Oracle**, **PostgreSQL** or **MS-SQL** database – the schema will be auto created in DE's first startup.

4.3 Containerized Image Deployment

Containerized image deployment is a method where application components are packaged as Docker images, providing flexibility, scalability, and ease of management across various containerized platforms.

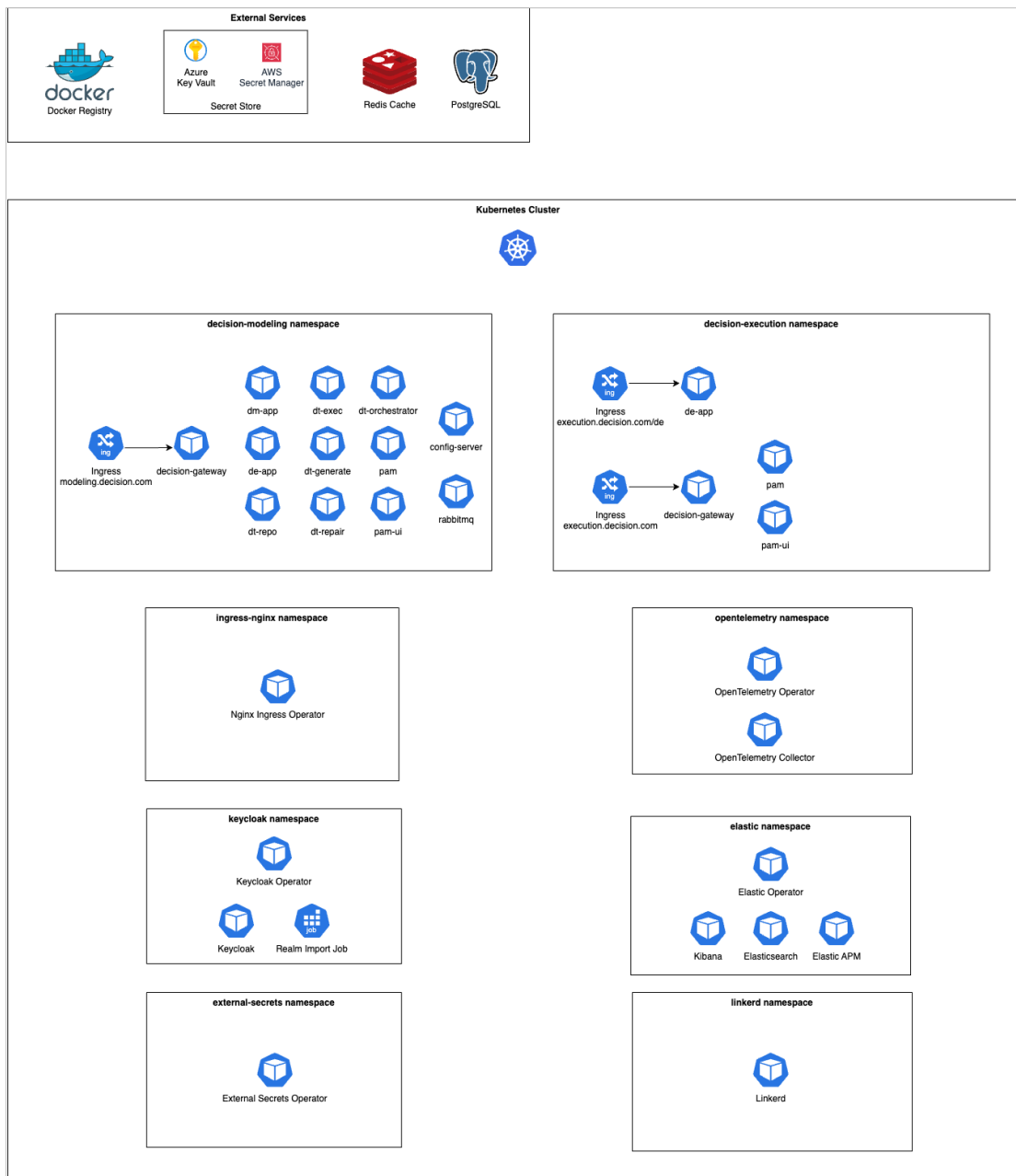
4.3.1 Overview

The containerized deployment is an advanced form of standalone deployment, where Sapiens provides the components as JAR files packaged within Docker images. These images can be deployed in various containerized environments, including Kubernetes, OpenShift, AKS, and EKS.

For detailed deployment instructions, refer to the Helm chart's README.md file, as explained in the [Accessing the README.md File for Deployment Instructions](#) section.

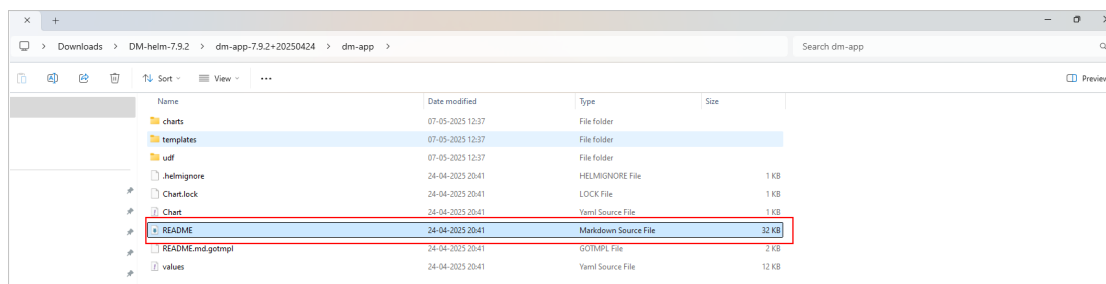
Advantages of Containerized Deployment:

- **Pre-Packaged Application JARs:** Application JARs are bundled within Docker images, simplifying deployment.
- **Isolated Failure Points:** A failure in one application component does not affect others, except where dependencies exist.
- **Flexibility and Scalability:** Components can be independently scaled and managed.
- **Fast Startup:** Each application component can be rapidly started in isolation.
- **Resource Optimization:** Independent memory and resource allocation for each component, tailored to its requirements.
- **Enhanced Monitoring:** Each application runs as an independent Java process, enabling granular monitoring.
- **Fine-Grained Scaling:** Individual components can be scaled as needed.



4.3.1.1 Accessing the README.md File for Deployment Instructions

- Locate the downloaded package file.
- Extract the package using any archive tool (e.g., WinRAR, 7-Zip).
- Navigate to the extracted folder and open the **dm-app** directory.
- The **README.md** file should be located in the root of this extracted folder.



Helm chart's README.md file

4.3.2 Package Content

This section lists the essential components and resources included in the containerized deployment package, ensuring all necessary files and images are available.

- **Docker Images** and files for each of the components of the application.
- **Helm Charts:** Configuration files to manage the deployment of these Docker images in Kubernetes environments.

4.3.3 Recommended Use Cases

This section provides scenarios where containerized deployment is most beneficial, helping users decide when to opt for this model.

Containerized deployment is recommended in the following scenarios:

- Deployment to a containerized environment (e.g., Kubernetes, OpenShift, AKS, EKS) is required.
- Preference for pre-packaged Docker images for simplified deployment.
- Requirement for per-application scalability, resource allocation, monitoring, and isolation.
- Environments requiring fast recovery and startup of each application component.
- High-availability production environments with a large number of users and/or high request rates.

4.3.4 Prerequisites

This section outlines the system and platform requirements necessary for a successful containerized deployment.

Ensure that the following platform dependencies are met before initiating containerized deployment:

- **Compatible Containerized Environment:** Kubernetes, OpenShift, AKS, or EKS.
- **DM:**
 - Docker Image
 - Database schema with all read/write permissions on any of the schema objects (tables, sequences, etc.). Refer to DM installation guide for database specific instructions.

- **DE:**
 - Docker Image
 - Database schema with all read/write permissions on any of the schema objects (tables, sequences, etc.). Refer to DE installation guide for database specific instructions.
- **DT Repository:**
 - Docker Image
 - Database schema with all read/write permissions on any of the schema objects (tables, sequences, etc.). Refer to DT installation guide for database specific instruction
- **DT Orchestrator:**
 - Docker Image
- **DT Executor:**
 - Docker Image
- **DT Generate:**
 - Docker Image
- **DT Repair:**
 - Docker Image
- **PaM:**
 - Docker Image
 - Database schema with all read/write permissions on any of the schema objects (tables, sequences, etc.). Refer to PaM installation guide for database specific instructions.
- **PaM UI:**
 - Docker Image
- **Decision Gateway:**
 - Docker Image
 - Redis Database.
- **Configuration Service (Optional):**
 - Docker Image for RabbitMQ.
 - RabbitMQ:
 - **Docker Image for RabbitMQ:** This is a pre-built, standardized package that allows you to run RabbitMQ easily using Docker.
 - **bitnami/rabbitmq:4.1.0-debian-12-r0:** This indicates a specific version of the RabbitMQ Docker image, with the "management" plugin enabled. The management plugin provides a user-friendly web-based management UI for RabbitMQ.
- **OAuth IDP (e.g., Keycloak):** Required for user authentication using OAuth 2.0.

- **Ingress NGINX Controller:** For external access management.
- **APM Tool (Optional):** Application Performance Monitoring (e.g., Elastic ECK).
- **External Secrets Operator (Optional)**
- **OpenTelemetry Configuration (Optional):**

OpenTelemetry is an open-source observability framework for generating, collecting, processing, and exporting telemetry data (like traces, metrics, and logs) from applications.

Components of OpenTelemetry Configuration:

- OpenTelemetry Operator
- OpenTelemetry Collector
- AutoInstrumentation Configuration

4.3.5 Installation

For detailed installation instructions, refer to the Helm chart's README.md file, as explained in the [Accessing the README.md File for Deployment Instructions](#) section.

4.4 “All-in-One” Deployment

The all-in-one deployment model is a traditional approach where all Decision components are deployed on a single host machine, operating under one application server (such as Tomcat). This approach simplifies management but has limitations in scalability and resource optimization.

4.4.1 Overview

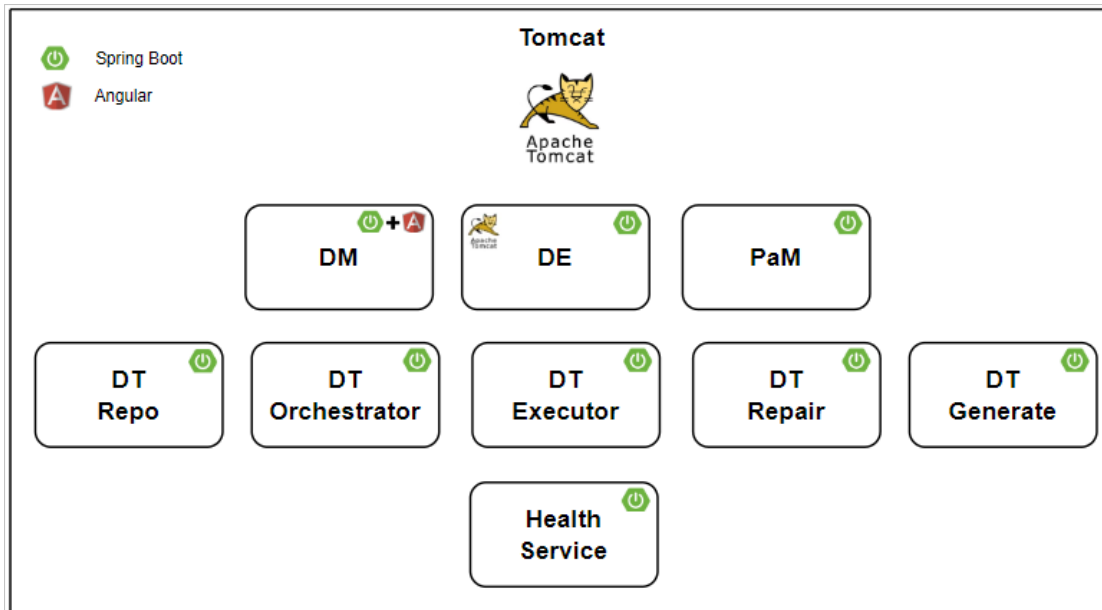
The term 'all-in-one' describes a deployment model where all Decision components are packaged and deployed together on a single host under one application server. This model is straightforward but has several advantages and disadvantages.

Advantages:

- A single configuration and maintenance point for all Decision components.
- No network delay between components since they run on the same host.

Disadvantages:

- Single point of failure – an issue with one component can impact the entire application.
- No fine-grained resource allocation for individual components.
- Larger application archive and memory footprint.
- Longer scaling time, as all components must be scaled together.



All-in-One deployment Architecture

4.4.2 Package Content

This section details the components included in the all-in-one deployment package.

- **Web Application Archives (WAR):** Application components are packaged as WAR files, which are deployed under a single application server (e.g., Tomcat).

4.4.3 Recommended Use Cases

The all-in-one deployment method is recommended in the following scenarios:

- The customer prefers to host the Decision application on an existing web application server.
- The convenience of a single application server configuration, a centralized deployment location, and a single port is prioritized over per-application instance scalability, resource allocation, monitoring, and isolation.

4.4.4 Prerequisites

This section outlines the essential system and platform requirements for deploying Decision using the all-in-one deployment model. Ensure all the following prerequisites are met before starting the installation.

Decision Manager (DM) Server

Note: Java JDK is not part of the application package. You must manually install the JDK recommended version mentioned in the table.

Software Requirements	
Java Application Server	Certified with Tomcat 10.1.42

Software Requirements	
Database	DM, DT, and DE schemas on either: Oracle - 19c or PostgreSQL DB – 15.7 ◦ Each Decision application must have a dedicated schema with full read/write permissions on all schema objects (tables, sequences, etc.). ◦ Multiple schemas can exist under the same database server instance, but each must be isolated per application.
Operating System (OS) with Java support	Certified with Red Hat Enterprise Linux 8 and 9.2, AWS Linux 2
Java JDK	Version 17.x Certified distributions: • Oracle • Amazon-Corretto • Red Hat OpenJDK The JDK installation path should be set in the JAVA_HOME environment variable, either by the operating system or directly within the Tomcat application server.
Messaging Queue Servers	RabbitMQ - 4.0.7 and above

Hardware Requirements	
For running the DM and DT machine (Also DT's DM internal DE instance which is optional), Tomcat application server and for DE installation and requirements, refer to the <i>Decision Execution (DE) Installation and Configuration Guide</i>.	
Hardware	Recommended Storage
CPU	4 CPU with 2.5 GHz or above *
Java heap size	3 GB minimum (recommended 8GB)
Hard disk	2 GB (for installation) 4 GB (for Database, or above depending on expected data capacity)
Note: These requirements could be reduced per machine when configured to work in clustering mode with more than one server machine	

Decision Web Client

To run the DM Web client, you require the following:

Software	Recommended version
Internet Browser	Google Chrome, Version 120.0.6099.217 (64-bit)
PC Hardware	1.6 GHz or higher processor and 1 GB of RAM
Network Bandwidth	Download speed: 5 Mb/sec Upload speed: 2.5 Mb/sec

Application Package (WAR Files):

The following web application archive (WAR) files are required for a complete all-in-one deployment:

Note: The placeholder `<version>` denotes the specific version number of the WAR file used in the release (e.g., 7.9.2, 1.10.2, etc.). Each component may have a different version based on its independent release cycle.

- **Mandatory Components:**
 - `dm-<version>.war` - Decision Manager (DM) Application.
- **Mandatory for Testing and Execution:**
 - `bdes-<version>.war` - Decision Execution (DE) Application.
 - `dt-repo-<version>.war` - DT Repository.
 - `dt-orchestrator-<version>.war` - DT Orchestrator.
 - `dt-exec-<version>.war` - DT Executor.
 - `dt-generate-<version>.war` - DT Generate.
 - `dt-repair-<version>.war` - DT Repair.
- **Mandatory for Parameter Management (PaM) and Lookup Functionality:**
 - `pam-<version>.war` - Parameter Management Application.
 - `pam-ui-<version>.war` - PaM User Interface.
- **Optional Components (Health Monitoring):**
 - `health-service-<version>.war` - Health Service Application.

4.4.5 Installation

This section provides a detailed, step-by-step guide for installing the Decision components in an all-in-one deployment on a Tomcat application server. Ensure you complete all prerequisite steps before proceeding with the installation.

Step 1: Set Up Global Class-Path and Configuration Folders

1. Edit the Tomcat Global Configuration:

- Locate and open the `catalina.properties` file in the `<Tomcat folder>/conf` directory.
- Set the `shared.loader` property to include the path to the Decision configuration

directory.:

- `shared.loader=<Decision home path>/config`

2. Create Configuration Directory:

- If it does not already exist, create a new directory named `config` under your `<Decision home path>`

3. Create Configuration Sub-Directories for Each Application:

- Inside the decision-config directory, create the following sub-directories for each Decision application:
 - `dm-config`
 - `de-config`
 - `dt-exec-config`
 - `dt-generate-config`
 - `dt-orchestrator-config`
 - `dt-repair-config`
 - `dt-repo-config`
 - PM (for PaM)
 - `health-service-config`

4. Add Configuration Files:

In each sub-directory, create a configuration file named either `application.properties` or `application.yml` or `application.yaml`, based on your preferred format.

These files will be used to define the properties of each application, as detailed in the Configuration chapter.

Step 2: Configure Application Contexts in Tomcat

1. Edit the Tomcat's server.xml File:

- Locate and open the `server.xml` file in the `<Tomcat folder>/conf` directory.
- Within the `<Host>` tag, add `<Context>` entries for each Decision application, as shown below:

```

<Host
  name="localhost" appBase="webapps" unpackWARs="true"
  autoDeploy="true" startStopThreads="10">
    <Context docBase="dt-repo" path="/dt-repo" />
    <Context docBase="dt-orchestrator" path="/dt-orchestrator"
  />
    <Context docBase="dt-exec" path="/dt-exec" />
    <Context docBase="dt-generate" path="/dt-generate" />
    <Context docBase="dt-repair" path="/dt-repair" />
    <Context docBase="dm" path="/dm" />
    <Context docBase="bdes" path="/bdes" />
    <Context docBase="pam" path="/pam" />
    <Context docBase="pam-ui" path="/pam-ui" />
    <Context docBase="health-service" path="/health-service"
  />
</Host>

```

2. Adjust Start-Stop Threads:

- Set the `startStopThreads` attribute based on the number of `<Context>` entries.
- The recommended value is the total number of applications you are deploying.

Step 3: Configure Tomcat HTTP and Shutdown Ports

1. Set HTTP Port:

- In the `<Connector>` tag, set the HTTP port (default is 8080):

```

<Connector
  port="9080"
  protocol="HTTP/1.1"
  connectionTimeout="20000"
  redirectPort="8443"
  maxParameterCount="1000"
/>

```

2. Set Shutdown Port:

- Change the shutdown port in the `<Server>` tag (default is 8005) to avoid conflicts:

```

<Server port="9005" shutdown="SHUTDOWN">

```

Step 4: Create and Configure Tomcat setenv.sh Script

1. Create the setenv.sh Script:

- If not already present, create a `setenv.sh` file in the `<Tomcat folder>/bin` directory.

2. Add Environment Variables and Java Options:

- Add the following content to the `setenv.sh` file, replacing `<Decision home path>` with the actual path where Decision is installed.:

```
#!/bin/sh

export JAVA_HOME="/home/.jdk/corretto-17.0.12"

export JAVA_OPTS="\
  -Xmx8g \
  -Ddecision.home=<Decision home path> \
  -Ddecision.de.home=<Decision home path> \
  -Dsun.io.useCanonCaches=true \
  -XX:+UseParallelGC \
  -Dfile.encoding=UTF-8 \
  -Dcom.sun.net.ssl.enableEC=true \
  --add-opens=java.base/sun.reflect.annotation=ALL-UNNAMED \
  --add-modules=java.se \
  --add-exports=java.base/jdk.internal.ref=ALL-UNNAMED \
  --add-opens=java.base/sun.nio.ch=ALL-UNNAMED \
  --add-opens=java.base/java.lang=ALL-UNNAMED \
  --add-opens=java.management/sun.management=ALL-UNNAMED \
  --add-
  opens=jdk.management/com.sun.management.internal=ALL-UNNAMED \
  --add-opens=java.base/java.util=ALL-UNNAMED"
```

3. Edit Configuration:

- Set the `JAVA_HOME` variable to your Java 17 JDK installation path.
- Adjust the `-Xmx8g` value (maximum memory) based on your system specifications.

Step 5: Prepare Database Schemas

- Ensure that the database schemas for each Decision application (DM, DE, DT, PaM, and Health Service) are ready.
- These schemas must have the necessary permissions (read/write) as specified in the Prerequisites section.

Step 6: Deploy Application WAR Files

1. Stop Any Existing Decision Applications:

- If any Decision applications are running in the target Tomcat server, stop the server.

2. Clean Up Existing Files:

- Delete any existing WAR files or folders of a Decision application you are about to redeploy.

3. Copy WAR Files:

- Copy the Decision application WAR files to the `<Tomcat folder>/webapps` directory.

4. Rename WAR Files:

- Remove the version numbers from the WAR filenames:

Note: The placeholder `<version>` denotes the specific version number of the WAR file used in the release (e.g., 7.9.2, 1.10.2, etc.). Each component may have a different version based on its independent release cycle.

- `dm-<version>.war` → `dm.war`
- `de-<version>.war` → `bdes.war`
- `dt-repo-<version>.war` → `dt-repo.war`
- `dt-orchestrator-<version>.war` → `dt-orchestrator.war`
- `dt-exec-<version>.war` → `dt-exec.war`
- `dt-generate-<version>.war` → `dt-generate.war`
- `dt-repair-<version>.war` → `dt-repair.war`
- `pam-<version>.war` → `pam.war`
- `pam-ui-<version>.war` → `pam-ui.war`
- `health-service-<version>.war` → `health-service.war`

5. Clean Up Existing Files:

- Delete any existing WAR files or folders of a Decision application you are about to redeploy.

Step 7: Configure Application Properties

- Follow the Configuration chapter instructions to set the required properties for each application under the "all-in-one" section.
- Populate the configuration files (`application.properties` or `application.yml`) created in Step 1 with these properties.

4.5 Standalone Deployment

4.5.1 Overview

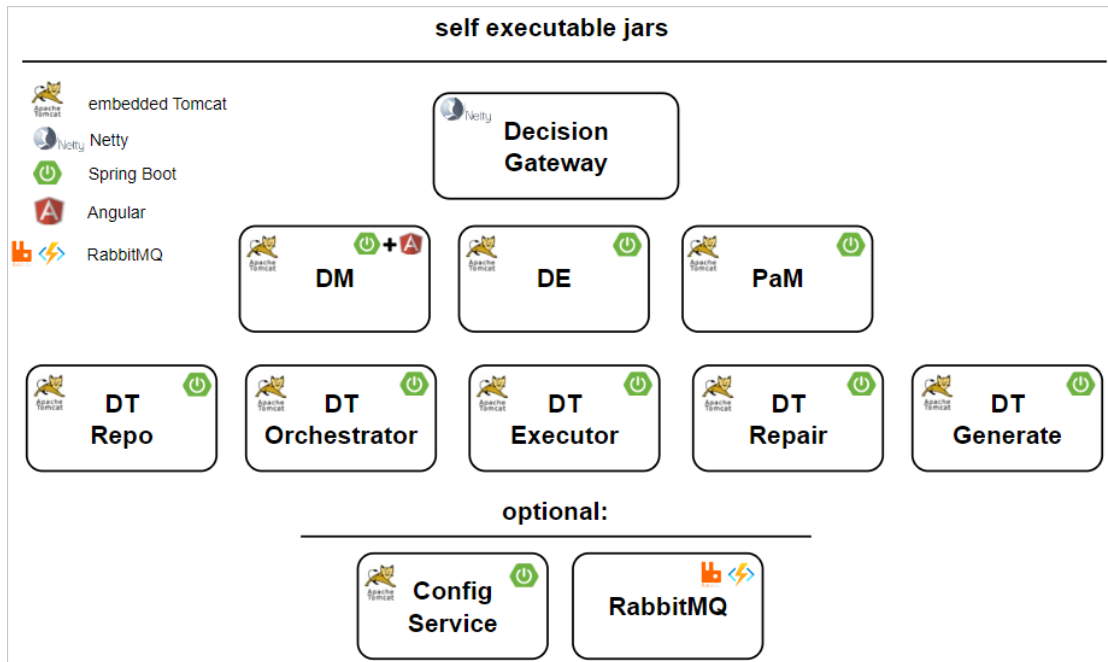
In standalone deployment, each Decision application is packaged as a self-executable JAR file generated using the Spring Boot framework. These JAR files contain an embedded Tomcat server, making them completely independent and self-sufficient.

- **Independent Execution:** Each application runs on its own port and can be hosted on the same or different machines.
- **Application-Specific Configuration:** Server-related properties (e.g., port, SSL) are managed directly in the application's properties file (`application.properties` or `application.yml`) or via Java command-line arguments.
- **Improved Flexibility and Scalability:** This model allows fine-grained control over resource allocation, monitoring, and scaling for each application.

Advantages:

- Distributed points of failure — Failure of one component does not affect others.
- Maximum flexibility, scalability, and isolation between application components.

- Fast startup due to isolated application instances.
- Independent memory and resource allocation for each component.
- Per-application monitoring using independent Java processes.
- Fine-grained scaling for each application.



Standalone Deployment Architecture

4.5.2 Package Content

The package for standalone deployment includes the following components:

- **Application Java Binary Files:** Compiled Java class files for each Decision application.
- **Spring Boot Framework Files:** Essential Spring Boot classes for server startup and management.
- **Java Third-Party Libraries:** JAR files of external libraries used by the applications.
- **Maven Build Metadata:** Details of build configuration and dependencies.

4.5.3 Recommended Use Cases

Standalone deployment is ideal in the following scenarios:

- When you prefer to build your own container images using the application JAR files.
- When fine-grained scalability, resource allocation, monitoring, and isolation for each application are required.
- When fast recovery and startup for each application is essential.

Common Use Cases:

- Production environments with a large number of users.
- High-traffic scenarios with frequent server requests.

4.5.4 Prerequisites

The standalone deployment of Decision requires specific platform dependencies for each component to ensure optimal performance and smooth operation. These prerequisites include Java installation, available disk space, memory allocation, database setup, and connectivity between components.

Java Installation (Global Requirement)

- **JDK Version:** Installed JDK distribution of Java 17.
- **JAVA_HOME Configuration:** Ensure the Java installation path is set in the `JAVA_HOME` environment variable, either at the OS level or through the application's Java command line.
- **Memory Allocation (Xmx):** The memory allocation size for each application is determined by the expected user load and concurrent request count. It can be set via Java command line arguments, for example:

```
java -Xmx1g -jar <app_jar_file_path>
```

- **Disk Space (Minimum Requirements):** Each application has specific disk space requirements listed below.

Component-Specific Prerequisites

Component	Disk Space	Memory (Xmx)	Additional Dependencies
DM	260 MB	Minimum: 3 GB Recommended: 5 GB	• Dedicated database schema (PostgreSQL/Oracle). • Connectivity to DT Orchestrator if installed on a different machine.
DE	185 MB	Minimum: 1.5 GB Recommended: 3 GB	• Dedicated database schema (PostgreSQL/Oracle). • Adjust memory based on DE asset artifacts cached.
DT Repository	115 MB	Minimum: 1.85 GB Recommended: 3 GB	• Dedicated database schema (PostgreSQL/Oracle). • Connectivity to DM if installed on a different machine.
DT Orchestrator	105 MB	Minimum: 1.2 GB Recommended: 2 GB	• Connectivity to other DT applications. • Connectivity to DM if installed on a different machine.

Component	Disk Space	Memory (Xmx)	Additional Dependencies
DT Executor	105 MB	Minimum: 1.5 GB Recommended: 2.5 GB	- Connectivity to DT Repository and DE. - Adjust memory for high-volume test execution.
DT Generate	105 MB	Minimum: 1.2 GB Recommended: 2 GB	Connectivity to DT Repository.
DT Repair	105 MB	Minimum: 1 GB Recommended: 1.5 GB	Connectivity to DT Repository.
PaM	120 MB	Minimum: 400 MB Recommended: 800 MB	Dedicated database schema (PostgreSQL/Oracle).
Decision Gateway (OAuth)	75 MB	Minimum: 500 MB Recommended: 900 MB	- Redis Database. - OAuth IDP (e.g., Keycloak). - Connectivity to other Decision applications.
Configuration Service (Optional)	75 MB	Minimum: 425 MB Recommended: 800 MB	RabbitMQ (version 4.0.7 and above).

4.5.5 Installation

To install any Decision application in standalone mode:

1. Database Schema Setup

- If the application requires a database schema, ensure it is created.
- If the schema does not exist, create it before proceeding.

2. Application Jar Placement

- Copy the self-executable JAR file of the application to your desired application home folder.

3. Configuration File Setup

- Create or copy an `application.properties` or `application.yml` file.
- Ensure this configuration file is in the same folder as the application JAR file.

4. Configuration Settings

- Refer to the **Configuration chapter**.
- Populate the configuration file with the required properties under the **Standalone sections**.

5. Configuration

To configure the installed Decision application(s):

- Follow each of the sections below and the specific configuration section for each installed application.
- Ensure all configuration sections are properly set for the application to start up and function correctly:
 - Basic Configuration
 - Relevant Authentication Type Configuration
 - Integration with Other Components (if applicable)

Configuration Format:

- Configuration can be set in either:
 - **YAML Format** (application.yml): Suitable for applications with extensive configuration properties (e.g., DM, DT).
 - **Key-Value Format** (application.properties): Suitable for applications with simpler configurations.
- **User-Specific Values:**
 - Any values that need user input are marked with <...>.

5.1 Configuration location

5.1.1 Containerized Deployment

- Configuration properties should be set under each component section in the Enterprise Chart Values under `configMapData`.
- For detailed deployment instructions, refer to the Helm chart's README.md file, as explained in the [Accessing the README.md File for Deployment Instructions](#) section.

5.1.2 All-in-One Deployment

- Configuration properties should be placed in the `application.properties` or `application.yml` file.
- These files should be located under each Decision component application's config folder within the `<Decision home path>/config` directory.
- Refer to [All-in-One Installation](#) section for more details.

5.1.3 Standalone Deployment

- Configuration properties should be set in an `application.properties` or `application.yml` file.
- This file must be in the same folder where the Decision component application JAR file is located.

5.2 Property Value encryption

5.2.1 Overview

Decision applications allow secure storage of sensitive configuration values (such as database passwords, API keys, and credentials) by supporting encrypted properties in the configuration files (application.yml or application.properties). Encrypted property values enhance security by ensuring that sensitive information is protected even if the configuration files are exposed.

5.2.2 How Property Encryption Works

- Properties marked as encrypted use the prefix `ENC:` to indicate they are securely encrypted.
- The encrypted value must be generated using the **Decision Encryption Tool**, which applies an RSA encryption algorithm.
- During runtime, the Decision application automatically decrypts these properties using the corresponding decryption key.

Example:

```
# Encrypted database password using Jasypt or similar encryption
library
spring.datasource.password=ENC:lmclyieQY87EJD.....
```

5.2.3 Encryption Tool Usage

The **Decision Encryption Tool** (decision-encrypt.2.1.jar) is a command-line utility that helps you generate encrypted values using RSA encryption. The tool (decision-encrypt.2.1.jar) is bundled in the main root level of the release package alongside the deployment artifacts for a given version release.

Supported Algorithms:

- RSA (Recommended)
- BCRYPT

5.2.4 Generating Encrypted Property Values

1. Step 1: Ensure Java is Installed

- Make sure JDK 17 is installed and configured on your system.
- Set the `JAVA_HOME` environment variable if not already set:

```
export JAVA_HOME="/path/to/jdk-17"
```

2. Step 2: Obtain the Encryption Tool

- Ensure that the `decision-encrypt-2.1.jar` file is available on your system.

3. Step 3: Run the Encryption Tool

- Use the command-line interface (CMD on Windows, Terminal on Linux) to run the tool.

Windows Command Example:

```
java -jar decision-encrypt-2.1.jar RSA value-to-encrypt
```

Linux Command Example:

```
java -jar decision-encrypt-2.1.jar 'RSA' 'value-to-encrypt'
```

- Replace value-to-encrypt with the actual sensitive value you want to secure (e.g., a database password).

4. Step 4: Review the Tool Output

- The tool will output the following information:

```
Entered Text          - (The value you entered for encryption)
Encryption Public Key - (Only for reference)
Encrypted Text        - (The encrypted value to be used in your
configuration file)
```

- Copy the Encrypted Text value.

5. Step 5: Add Encrypted Value to Configuration

- Place the encrypted value in the configuration file (application.yml or application.properties) with the prefix ENC:.

Example for application.yml:

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/decision_dm
    username: dm_user
    password: ENC:lmclyieQY87EJD..... # Encrypted password
```

Example for application.properties:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/decision_dm
spring.datasource.username=dm_user
spring.datasource.password=ENC:lmclyieQY87EJD.....
```

6. Step 6: Verify Decryption During Application Startup

- Start the Decision application.
- Check the application logs to ensure there are no decryption errors.
- The application should automatically decrypt the value at runtime.

5.3 Authentication Types

Each Decision application component supports multiple authentication types, as many of them can function as independent applications. However, when combined into a single, integrated Decision environment, not all authentication type combinations are supported.

Refer to the "Authentication" section of each application configuration chapter for details on the available options, limitations, and configuration steps for authenticated integration between components.

Note: In "all-in-one" deployment, OAuth is supported in DM and DE only for Machine-to-Machine (M2M) authentication.

5.4 DM

5.4.1 Containerized Deployment

- **Configuration Location:**

Properties should be set under the `dm-app` section in the enterprise chart values under `configMapData`.

- **Reference:**

For detailed deployment instructions, refer to the Helm chart's README.md file, as explained in the [Accessing the README.md File for Deployment Instructions](#) section.

5.4.2 All-in-One Deployment

- **Configuration File Location:**

The properties file must be located at: `<Decision home path>/config/dm-config..`

- **Configuration Guidelines:**

Use the Tomcat port number specified in the `server.xml` file for all `<* port number>` placeholders. (Refer to [All-in-One Deployment Installation](#) section).

- Set the hostname for all `<* host>` placeholders (except `<db host>`) as `localhost`.

5.4.3 Standalone Deployment

- **Configuration File Location:** The properties file must be located in the same directory as the `dm-<version>.jar` file.

Note: The placeholder `<version>` denotes the specific version number of the JAR file used in the release.

5.4.4 Basic Configuration

In the DM properties file, configure the following basic properties. If a placeholder includes a default value, the property is optional and will default to the specified value if left unset. An `application.yml` file is shown below, but the same configurations can also be done in an `application.properties` file:

```
# Server Configuration
server:
  port: <DM port number, default is 9080>

# PostgreSQL Database Configuration
postgres:
  jdbc:
    url: jdbc:postgresql://<db host>:5432/<db name>
    driverClassName: org.postgresql.Driver
    username: <username>
    password: <password>
  hibernate:
    dialect:
      com.sapiens.decision.persistence.hibernate.postgres.PostgreSQLCrossVendorC
      ompatibleDialect
    enable:
      audit: false

# Oracle Database Configuration
oracle:
  jdbc:
    url: jdbc:oracle:thin:@<db host>:1521:<db name>
    driverClassName: oracle.jdbc.OracleDriver
    username: <username>
    password: <password>
  hibernate:
    dialect: org.hibernate.dialect.Oracle10gDialect
```

5.4.5 Authentication

Authentication in Decision Manager (DM) determines how users and applications are verified before accessing protected resources. Depending on the deployment and security requirements, DM supports multiple authentication methods, including the default username-password login, SAML, LDAP, and OAuth 2.0 for API authentication. This section provides configuration instructions for each supported authentication type, ensuring secure access management across the Decision environment.

Note: For advanced authentication configurations such as SAML, LDAP, or others not covered in this document, refer to the Advanced Configuration section.

Username-Password Login Form (default)

This is the default authentication setup when no specific authentication type is configured. It uses DM's internal authentication mechanism, presenting users with an HTTP login form for session-based security. User credentials are stored encrypted in DM's database, and user management (including password changes) is performed through the user administration section of the Decision client.

Configuration Instructions:

- **Configuration File:** DM's `application.yml`
- **Setup:**
 - Either delete any existing authentication type under the `spring.active.profiles` property to enable the default username-password login form, or explicitly set

the value to `user-password-internal-auth` (along with any other active profiles).

```
spring:
  profiles:
    active: user-password-internal-auth, other-profile-name,
    another-profile
```

- **Optional - OAuth 2.0 Authentication for API Requests:**

- This setup allows OAuth 2.0 authentication for API requests from other applications, separate from user login form authentication.

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: <IDP issuer URL>
          jwk-set-uri: <IDP jwk set-uri URL - Optional>
```

- **Note:** The `jwk-set-uri` is optional but recommended for enhanced security.

Header-Based Authentication

Header-based authentication allows users to authenticate via a specific HTTP header containing the username. This method does not involve any password or key authentication, making it suitable for environments where access is controlled by an external authentication mechanism. Typically, this setup is used in secure environments where an external authentication page verifies user credentials and then forwards the authenticated username to DM.

Configuration Instructions:

- In the DM `application.yml` file, set the authentication type to "header-authentication" under the `spring.active.profiles` property. Ensure no other authentication types are included in the list.

```
spring:
  profiles:
    active: header-authentication
```

OAuth 2.0 Authentication

OAuth 2.0 authentication allows users and external applications to authenticate with DM using an OAuth 2.0 Identity Provider (IDP). Depending on the setup, this can be used for user authentication via the Decision Client or for API authentication from other applications.

Configuration Instructions:

- To enable OAuth 2.0 authentication for users accessing DM (and other apps) from the Decision Client, ensure the Decision Gateway is included in the environment (required for standalone and containerized deployments).

- OAuth 2.0 can also be enabled alongside other authentication methods (like user-password login form, LDAP, or SAML) to support API authentication for external applications.
- To configure DM to accept only OAuth credentials (for users or applications), set the `spring.active.profiles` property to "oauth-only-authentication" in the `application.yml` file.

```
spring:
  profiles:
    active: oauth-only-authentication

  security:
    oauth2:
      resourceserver:
        jwt:
          audiences:
            - EnterpriseApp
            - https://dm
          issuer-uri: <IDP issuer URL>
          jwk-set-uri: <IDP jwk set-uri URL - Optional>

      client:
        provider:
          sapidp:
            token-uri: <IDP token URL>
            jwk-set-uri: <IDP jwk set-uri URL - Optional>
            user-name-attribute: <User name attribute in token - Optional>
            user-info-uri: <IDP user info URL - Optional>
```

- `issuer-uri`: The URL of the IDP issuer for JWT validation.
- `jwk-set-uri`: The URL of the JSON Web Key Set provided by the IDP for verifying tokens.
- `token-uri`: The URL for obtaining OAuth tokens from the IDP.
- `user-name-attribute`: The attribute in the token representing the username.
- `user-info-uri`: The URL for retrieving user information from the IDP.

SAML Single Sign-On Authentication

- Refer to [SAML Single sign-on \(SSO\) Authentication](#) section in the **Advanced Configuration Topics** for more information.

LDAP Authentication

- Refer to [Configuring LDAP for DM](#) section in the **Advanced Configuration Topics** for more information.

Integration with other Decision Components Configuration

This section outlines how to integrate DM with other Decision components, such as DE (Decision Execution) and DT (Decision Testing) applications. Depending on the authentication method configured for each component, the configuration properties may vary.

1. Integrating with DE (Decision Execution) Testing Instance

DM is responsible for deploying assets to the DE instance for execution tests. The integration requires specifying the DE instance details and the appropriate authentication method.

- **Configuring DE Instance Details:**

- Add or edit the `testing.de.url` property in the `application.yml` file as shown below. If not set, a default value of `http://localhost:8090/bdes` is used:

```
testing:
  de:
    url: http://<DE host>:<DE port>/bdes
```

- **Authentication Configuration:**

- If DE is configured as a secured instance, authentication credentials must be provided based on the DE authentication type:

Basic Authentication:

```
testing:
  de:
    username: <username, default is admin>
    password: <password, default is bdms>
```

Note: The specified user (`<username>`) must have at least the "DE_USER" role assigned.

OAuth Authentication:

```
spring:
  security:
    oauth2:
      client:
        provider:
          sapidp:
            token-uri: <IDP token URL>
            jwk-set-uri: <IDP jwk set-uri URL -
Optional>
            user-name-attribute: <User name attribute in
token - Optional>
            user-info-uri: <IDP user info URL -
Optional>
        registration:
          de:
            provider: sapidp
            authorization-grant-type: client_credentials
            scope:
              - DE_ADMIN
              - user-token-exchange
```

Note: If the OAuth Identity Provider (IDP) protecting DE requires an additional parameter (like "audience" for Auth0), the following configuration is required:

```
decision:
  security:
    oauth2:
      client:
        registration-extension:
          de:
            query-parameters:
              audience: https://de-for-testing
```

2. Integrating with DT (Decision Testing) Applications

If DM and DT are not using OAuth authentication (such as in all-in-one deployments), DM acts as an authentication proxy for all Decision Client requests to DT. In such cases, the following configuration is required to specify the details of each DT application instance.

- **Configuring DT Instances:**

```
decision:
  dt:
    repo:
      base-url: http://<DT Repository host>:<DT Repository
port number>/dt-repo
    orch:
      base-url: http://<DT Orchestrator host>:<DT Orchestrator
port number>/dt-orchestrator
    repair:
      base-url: http://<DT Repair host>:<DT Repair port
number>/dt-repair
    exec:
      base-url: http://<DT Executor host>:<DT Executor port
number>/dt-exec
    generate:
      base-url: http://<DT Generate host>:<DT Generate port
number>/dt-generate
```

- Ensure the correct URLs and port numbers are specified for each DT component. If not set these the following default values will be used:
 - decision.dt.repo.base-url=http://localhost:9080/dt-repo
 - decision.dt.orch.base-url=http://localhost:9080/dt-orchestrator
 - decision.dt.repair.base-url=http://localhost:9080/dt-repair
 - decision.dt.exec.base-url=http://localhost:9080/dt-exec
 - decision.dt.generate.base-url=http://localhost:9080/dt-generate

5.5 DE

This section provides detailed instructions for configuring the Decision Execution (DE) application in different deployment types, including All-in-One, Standalone, and Containerized deployments.

5.5.1 Containerized Deployment

- Configuration properties for DE must be set under the `de-app` section of the enterprise chart values (`configMapData`). For detailed instructions, refer to the chart README file.

5.5.2 All-in-One Deployment

- The DE configuration file must be located within the following directory: `<Decision home path>/config/de-config`
- The port number used for DE (`<DE port number>`) must match the port configured in Tomcat's `server.xml` file (refer to the Installation chapter for `server.xml` configuration).

5.5.3 Standalone Deployment

The DE configuration file must be in the same directory as the DE application JAR file (`de-<version>.jar`).

Note: The placeholder `<version>` denotes the specific version number of the JAR file used in the release.

5.5.4 Basic Configuration

- **Server Port Configuration:**

Define the server port for DE using the following property:

```
server.port=<DE port number, default is 8090>
```

- **Database Configuration:**

PostgreSQL:

```
spring.datasource.url=jdbc:postgresql://<db host>:5432/<db name>  
spring.datasource.username=<username>  
spring.datasource.password=<password>
```

Oracle:

```
spring.datasource.url=jdbc:oracle:thin:@<db host>:1521:<db name>  
spring.datasource.username=<username>  
spring.datasource.password=<password>
```

Microsoft SQL Server:

```
spring.datasource.url=jdbc:sqlserver://<db  
host>:1433;databaseName=<db name>  
spring.datasource.username=<username>  
spring.datasource.password=<password>
```

- **Non-Clustered Configuration:**

If DE is not part of a cluster, disable the repository monitor using:

```
decision.de.db.repository.monitor.disabled=true
```

5.5.5 Authentication

DE does not perform user authentication by default. To enable authentication, choose one of the following methods:

Basic Authentication

- User credentials (username, encrypted password, and roles) must be defined in a file named `security-users.properties`. This file should be placed:
 - In the DE class-path directory (alongside `de-<version>.jar` for standalone deployments).

Note: The placeholder `<version>` denotes the specific version number of the JAR file used in the release.

 - In `<Decision home path>/config` for all-in-one deployments.
- For instructions on encrypting and configuring user credentials, refer to the DE Installation and Configuration Guide.

Enabling Basic Authentication:

- Set the following values to the `spring.profiles.active` property in the `application.properties` file:

```
spring.profiles.active=secured,static,pojo,db-repository
```

Important:

The values `pojo` and `db-repository` must always be part of the active profiles list, regardless of the authentication type.

OAuth 2.0 Authentication

To configure DE for OAuth 2.0 authentication, use the following properties in the `application.properties` file:

```
spring.security.oauth2.resourceserver.jwt.audiences=https://de-for-testing
spring.security.oauth2.resourceserver.jwt.issuer-uri=<IDP issuer URL>
spring.security.oauth2.resourceserver.jwt.jwk-set-uri=<IDP jwk set-uri URL - Optional>
```

Note: The `jwk-set-uri` property is optional but recommended for enhanced security.

5.6 DT

This section provides the configuration details for the Decision Testing (DT) applications, including authentication methods.

Important Note:

If another DT application instance exists in a different environment (such as another UAT or DEV environment), you must set a unique cluster name for that environment in each of the five DT applications (DT Orchestrator, DT Repository, DT Executor, DT Generate, and DT Repair). Use the following property in the configuration files of all DT applications in that environment:

```
dt.hz.cluster.name=<unique name>
```

- `<unique name>` can be any text value, but it must be unique to that environment and consistent across all DT applications in that environment.

5.6.1 Authentication

DT applications support two primary authentication methods: Restricted Origin (default) and OAuth 2.0.

Restricted Origin (default)

- **Default Authentication Mode:**

This is the default setting for DT applications if OAuth is not configured. It is also the only authentication method available for **All-in-One** deployments.

- **Functionality:**

This method does not perform user authentication. Instead, it restricts access by rejecting any requests that do not originate from:

- Another Decision application host, or
- Any of the hosts/IPs specified in the `dt.additionally.allowed.access.csv.hosts` property.

- **Usage in All-in-One Deployment:**

- In an All-in-One environment, this method is generally sufficient because all requests to DT from the Decision Client must first pass through Decision Manager (DM), which handles user authentication.

- **Configuring Additional Allowed Hosts:**

If any machine (other than the Decision applications) needs access to DT services, you must whitelist it using the following configuration in the `application.yml` file of the DT application:

```
dt:
  additionally:
    allowed:
      access:
        csv:
          hosts: 123.54.2.3,some.host.com
```

The example above allows access to the DT application from:

- A machine with IP 123.54.2.3.
- A host named some.host.com.

OAuth 2.0 Authentication

To enable OAuth 2.0 authentication for DT applications:

- **Environment Requirements:**
 - The Decision Gateway must be part of the environment.
 - All DT applications in the environment must be configured for OAuth authentication.
 - The deployment type of the environment must be either **Standalone** or **Containerized**.
- **Configuration Instructions:**
 - Follow the OAuth configuration instructions in the relevant sections of each DT application (DT Orchestrator, DT Repository, DT Executor, DT Generate, and DT Repair).
 - Ensure that all DT applications share a consistent OAuth configuration for seamless authentication.

5.6.2 DT Repository

This section provides detailed configuration instructions for the DT Repository application across different deployment types (All-in-One, Standalone, and Containerized) and covers basic settings, OAuth 2.0 authentication, and integration with Decision Manager (DM).

Containerized Deployment

- **Configuration Location:**

The properties should be defined under the `dt-repo` section in the enterprise chart values under `configMapData`.

For detailed instructions, refer to the chart README file.

All-in-One Deployment

- **Configuration Location:**

The properties file must be placed at:

```
<Decision home path>/config/dt-repo-config
```
- **Configuration Guidelines:**
 - The port number for `<* port number>` placeholders must match the one set in the Tomcat server.xml (as explained in the Installation chapter).
 - The hostname for `<* host>` placeholders (except `<db host>`) should be set to `localhost`.

Standalone Deployment

- **Configuration Location:**

The properties file must be placed in the same directory where the `dt-repo-<version>.jar` file is located.

Note: The placeholder `<version>` denotes the specific version number of the JAR file used in the release.

Basic Configurations

- **Server Configuration:**

```
server:
  port: <DT Repository port number - default is 8082>

testing:
  repo:
    rsocket:
      server:
        port: <DT Repository RSocket port number - default is
7001>
```

- **Database Configuration:**

PostgreSQL Configuration:

```
repo:
  datasource:
    vendor: postgres
    host: <DT Repository db host>
    port: 5432
    database: <DT Repository db name>
    user: <DT Repository db username>
    password: <DT Repository db password>
```

Oracle Configuration:

```
repo:
  datasource:
    vendor: oracle
    host: <DT Repository db host>
    port: 1521
    database: <DT Repository db name>
    user: <DT Repository db username>
    password: <DT Repository db password>
```

- **RSocket Authentication:**

Not required when using OAuth, but required and used automatically when not using OAuth in DT. Note that the username and password are already preset with default values, but they should be changed for better security.

```
testing:
  repo:
    rsocket:
      user: <RSocket username>
      password: <RSocket password>
```

- `testing.repo.rsocket.user`

The username used for authentication for the RSocket communication.

- `testing.repo.rsocket.password`

The password used for authentication for the RSocket communication. The password value can be in plain text or encrypted using the `ENC:` prefix. Refer to "Properties value encryption" for more details.

OAuth 2.0 Authentication

For standalone and containerized deployments, the DT Repository can be configured with OAuth 2.0 authentication using the following configuration in the `application.yml`:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          audiences:
            - EnterpriseApp
            - https://dt-repo
          issuer-uri: <IDP issuer URL>
          jwk-set-uri: <IDP jwk set-uri URL - Optional>
```

Note:

- The `jwk-set-uri` is not mandatory, but it is recommended.
- Setting this will prevent server startup failure if the IDP is unavailable during startup.

Integration with Decision Manager (DM)

- **Purpose:** DT Repository interacts with DM for specific operations.
- **Configuration:**

To integrate with DM, add the following properties in the `application.yml`:

```
dm:
  port: <DM port number - 9080 by default>
  host: <DM host - localhost by default>
```

5.6.3 DT Orchestrator Configuration

This section provides detailed configuration instructions for the DT Orchestrator application across different deployment types (All-in-One, Standalone, and Containerized). It covers basic settings, OAuth 2.0 authentication, and integration with other Decision components.

Containerized Deployment

- **Configuration Location:**

The properties should be defined under the `dt-orchestrator` section in the enterprise chart values under `configMapData`.

For detailed instructions, refer to the chart README file.

All-in-One Deployment

- **Configuration Location:**

The properties file must be placed at:

`<Decision home path>/config/dt-orchestrator-config`

- **Configuration Guidelines:**

- The port number for `<* port number>` placeholders must match the one set in the Tomcat `server.xml` (as explained in the Installation chapter).
- The hostname for `<* host>` placeholders (except `<db host>`) should be set to `localhost`.

Standalone Deployment

- **Configuration Location:**

The properties file must be placed in the same directory where the `dt-orchestrator-<version>.jar` file is located.

Note: The placeholder `<version>` denotes the specific version number of the JAR file used in the release.

Basic Configurations

- **Server Configuration:**

```
server:
  port: <DT Orchestrator port number, default is 8085>
```

OAuth 2.0 Authentication

To set DT Orchestrator with OAuth 2.0 authentication, include the following in the `application.yml`:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: <IDP issuer URL>
          jwk-set-uri: <IDP jwk set-uri URL - Optional>
```

Note:

- The jwk-set-uri is not mandatory, but it is recommended.
- Setting this will prevent server startup failure if the IDP is unavailable during startup.

Integration with other Decision Components

- **Purpose:**

DT Orchestrator interacts with the following Decision components:

- Decision Manager (DM)
- DT Repair
- DT Repository
- DT Generate
- DT Executor

- **Configuration:**

Define the details of these components in the `application.yml`:

```
dm:
  port: <DM port number - 9080 by default>
  host: <DM host - localhost by default>

testing:
  repair:
    port: <DT Repair port number - 8081 by default>
    host: <DT Repair host - localhost by default>
  repo:
    port: <DT Repository port number - 8082 by default>
    host: <DT Repository host - localhost by default>
  generate:
    port: <DT Generate port number - 8083 by default>
    host: <DT Generate host - localhost by default>
  exec:
    port: <DT Executor port number - 8084 by default>
    host: <DT Executor host - localhost by default>
```

- **OAuth 2.0 Authentication for Components:**

In standalone and containerized deployments, if these components are protected by OAuth, you must add authentication credentials for each of them.

```

spring:
  security:
    oauth2:
      client:
        provider:
          sapidp:
            token-uri: <IDP issuer token URL>
            client-id: <DT client id defined in IDP>
            client-secret: <DT client secret defined in IDP>
        registration:
          dt-repo:
            provider: sapidp
            authorization-grant-type: client_credentials
            scope:
              - user-token-exchange

```

- **Additional Parameter for IDP (Optional):**

If the IDP requires an extra parameter (like audience in Auth0), add the following configuration:

```

decision:
  security:
    oauth2:
      client:
        registration-extension:
          dt-repo:
            query-parameters:
              audience: https://dt-repo

```

5.6.4 DT Executor

This section provides detailed configuration instructions for the DT Executor application across different deployment types (All-in-One, Standalone, and Containerized). It covers basic settings, OAuth 2.0 authentication, and integration with other Decision components.

Containerized Deployment

- **Configuration Location:**

The properties should be defined under the `dt-exec` section in the enterprise chart values under `configMapData`.

For detailed instructions, refer to the chart README file.

All-in-One Deployment

- **Configuration Location:**

The properties file must be placed at:

`<Decision home path>/config/dt-executor-config`

- **Configuration Guidelines:**

- The port number for `<* port number>` placeholders must match the one set in the Tomcat `server.xml` (as explained in the Installation chapter).

- The hostname for `<* host>` placeholders (except `<db host>`) should be set to `localhost`.

Standalone Deployment

- **Configuration Location:**

The properties file must be placed in the same directory where the `dt-executor-<version>.jar` file is located.

Note: The placeholder `<version>` denotes the specific version number of the JAR file used in the release.

Basic Configurations

- **Server Configuration:**

```
server:
  port: <DT Executor port number, default is 8084>

testing:
  repo:
    rsocket:
      server:
        port: <DT Repository RSocket port number - default is
7001>
```

OAuth 2.0 Authentication

To set DT Executor with OAuth 2.0 authentication, include the following in the `application.yml`:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          audiences: EnterpriseApp
          issuer-uri: <IDP issuer URL>
          jwk-set-uri: <IDP jwk set-uri URL - Optional>
```

Note:

- The `jwk-set-uri` is not mandatory, but it is recommended.
- Setting this will prevent server startup failure if the IDP is unavailable during startup.

Integration with other Decision Components

- **Purpose:**

DT Executor interacts with the following Decision components:

- Decision Execution(DE)
- DT Repository

- **Configuration:**

Define the details of these components in the `application.yml`:

```

de:
  port: <DE port number, default is 8090>
  host: <DE host, default is localhost>

testing:
  repo:
    port: <DT Repository port number, default is 8082>
    host: <DT Repository host, default is localhost>

```

- **Authentication Configuration for DE**

- **If DE is set with Basic Authentication:**

```

de:
  user: <username>
  password: <password>

```

- **Note:** The specified user (<username>) must have at least the "DE_USER" role assigned.

- **If DE and/or DT Repository are set with OAuth Authentication:**

(Assuming both are protected by the same IDP, here referred to as "sapidp")

```

spring:
  security:
    oauth2:
      client:
        provider:
          sapidp:
            token-uri: <IDP issuer token URL>
            client-id: <DT client id defined in IDP>
            client-secret: <DT client secret defined in IDP>
        registration:
          de:
            provider: sapidp
            authorization-grant-type: client_credentials
            scope:
              - user-token-exchange
              - DE_USER
          dt-repo:
            provider: sapidp
            authorization-grant-type: client_credentials
            scope:
              - user-token-exchange

```

- **Additional Parameter for IDP (Optional):**

If the IDP requires an extra parameter (like audience in Auth0), add the following configuration:

```
decision:
  security:
    oauth2:
      client:
        registration-extension:
          de:
            query-parameters:
              audience: https://de-for-testing
      dt-repo:
        query-parameters:
          audience: https://dt-repo
```

5.6.5 DT Generate

This section provides comprehensive configuration instructions for the DT Generate application across different deployment types (All-in-One, Standalone, and Containerized). It covers basic settings, OAuth 2.0 authentication, and integration with DT Repository.

Containerized Deployment

- **Configuration Location:**

The properties should be defined under the `dt-generate` section in the enterprise chart values under `configMapData`.

For detailed instructions, refer to the chart README file.

All-in-One Deployment

- **Configuration Location:**

The properties file must be placed at:

`<Decision home path>/config/dt-generate-config`

- **Configuration Guidelines:**

- The port number for `<* port number>` placeholders must match the one set in the Tomcat `server.xml` (as explained in the Installation chapter).
- The hostname for `<* host>` placeholders (except `<db host>`) should be set to `localhost`.

Standalone Deployment

- **Configuration Location:**

The properties file must be placed in the same directory where the `dt-generate-<version>.jar` file is located.

Note: The placeholder `<version>` denotes the specific version number of the JAR file used in the release.

Basic Configurations

- **Server Configuration:**

```
server:
  port: <DT Generate port number, default is 8083>

testing:
  repo:
    rsocket:
      server:
        port: <DT Repository RSocket port number - default is 7001>
```

OAuth 2.0 Authentication

To set DT Generate with OAuth 2.0 authentication, include the following in the `application.yml`:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          audiences: EnterpriseApp
          issuer-uri: <IDP issuer URL>
          jwk-set-uri: <IDP jwk set-uri URL - Optional>
```

Note:

- The `jwk-set-uri` is not mandatory, but it is recommended.
- Setting this will prevent server startup failure if the IDP is unavailable during startup.

Integration with DT Repository

- **Purpose:**

DT Generate interacts with DT Repository for various operations.

- **Configuration:**

Define the DT Repository details in the `application.yml`:

```
testing:
  repo:
    port: <DT Repository port number, default is 8082>
    host: <DT Repository host, default is localhost>
```

- **Authentication Configuration for DT Repository**

- **If DT Repository is set with OAuth Authentication:**

(Assuming it is protected by an IDP, referred to here as "sapidp")

```

spring:
  security:
    oauth2:
      client:
        provider:
          sapidp:
            token-uri: <IDP issuer token URL>
            client-id: <DT client id defined in IDP>
            client-secret: <DT client secret defined in IDP>
        registration:
          dt-repo:
            provider: sapidp
            authorization-grant-type: client_credentials
            scope:
              - user-token-exchange

```

- **Additional Parameter for IDP (Optional):**

If the IDP requires an extra parameter (like audience in Auth0), add the following configuration:

```

decision:
  security:
    oauth2:
      client:
        registration-extension:
          dt-repo:
            query-parameters:
              audience: https://dt-repo

```

5.6.6 DT Repair Configuration

This section provides detailed configuration guidelines for the DT Repair application, covering different deployment types (All-in-One, Standalone, and Containerized), basic settings, OAuth 2.0 authentication, and integration with DT Repository.

Containerized Deployment

- **Configuration Location:**

The properties should be defined under the `dt-repair` section in the enterprise chart values under `configMapData`.

For detailed instructions, refer to the chart README file.

All-in-One Deployment

- **Configuration Location:**

The properties file must be placed at:

`<Decision home path>/config/dt-repair-config`

- **Configuration Guidelines:**

- The port number for `<* port number>` placeholders must match the one set in the Tomcat `server.xml` (as explained in the Installation chapter).

- The hostname for `<* host>` placeholders (except `<db host>`) should be set to `localhost`.

Standalone Deployment

- **Configuration Location:**

The properties file must be placed in the same directory where the `dt-repair-<version>.jar` file is located.

Note: The placeholder `<version>` denotes the specific version number of the JAR file used in the release.

Basic Configurations

- **Server Configuration:**

```
server:
  port: <DT Repair port number, default is 8081>

testing:
  repo:
    rsocket:
      server:
        port: <DT Repository RSocket port number - default is 7001>
```

OAuth 2.0 Authentication

To set DT Repair with OAuth 2.0 authentication, include the following in the `application.yml`:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          audiences: EnterpriseApp
          issuer-uri: <IDP issuer URL>
          jwk-set-uri: <IDP jwk set-uri URL - Optional>
```

Note:

- The `jwk-set-uri` is not mandatory, but it is recommended.
- Setting this will prevent server startup failure if the IDP is unavailable during startup.

Integration with DT Repository

- **Purpose:**

DT Repair interacts with DT Repository for various operations.

- **Configuration:**

Define the DT Repository details in the `application.yml`:

```
testing:
  repo:
    port: <DT Repository port number, default is 8082>
    host: <DT Repository host, default is localhost>
```

- **Authentication Configuration for DT Repository**

- **If DT Repository is set with OAuth Authentication:**

(Assuming it is protected by an IDP, referred to here as "sapidp")

```
spring:
  security:
    oauth2:
      client:
        provider:
          sapidp:
            token-uri: <IDP issuer token URL>
            client-id: <DT client id defined in IDP>
            client-secret: <DT client secret defined in IDP>
        registration:
          dt-repo:
            provider: sapidp
            authorization-grant-type: client_credentials
            scope:
              - user-token-exchange
```

- **Additional Parameter for IDP (Optional):**

If the IDP requires an extra parameter (like audience in Auth0), add the following configuration:

```
decision:
  security:
    oauth2:
      client:
        registration-extension:
          dt-repo:
            query-parameters:
              audience: https://dt-repo
```

5.7 Decision Gateway

Decision Gateway is required in **Standalone** or **Containerized** deployments of the Decision environment. It acts as a centralized entry point for managing user authentication, routing requests to other Decision components, and handling user sessions. This section provides a complete configuration guide for Decision Gateway, including deployment, authentication, integration with other components, and session management.

5.7.1 Containerized Deployment

- **Configuration Location:**

The properties should be defined under the decision-gateway section in the enterprise

chart values under `configMapData`.

For detailed instructions, refer to the chart README file.

5.7.2 Standalone Deployment

- **Configuration Location:**

The properties file must be placed in the same directory where the `decision-gateway-<version>.jar` file is located.

Note: The placeholder `<version>` denotes the specific version number of the JAR file used in the release.

5.7.3 Basic Configuration

- **Server Configuration:**

```
server:
  port: <Decision Gateway port number, default is 8181>
```

5.7.4 OAuth 2.0 Authentication

Decision Gateway only supports OAuth 2.0 authentication. To set Decision Gateway with OAuth 2.0 authentication, include the following in the `application.yml`:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: <IDP issuer URL>
          jwk-set-uri: <IDP jwk set-uri URL - Optional>
      client:
        provider:
          sapidp:
            token-uri: <IDP issuer token URL>
            authorization-uri: <IDP authorization URL - Optional>
            user-name-attribute: <user name attribute in token - Optional>
            user-info-uri: <IDP user info URL - Optional>
            jwk-set-uri: <IDP jwk set-uri URL - Optional>
        registration:
          sapidp:
            client-id: <DG client id defined in IDP>
            client-secret: <DG client secret defined in IDP>
```

Note:

- The `jwk-set-uri` is not mandatory, but it is recommended.
- Setting this will prevent server startup failure if the IDP is unavailable during startup.

5.7.5 Integration with Other Decision Components

- **Purpose:**

Decision Gateway must be configured to communicate with all other Decision components in the environment.

- **Configuration:**

Define the decision components details in the `application.yml`:

```

dt:
  repair:
    uri: http://<DT Repair host>:<DT Repair port number>
  repo:
    uri: http://<DT Repository host>:<DT Repository port number>
  generate:
    uri: http://<DT Generate host>:<DT Generate port number>
  exec:
    uri: http://<DT Executor host>:<DT Executor port number>
  orchestrator:
    uri: http://<DT Orchestrator host>:<DT Orchestrator port number>
  dm:
    uri: http://<DM host>:<DM port number>
  pam:
    uri: http://<PAM host>:<PAM port number>
    ui:
      uri: http://<PAM UI host>:<PAM UI port number>

```

If not set the following default values will be used:

- dt.repo.uri =http://localhost:8082
- dt.orchestrator.uri=http://localhost:8085
- dt.repair.uri=http://localhost:8081
- dt.exec.uri=http://localhost:8084
- dt.generate.uri=http://localhost:8083
- dm.uri=http://localhost:9080
- pam.uri=http://localhost:8123
- pam.ui.uri=http://localhost:4200

5.7.6 Session Management

The Decision Gateway component is designed to manage user sessions efficiently. Sessions can be stored either in the Decision Gateway's **internal memory** or in a **Redis datastore**. While in-memory storage offers simplicity, using Redis is recommended for enhanced durability and scalability. Redis ensures that session data persists even if the Decision Gateway is restarted, preventing session loss. Additionally, it enables multiple instances of the Decision Gateway to share session data seamlessly, supporting scalable deployments and high availability.

Redis Session Store Configuration

- **Recommended for:**
 - Durable sessions (sessions that persist even after a restart).

- Scalable environments (multiple instances of Decision Gateway).

- **Configuration Example:**

```
decision:
  session:
    store-type: redis

  spring:
    data:
      redis:
        ssl:
          enabled: true # Set to false if SSL is not required
        host: <Redis host>
        port: <Redis port>
        username: <Redis username>
        password: <Redis password>

    session:
      redis:
        namespace: <Namespace for Redis keys - useful when multiple
environments share the same Redis instance>
```

In-Memory Session Store Configuration

- **Recommended for:**

- Simple deployments without the need for session durability.
- Single-node deployments.

- **Configuration Example:**

The following is also the default setting.

```
decision:
  session:
    store-type: none
  management:
    health:
      redis:
        enabled: false
```

5.8 Config Server

Config Server is optional in Standalone or Containerized deployments of the Decision environment. It is not required in the All-in-One deployment.

5.8.1 Containerized Deployment

- **Configuration Location:**

The properties should be defined under the `config-server` section in the enterprise chart values under `configMapData`.

For detailed instructions, refer to the chart README file.

5.8.2 Standalone Deployment

The application.yml properties file must be placed in the same directory as the config-server-`<version>.jar` file.

Note: The placeholder `<version>` denotes the specific version number of the JAR file used in the release.

5.8.3 Basic Configurations

- **Server and Messaging Configuration:**

```
server:
  port: <Config Server port number, default is 8888>

spring:
  rabbitmq:
    host: <RabbitMQ host, default is localhost>
    port: <RabbitMQ port, default is 5672>
```

- The **server port** defines the port on which the Config Server listens.
- The **RabbitMQ configuration** specifies the host and port of the RabbitMQ server, which Config Server may use for messaging.

5.8.4 Authentication (Basic Authentication)

- **Supported Authentication Type:**

Config Server only supports Basic Authentication for secure access.

- **Configuration Example:**

```
spring:
  security:
    user:
      name: <username, default is root>
      password: <password, default is s3cr3t>
```

- Set the username and password for basic authentication access.
- This ensures that only authorized users can access the configuration data.

5.9 RabbitMQ

RabbitMQ is **optional** in **Standalone** or **Containerized** deployments of the Decision environment. It is not required in the All-in-One deployment.

- **Supported RabbitMQ Versions:**

- Only RabbitMQ version 4.0.7 and above are supported.

5.9.1 Standalone Deployment

For installing RabbitMQ in standalone deployment, please refer to the official RabbitMQ Documentation.

5.9.2 Containerized Deployment

- **Configuration Location:**

The properties should be defined under the `rabbitmq` section in the enterprise chart values under `configMapData`.

For detailed instructions, refer to the chart README file.

5.10 Parameter Management (PaM)

Parameter Management supports both Standalone and Containerized Deployments.

5.10.1 Containerized Deployment

- **Configuration Location:**

The properties should be defined under the `pam` section in the enterprise chart values under `configMapData`.

For detailed instructions, refer to the chart README file.

5.10.2 Standalone Deployment

For deploying PaM in a standalone environment, please refer to the Parameter Management Installation Guide.

5.11 Health Service

The **Health Service** is an optional component in the **All-in-One deployment** of the Decision environment. It is **not required** in **Standalone** or **Containerized** deployments, which typically use the platform's own health service for monitoring.

5.11.1 All-in-One Deployment

- **Configuration Location:**

The properties file must be placed within the `<Decision home path>/config/health-service-config` directory.

- **Configuration Instructions:**

- The **port number** must match the one configured for Tomcat, as specified in the `server.xml` file.
- The **host name** must be set to `localhost` in all configurations

5.11.2 Basic Configurations

- **Port Number Configuration:**

```
server:
  port: <port number, default is 8989>
```

- **Service Health Check Configuration:**

The Health Service can monitor the health status of various Decision components. This is defined in the configuration file as follows:

```
health:
  service:
    service-info:
      - name: DM
        url: <all-in-one tomcat full http/https host and port, e.g.
http://localhost:9080>/dm/ws/api/actuator/health
      - name: DE
        url: <all-in-one tomcat full http/https host and
port>/bdes/ws/rs/api/actuator/health
      - name: DT Orchestrator
        url: <all-in-one tomcat full http/https host and port>/dt-
orchestrator/actuator/health
      - name: DT Repository
        url: <all-in-one tomcat full http/https host and port>/dt-
repo/actuator/health
      - name: DT Executor
        url: <all-in-one tomcat full http/https host and port>/dt-
exec/actuator/health
      - name: DT Generate
        url: <all-in-one tomcat full http/https host and port>/dt-
generate/actuator/health
      - name: DT Repair
        url: <all-in-one tomcat full http/https host and port>/dt-
repair/actuator/health
      - name: PaM
        url: <all-in-one tomcat full http/https host and
port>/pam/actuator/health
```

- Each service is listed under `service-info` with a **name** and the **URL endpoint** for its health check.
- Ensure that the URL uses the correct protocol (`http` or `https`) and the correct Tomcat host and port.

- If not set, the following default values are used:

```
health:
  service:
    service-info:
      - name: dmService
        url: http://localhost:${server.port}/dm/actuator/health
      - name: de
        url:
http://localhost:${server.port}/bdes/ws/rs/api/actuator/health
      - name: dt-repair
        url: http://localhost:${server.port}/dt-
repair/actuator/health
      - name: dt-orchestrator
        url: http://localhost:${server.port}/dt-
orchestrator/actuator/health
      - name: dt-repo
        url: http://localhost:${server.port}/dt-repo/actuator/health
      - name: dt-exec
        url: http://localhost:${server.port}/dt-exec/actuator/health
      - name: dt-generate
        url: http://localhost:${server.port}/dt-
generate/actuator/health
```

- **RabbitMQ Health Check Configuration:**

The Health Service can monitor the health status of RabbitMQ. By default, the RabbitMQ health indicator is not enabled. To include RabbitMQ in the health check response, it must be specified during configuration.

To enable this optional check, add the following properties to the `health-check` properties file:

```
health.service.service-info.7.name=rabbit-mq
health.service.service-info.7.url=http://{username}:{password}@
{host}:{port}/api/healthchecks/node
```

Where, 7 indicates the index of services configured in health service.

For example,

```
health.service.service-info.7.name=rabbit-mq
health.service.service-
info.7.url=http://guest:guest@localhost:8119/api/healthchecks/node
```

Note: Ensure to specify the correct `management.listener.port` in the URL.

Adding this configuration allows the health-service to include RabbitMQ's health status in its response. This enables improved monitoring and observability. However, this configuration is optional and not required for the health-service to function.

The following is a sample response of the API:

```
health.service.service-info.7.name=rabbit-mq
health.service.service-
info.7.url=http://guest:guest@localhost:8194/api/healthchecks/node

"rabbit-mq": {
  "time": 5,
  "message": "Status code '-1' should be a three-digit positive
integer",
  "status": "DOWN"
}
```

5.11.3 Authentication

The Health Service does not require authentication for its health check endpoints.

5.12 Logging

Logging is an essential feature for monitoring and troubleshooting within the Decision applications. This section describes how to configure logging for different deployment types.

5.12.1 Log Files Location

By default, all Decision applications store their log files in a "logs" folder, with the location varying based on the deployment type:

- **Standalone Deployment:**
 - The logs folder is created in the **same directory** where the application .jar file is located.
- **All-in-One Deployment:**
 - The logs folder is created in the directory specified by the `decision.home` property, which is set in the Tomcat setenv script (JAVA_OPTS variable), as detailed in the [All-in-One Installation](#).
- **Custom Log Location:**
 - To change the log file location, use the `logging.file.path` property in the application's configuration file (`application.properties` or `application.yml`).
 - Example:

```
logging.file.path=/home/my/path/to/logs
```

5.12.2 Log Level Configuration

The default log level for all Decision applications is set to **INFO**. You can modify the log level for either the entire application or specific packages. you can set the `logging.level` property in the configuration file (`application.properties` or `application.yml`).

- **Set Logging Level for the Entire Application:**

Use the `logging.level.root` property to set the logging level globally.

Example:

```
logging.level.root=WARN
```

- **Set Logging Level for Specific Packages:**

- To control logging for specific packages or classes, use the package name instead of root.
- For instance, to set the logging level to DEBUG only for Decision and Flow execution in the DE component:

```
logging.level.decision.execution=DEBUG  
logging.level.flow.execution=DEBUG
```

- **Available Logging Levels:**

- **ERROR** - Only error messages are logged.
- **WARN** - Warnings and errors are logged.
- **INFO** - General information, warnings, and errors are logged.
- **DEBUG** - Debug information along with all the above is logged.
- **TRACE** - Detailed trace information for debugging is logged.

5.12.3 External Log Configuration File

If the default logging configuration is insufficient, you can use an external log configuration file.

- **Logback Configuration File:**

- The external log configuration file must be named logback-spring.xml.
- The file should follow the standard Logback format .
- Sample Logback Configuration (logback-spring.xml)



- **Logback Configuration File Location:**

- **All-in-One Deployment:**
 - Place the file in <Decision home path>/config/config.
- **Standalone Deployment:**
 - Place the file in <application JAR file directory>/config.
Sample Logback Configuration (logback-spring.xml):

6. Startup

This section provides instructions for starting the Decision applications across different deployment types: Containerized (Docker/Kubernetes), All-in-One (Tomcat-based), and Standalone (self-executable JARs). Follow the appropriate method based on your deployment type to ensure a smooth startup.

6.1 Containerized

- **Startup Instructions:**

Refer to the **enterprise chart README.md** file for detailed information about chart installation and starting the containerized applications.

- **Note:** Ensure all containerized application configurations are correctly set in the Helm chart values before deployment.

6.2 All-in-one Deployment

- **Startup Command:** To start all Decision applications deployed under Tomcat, execute the following command:

```
<Tomcat root>/bin/startup.sh
```

- **Note:** Ensure that all necessary configurations are correctly set in the `<Decision home path>/config` directory before starting.

6.3 Standalone

- **DM Application:**

To start the DM application, navigate to the directory where `dm-<version>.jar` is located and execute the following command:

Note: The placeholder `<version>` denotes the specific version number of the JAR file used in the release.

```
java -Xmx<max memory> -jar dm-<version>.jar \  
  --add-opens=java.base/sun.reflect.annotation=ALL-UNNAMED \  
  --add-modules=java.se \  
  --add-exports=java.base/jdk.internal.ref=ALL-UNNAMED \  
  --add-opens=java.base/sun.nio.ch=ALL-UNNAMED \  
  --add-opens=java.base/java.util=ALL-UNNAMED \  
  --add-opens=java.base/java.lang=ALL-UNNAMED \  
  --add-opens=java.management/sun.management=ALL-UNNAMED \  
  --add-opens=jdk.management/com.sun.management.internal=ALL-UNNAMED
```

- Replace `<max memory>` with the desired memory allocation for DM, as specified in the [Prerequisites](#) section.

- **DE Application:**

- Without UDF (User Defined/Custom Functions):

Use the same startup instructions as for other applications.

- With UDF (User Defined/Custom Functions):

Follow the startup instructions provided in the [Standalone DE Custom Functions Deployment](#) section.

- **All other Decision applications:**

To start any other Decision application, navigate to the directory of the application's jar file and execute:

```
java -Xmx<max memory> -jar <jar filename> \  
--add-opens=java.base/sun.reflect.annotation=ALL-UNNAMED \  
--add-modules=java.se \  
--add-exports=java.base/jdk.internal.ref=ALL-UNNAMED \  
--add-opens=java.base/sun.nio.ch=ALL-UNNAMED \  
--add-opens=java.base/java.util=ALL-UNNAMED \  
--add-opens=java.base/java.lang=ALL-UNNAMED \  
--add-opens=java.management/sun.management=ALL-UNNAMED \  
--add-opens=jdk.management/com.sun.management.internal=ALL-UNNAMED
```

- Replace `<jar filename>` with the application's jar file name.
- Replace `<max memory>` with the desired memory allocation for DM, as specified in the [Prerequisites](#) section.

7. Post Installation Validation

This section provides guidelines for verifying the successful installation and proper functioning of the Decision applications through **Health Check** and **Heartbeat APIs**.

7.1 Health Check Validation

Each Decision service exposes two essential endpoints for validating service availability and health status:

- **Heartbeat API:**
 - Used to check if the service is running.
 - Returns the service version.

Example Response:

```
{
  "version": "7.1"
}
```

Example Endpoint:

```
https://<host>:<port>/bdes/ws/rs/api/actuator/health
```

- **Health Check API:**
 - Provides detailed information about the service's health and its sub-components.

Example Response:

```
{
  "database": {
    "status": "UP",
    "details": {
      "time": 2
    }
  },
  "serviceDetails": {
    "status": "UP",
    "details": {
      "serviceName": "de",
      "version": "7.1",
      "time": 0
    }
  }
}
```

7.2 Health Check and Heartbeat API Endpoints

This section lists the Health Check and Heartbeat API endpoints for each Decision application. The Health Check API provides detailed information about the application's health status, while the Heartbeat API confirms that the service is running and returns the service version.

Service	Health Check API Path	Heartbeat API Path
Config Server	/actuator/health	/heartbeat
DE	/bdes/ws/rs/actuator/health	/bdes/ws/rs/api/heartbeat
DM	/dm/actuator/health	/dm/ws/rs/heartbeat
DT Executor	/dt-exec/actuator/health	/dt-exec/heartbeat
DT Generate	/dt-generate/actuator/health	/dt-generate/heartbeat
DT Orchestrator	/dt-orchestrator/actuator/health	/dt-orchestrator/heartbeat
DT Repair	/dt-repair/actuator/health	/dt-repair/heartbeat
DT Repository	/dt-repo/actuator/health	/dt-repo/heartbeat
Gateway	/actuator/health	/heartbeat
PaM	/pam/actuator/health	/pam/api/parameters/heartbeat

- Ensure that the host and port values in the URLs are correctly set based on your deployment configuration.
- The services should return a status of "UP" in the Health Check API response for a successful installation.

8. Advanced Configuration Topics

8.1 Customized Reports

8.1.1 Prerequisites

To be able to create a report, you should be able to write SQL/HQL queries and be familiar with Decision schema.

8.1.2 Adding a New Report to Decision

To add a report to Decision, you need to create a report XML file with the suffix `'.decReport.xml'`, as will be explained in section 4 of this document.

This XML file should be copied to the `generic-reports` directory either in the `classpath` or in the file system directory under `<decision.home>/data/generic-reports`.

If you add a report to the classpath directory, the server should be restarted to view the new report in Decision, since the classpath is not refreshed.

8.1.3 About Reports

A report consists of three parts: input, data, and output.

Input

The input includes the following parameters and their value providers:

1. The parameters are the parameters for the main query, according to which the report results will be generated. The input parameter may be one of the following types:
 - Combo box - single selection
 - Combo box - multiple selection
 - Date
 - Checkbox
 - Free text

For example, in the Administration report, there are six input parameters, two of which are date parameters, and the rest are combo boxes.

▼ Description

A report showing information about all changes made in Decision according to the specified parameters

▼ Parameters

Format *

CSV File

Page Type *

A4 Landscape

Output Type *

Download

Action

Action

Entity Type

Entity Type

User

User

From Date

From Date

To Date

To Date

Sorted By

Sorted By

Generate Report

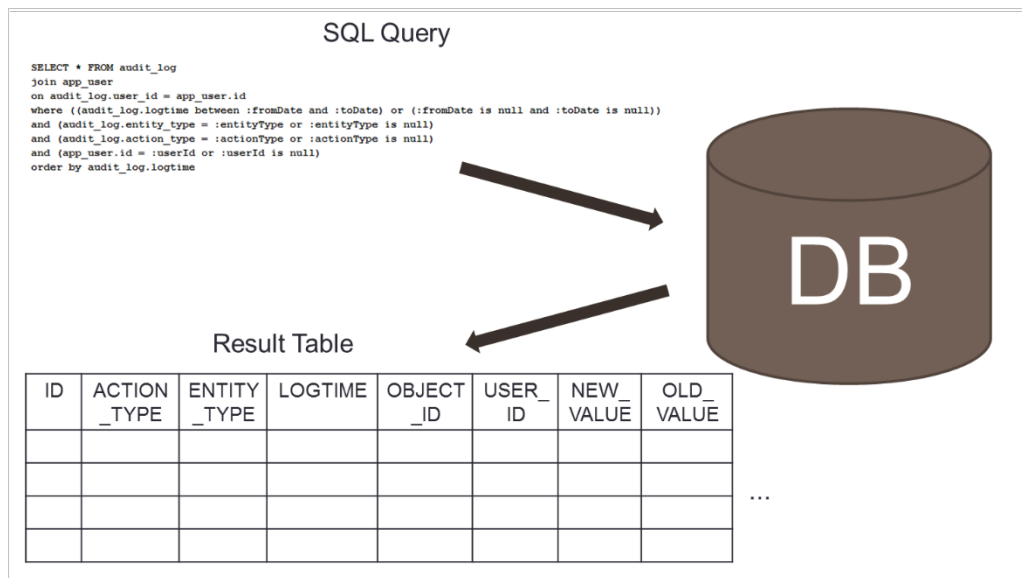
- Value providers should be defined for combo box parameters. In order to define a value provider for a specific parameter, a query resulting in its possible values should be defined. For example, for 'Action' input parameter the corresponding query for the value provider is `SELECT distinct action_type FROM audit_log`.

Action	Action
Entity Type	DELETE INSERT UPDATE
User	USER

Data

The report data includes the main query that will generate the report results.

After choosing values for report input parameters, the report can be generated. The query is set by the selected input parameter values, executed on the database, and returns a result table.



Output

The columns that will appear in the report output, which are a subset of the table columns.

8.1.4 XML File Attributes

The attributes listed in the following table should be defined. Some are mandatory, as denoted in the last column.

XML Attribute	Explanation	Mandatory
Id	Unique identifier string in all reports	+
name	The name of the report that will appear in the report list screen	+
title	Title in the report output	-
description	A description that will appear in the report screen	-
query	The main query of the report, written in SQL/HQL	+
parameters	Input parameter definition for the main query	+
valueProviders	Query that is defined only for combo box parameters	+
columns	Report column definition	+
Permissions	Defines the required permissions to execute the report. If no permission is defined, then only the standard reports permission ALLOW_RUN_REPORTS is required.	-

1. Query

The following query attributes should be defined:

- i. Type: SQL/HQL.
- ii. Extension: Query ("select ... from ..."), where parameter names are prefixed with a semi-colon (;).

For example:

```
<!-- the main query of the report -->
<!-- type: "sql" or "hql" -->
<query type="sql">

    <!-- valid query in the chosen language as will be run on your data base -->
    <!-- query parameters will be denoted by ':' before their name (see userId as an example) -->
    <!-- please notice that the query language defines the way you define your parameters in the paramters
        section of the xml -->

    SELECT * FROM audit_log
    join app_user
    on audit_log.user_id = app_user.id
    where ((audit_log.logtime between :fromDate and :toDate) or (:fromDate is null and :toDate is null))
    and (audit_log.entity_type = :entityType or :entityType is null)
    and (audit_log.action_type = :actionType or :actionType is null)
    and (app_user.id = :userId or :userId is null)
    order by audit_log.logtime
```

2. Parameters

The attributes listed in the following table should be defined. Some are mandatory, as denoted in the last column.

Parameter Attribute	Value	Mandatory
id	The parameter unique identifier as appears in the main query	+
name	The parameter name that will appear in the report screen	+
description	A description of the parameter that will appear in the report screen	-
order	The placement of the parameter. For example, the order of the first parameter is "1"	+
isOptional	One of the following values: true, false, 1, or 0. If the parameter is required (isOptional="false" or "0"), the report cannot be generated without providing a parameter value	+
valueProviderId	Non-empty string that references the valueProvider element (only for combo box parameters).	-
entityType	In case the parameter is a Decision entity, its entity type should be defined	-
dataType	One of the following data type: boolean, date, freetext, multiple, text	-
dependenciesId	Parameter Ids that are separated by commas (,)	-

3. entityType

For a user parameter that lists all possible users in Decision, "USER" entity type should be defined.

Examples of entity types

Entity
DECISION_VIEW
RULE_FAMILY_VIEW
FACT_TYPE
VIEW_GROUP
WHITEBOARD
USER

For more entity type, refer Entity Type Names and Classes

4. datatype

Data Type	Parameter Type
boolean	Checkbox
date	Date
freetext	Text box
multiple	Multi selection combo box
text	Combo box, the parameter of which is a string (not a Decision entity)

5. entityType and dataType allowed combinations

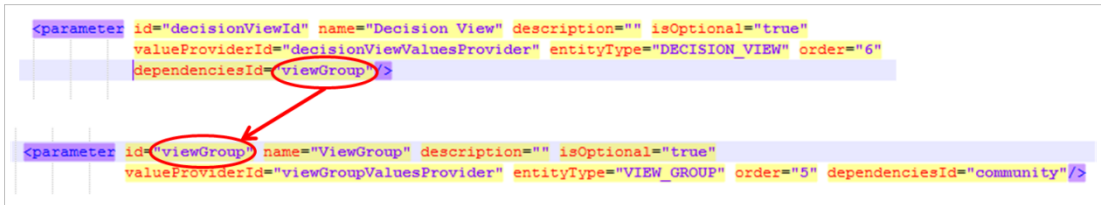
Parameter Type	Data Type	Entity Type
Checkbox	boolean	-
Date	date	-
Text box	freeText	-
Multi-selection combo box of Decision entity	multiple	Decision entity
Multi-selection combo box of strings	multiple	-
Single-selection combo box of Decision entity	-	Decision entity
Single-selection combo box of strings	text	-

6. dependenciesId

In some cases, the values of an input parameter depend on other parameter values. In such cases, users will not be able to choose a value for the parameter before specifying a value for the other parameter.

To define a parameter as dependent, its Id value must be specified for dependenciesId, separated by commas (,).

For example, the input parameter of a Decision View dependent on ViewGroup selection.



```
<parameter id="decisionViewId" name="Decision View" description="" isOptional="true"
valueProviderId="decisionViewValuesProvider" entityType="DECISION_VIEW" order="6"
dependenciesId="viewGroup"/>

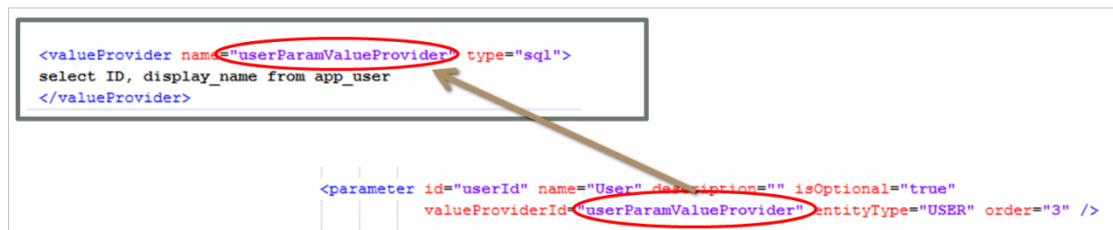
<parameter id="viewGroup" name="ViewGroup" description="" isOptional="true"
valueProviderId="viewGroupValuesProvider" entityType="VIEW_GROUP" order="5" dependenciesId="community"/>
```

7. valueProviderId

Attribute	Description
Type	SQL/HQL
Name	The value provider Id (to be referenced in the parameter attribute)
Extension	Valid query of "select .. from .." that returns one or two columns

The columns returned by the query represent the Id of the entity and the name of the element, as they will appear in the combo box. If there is only one column, the name must be unique, so that it can be referenced as the id as well.

Example of a value provider for a user parameter:



```
<valueProvider name="userParamValueProvider" type="sql">
select ID, display_name from app_user
</valueProvider>

<parameter id="userId" name="User" description="" isOptional="true"
valueProviderId="userParamValueProvider" entityType="USER" order="3" />
```

8. Columns - name and value attributes

Attribute	Description
name	Column header name
value	Depends on the query type: SQL - value should correspond to the column name of the table in the database HQL - value should correspond to the field name of the returned entity

For example, this is a definition of the columns section in a report:

```
<columns>
  <column name="Time" value="LOGTIME"/>
  <column name="Entity Type" value="ENTITY_TYPE"/>
  <column name="User" value="DISPLAY_NAME"/>
  <column name="Action" value="ACTION_TYPE"/>
  <column name="Old Value" value="OLD_VALUE"/>
  <column name="New Value" value="NEW_VALUE"/>
</columns>
```

Correspondingly, column headers in the report output will appear as follows:

Administration Generic Report					
Time	Entity Type	User	Action	Old Value	New Value

9. Columns - resolver-id and resolver-params

The resolver-id and resolver-params attributes should be used to show entity names that are not persistent in the database.

resolverId	Corresponding resolverParams
classNameAndIdNameResolver	<object-id>, <class-name>
classNameAndIdWithContextClassAndIdNameResolver	<object-id>, <class-name>, <object-id-of-context>, <class-name-of-context>
entityTypeAndIdNameResolver	<object-id>,<entity-type>

Examples:

```
<column name="User Name" resolver-id="classNameAndIdNameResolver"
resolver-params="USER_ID,com.sapiens.bdms.core.common.model.security.User"/>
```

```
<column name="Decision View Name" resolver-id="classNameAndIdWithContextClassAndIdNameResolver"
resolver-params="DECISION_VIEW_ID,com.sapiens.bdms.core.common.model.decision.DecisionView,
VIEW_GROUP_ID, com.sapiens.bdms.core.common.model.decision.viewgroup.ViewGroup"/>
```

```
<column name="User Name" resolver-id="classNameAndIdNameResolver"
resolver-params="USER_ID,com.sapiens.bdms.core.common.model.security.User"/>
```

10. Permissions

To define the required permission for the custom report, add the permissions elements as shown below. If multiple permissions are required, each permission can be listed as shown. When multiple permissions are defined, a user must have either one of the permissions to generate the report.

```
<permissions>
```

```

<permission>Permission 1 name</permission>
<permission> Permission 2 name </permission>
</permissions>
For example,
<permissions>
  <permission>KS_INSTANCE_READ</permission>
  <permission>DEC_READ</permission>
</permissions>

```

8.1.5 Important Issues

1. For each parameter id in the main query, a corresponding parameter should be defined in the parameters section

```

<query type="sql">
  SELECT * FROM audit_log
  join app_user
  on audit_log.user_id = app_user.id
  where ((audit_log.logtime between :fromDate and :toDate) or (:fromDate is null and :toDate is null))

  and (audit_log.entity_type = :entityType or :entityType is null)

  and (audit_log.action_type = :actionType or :actionType is null)

  and (app_user.id = :userId or :userId is null)
  order by audit_log.logtime
</query>

<parameters>
  <parameter id="actionType" name="Action" description="" isOptional="true"
    valueProviderId="actionTypeParamValueProvider" dataType="text" order="1" />

  <parameter id="entityType" name="Entity Type" description="" isOptional="true"
    valueProviderId="entityTypeParamValueProvider" dataType="text" order="2" />

  <parameter id="userId" name="User" description="" isOptional="true"
    valueProviderId="userParamValueProvider" entityType="USER" order="3" />

  <parameter id="fromDate" name="From Date" description="" isOptional="true" dataType="date" order="4" />

  <parameter id="toDate" name="To Date" description="" isOptional="true" dataType="date" order="5" />
</parameters>

```

2. For each value provider defined in a parameter, a corresponding value provider should be defined in the value providers section. For more details, refer About Reports.

```

<parameter id="actionType" name="Action" description="" isOptional="true"
  valueProviderId="actionTypeParamValueProvider" dataType="text" order="1" />

<valueProviders>
  <valueProvider name="actionTypeParamValueProvider" type="sql">

    <!-- the valid query to be run to provide distinct values to the parameter combo-box -->
    SELECT distinct action_type FROM audit_log

  </valueProvider>

```

3. The column value should match a column name in the resulting table in the database.

For example, for the query select * from audit_log, the column names can be:

ID	ACTION_TYPE	ENTITY_TYPE	LOGTIME	OBJECT_ID	USER_ID	NEW_VALUE	OLD_VALUE
----	-------------	-------------	---------	-----------	---------	-----------	-----------

8.1.6 Entity Type Names and Classes

The following table lists all entity type names and their corresponding classes that can be used in XML files.

Entity Name	Corresponding Entity Class
ANALYZED_TEXT_INFO	com.sapiens.bdms.text.analyzer.common.model.AnalyzedTextInfo
ARD	com.sapiens.bdms.ard.common.model.Ard
ASSET	com.sapiens.bdms.core.common.model.decision.asset.Asset
BCD	com.sapiens.bdms.core.common.model.decision.asset.Bcd
BCD_TASK	com.sapiens.bdms.core.common.model.decision.asset.BcdTask
BKM_INVOCATION	com.sapiens.bdms.core.common.model.decision.BkmInvocation
Branch	com.sapiens.bdms.core.common.model.deploy.project.Branch
BUSINESS_CONCEPT	com.sapiens.bdms.core.common.model.decision.fact.BusinessConcept
CONCLUSION_NOTES	com.sapiens.bdms.core.common.model.decision.ConclusionNotes
CONTEXT_DIAGRAM	com.sapiens.bdms.contextDiagram.common.model.ContextDiagram
Contract	com.sapiens.bdms.contextDiagram.common.model.proxy.definition.Proxy Definition
Customer	com.sapiens.bdms.contextDiagram.common.model.proxy.definition.Proxy Definition
DECISION	com.sapiens.bdms.core.common.model.decision.Decision
DECISION_VIEW	com.sapiens.bdms.core.common.model.decision.DecisionView
DEPLOYMENT_DESCRIPTOR	com.sapiens.bdms.core.common.model.deploy.DeploymentDescriptor
Environment	com.sapiens.bdms.exportCore.common.model.Environment
FACT_DOMAIN	com.sapiens.bdms.core.common.model.decision.fact.FactDomain
FACT_DOMAIN_VALUE	com.sapiens.bdms.core.common.model.domain.DomainValue

Entity Name	Corresponding Entity Class
FACT_TYPE	com.sapiens.bdms.core.common.model.decision.fact.FactType
FACT_TYPE_DEFINITION	com.sapiens.bdms.core.common.model.decision.fact.FactTypeDefinition
FORMULA	com.sapiens.bdms.core.common.model.decision.formula.Formula
GLOSSARY	com.sapiens.bdms.core.common.model.decision.glossary.Glossary
GLOSSARY_ARTIFACT	com.sapiens.bdms.core.common.model.decision.glossary.GlossaryArtifact
MESSAGE_CATEGORY	com.sapiens.bdms.core.common.model.decision.message.RowMessageCategory
MODE_DEFINITION	com.sapiens.bdms.core.common.model.decision.fact.element.ModelDefinition
MODEL_MAPPING	com.sapiens.bdms.core.common.model.decision.fact.element.ModelMapping
MODEL_VERSION	com.sapiens.bdms.core.common.model.decision.fact.element.ModelVersion
OPERATOR	com.sapiens.bdms.core.common.model.decision.CellOperator
PERMISSION_GROUP	com.sapiens.bdms.core.common.model.security.PermissionGroup
Process Task	com.sapiens.bdms.contextDiagram.common.model.proxy.definition.ProxyDefinition
Project	com.sapiens.bdms.core.common.model.deploy.project.Project
PROXY_DEFINITION	com.sapiens.bdms.contextDiagram.common.model.proxy.definition.ProxyDefinition
PROXY_INSTANCE	com.sapiens.bdms.contextDiagram.common.model.proxy.ProxyInstance
PULL_REQUEST	com.sapiens.bdms.core.common.model.deploy.project.PullRequest
RANGE	com.sapiens.bdms.core.common.model.domain.Range
REPO_QUERY_BASKET	com.sapiens.bdms.core.common.model.decision.query.RepoQueryBasket
Revision	com.sapiens.bdms.core.common.model.deploy.project.Revision
ROLE	com.sapiens.bdms.core.common.model.security.Role

Entity Name	Corresponding Entity Class
RULE_ FAMILY_ VIEW	com.sapiens.bdms.core.common.model.decision.RuleFamilyView
RULE_ FAMILY_ VIEW_ CONCLUSION	com.sapiens.bdms.core.common.model.decision.fact.FactTypeDefinition
RULE_ FAMILY_ VIEW_ CONDITION	com.sapiens.bdms.core.common.model.decision.Condition
RULE_ FAMILY_ VIEW_ ROW	com.sapiens.bdms.core.common.model.decision.RuleRow
RULE_ FAMILY_ VIEW_ ROW_ CONCLUSION_ CELL	com.sapiens.bdms.core.common.model.decision.ConclusionCell
RULE_ FAMILY_ VIEW_ ROW_ CONDITION_ CELL	com.sapiens.bdms.core.common.model.decision.ConditionCell
RULE_ FAMILY_ VIEW_ ROW_ MESSAGE	com.sapiens.bdms.core.common.model.decision.message.RuleRowMessage
RULE_FLOW	com.sapiens.bdms.core.common.model.decision.ruleFlow.RuleFlow
SYSTEM_ OPTIONS	com.sapiens.bdms.core.common.model.property.SystemProperty
USER	com.sapiens.bdms.core.common.model.security.User
VALIDATION_ RULE	com.sapiens.bdms.core.common.model.validation.ValidationDefinition
VIEW	com.sapiens.bdms.core.common.model.decision.View
VIEW_GROUP	com.sapiens.bdms.core.common.model.decision.viewgroup.ViewGroup

Entity Name	Corresponding Entity Class
WEB_SERVICE_KNOWLEDGE_MODEL	com.sapiens.bdms.core.common.model.decision.bkm.WebServiceKnowledgeModel
WHITEBOARD	com.sapiens.bdms.core.common.model.decision.asset.Whiteboard
WORKFLOW	com.sapiens.bdms.core.common.model.decision.asset.Workflow

8.2 Customized Configuration Properties

In Decision, **properties** serve as control parameters that govern how various system components behave. These properties are **customizable**, enabling system administrators to tailor the environment to suit operational or integration needs.

Customized properties are defined within dedicated configuration files, which are loaded and interpreted by Decision during the startup process. These files allow fine-tuned control over functionality, performance, and connectivity for each Decision component.

This section outlines the custom property files and the associated configuration options available to system administrators. Modifications to these files should be performed **only by authorized personnel** to ensure consistency, stability, and compliance with organizational standards.

Note: The properties and files described in this documentation are not tied to a specific Decision release. Some configurations may apply only to certain versions. It is strongly advised to verify compatibility with your specific deployment version before making any changes.

8.2.1 Configuration Files and Location

In Decision applications, the configuration properties define the application's behavior and are crucial for setting up the environment. The method of configuring these properties varies based on the type of deployment used. Here, we outline how to manage configuration properties for three common deployment types: **All-in-One Deployment**, **Standalone Deployment**, and **Containerized Deployment**

Configuration files and property names are always specific to each Decision application and must be considered in the context of the respective application.

All deployment types support environment variables or command-line system properties to override default configurations or custom file configurations. Both Standalone and All-in-One deployments support custom properties files in the file system, while Containerized deployments rely on the container platform to override properties, as elaborated below.

All required basic configuration properties are covered in the [Configuration](#) section of this document.

Refer to the [Decision Testing \(DT\) Advanced Configuration](#) section for performance fine-tuning or change of default behavior.

8.2.1.1 Containerized Deployment

For Containerized deployments, the properties are injected into the container as environment variables from ConfigMaps in Kubernetes. This allows for a flexible and scalable way to manage configuration across multiple instances of the application without needing to rebuild the container image.

8.2.1.2 All-in-One Deployment

In All-in-One deployment, all Decision applications are hosted under a single Tomcat server, with each application having a dedicated configuration directory. Configuration files are typically named `application.properties` or `application.yml`, depending on preference.

For Decision Manager (DM), two special optional property files are supported:

- `externalSecurity.properties` for LDAP integration.
- Custom properties for "Customized Reports" feature.

The table below lists the configuration file paths for each application in All-in-One deployment.

Application	Configuration File Path
Decision Manager (DM)	<Decision home path>/config/dm-config/application.properties (or .yml)
	Optional: <code>externalSecurity.properties</code> (for LDAP integration)
Decision Execution (DE)	<Decision home path>/config/de-config/application.properties (or .yml)
DT Repository (repo)	<Decision home path>/config/dt-repo-config/application.properties (or .yml)
DT Execution (exec)	<Decision home path>/config/dt-exec-config/application.properties (or .yml)
DT Repair (repair)	<Decision home path>/config/dt-repair-config/application.properties (or .yml)
DT Generation (generate)	<Decision home path>/config/dt-generate-config/application.properties (or .yml)
DT Orchestration	<Decision home path>/config/dt-orchestrator-config/application.properties (or .yml)
Health Service	<Decision home path>/config/health-service-config/application.properties (or .yml)

8.2.1.3 Standalone Deployment

In Standalone deployment, each Decision application operates independently, with configuration files stored in the same folder as the application JAR file. These files are named `application.properties` or `application.yml`.

For Decision Manager (DM), two special optional property files are supported:

- `externalSecurity.properties` for LDAP integration.
- Custom properties for "Customized Reports" feature.

The table below lists the configuration file paths for each application in Standalone deployment.

Application	Configuration File Path
All Applications	<code>application.properties</code> or <code>application.yml</code> (same folder as JAR)
Decision Manager (DM)	Optional: <code>externalSecurity.properties</code> (LDAP)

8.2.2 Configuration Files

Refer to the [Configuration Files](#) section in the Appendix for details on the various configuration files and the properties available in each file.

8.2.3 Advanced DT related properties on DM

DM serves as an authentication proxy for all DT applications, meaning that all DT-related traffic first passes through DM for authentication. Refer to the [Advanced DT-related properties on DM](#) section in the Appendix for properties that can be added to DM's `application.properties` file to modify the proxy's default behavior.

8.2.4 Optional Configuration

Change the line given below in `catalina.properties` to disable the TLD scan:

- `tomcat.util.scan.StandardJarScanFilter.jarsToSkip=\*`
- `tomcat.util.scan.StandardJarScanFilter.jarsToScan=\`
- `log4j-taglib*.jar,\`
- `log4j-web*.jar,\`
- `log4javascript*.jar,\`
- `slf4j-taglib*.jar, \`
- `jstl*.jar, \`
- `bdms*.jar, \`
- `decision*.jar,\`
- `bdes*.jar, \`
- `byte-buddy*.jar, \`
- `reactor*.jar, \`
- `spring*.jar`

8.3 Custom Functions in Decision Modeling

Decision offers various core functions to use in modeling.

In some cases, you may want to override the behavior of such functions with your own custom implementation of the core function or define an entirely new custom function.

To define a new function, you will need to:

- Define the prototype of the function so that it can be used for modeling in Decision.
- Add your custom implementation of the function to the DM and DE testing instance deployment so that you can compile and test your business decisions within Decision Testing.
- Define the mapping of your custom function for your adapters (such as the Decision Java adapter).
- Add your custom implementation of the function to the executing target system build-path (such as in the DE environment).

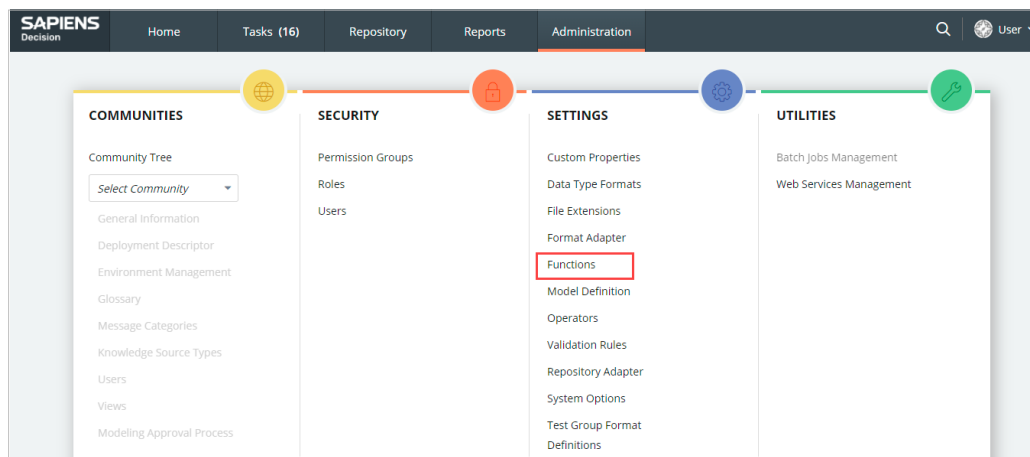
8.3.1 Defining a Custom Function Prototype

The first step in adding a custom function is to define the function prototype in Decision. Doing so will allow modelers to use the new function in the function wizard when writing Rule Family View formulas in Decision.

This step is only required if you are defining a new custom function or modifying an existing one (as opposed to overriding one that has the exact same prototype). Skip this section if you want to override a core function that receives the same parameters as the function you want to create.

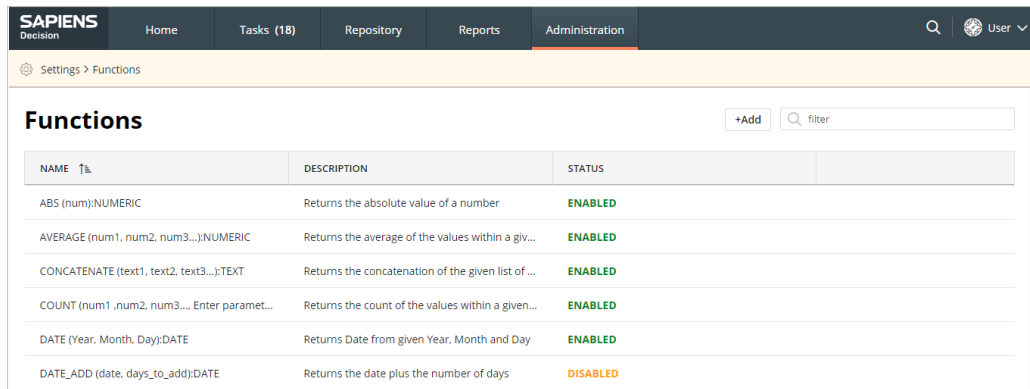
To define a custom function prototype in Decision:

1. Log in to Decision Manager with administrator privileges.
2. Click Administration.
3. Click Functions under Settings.



Functions option

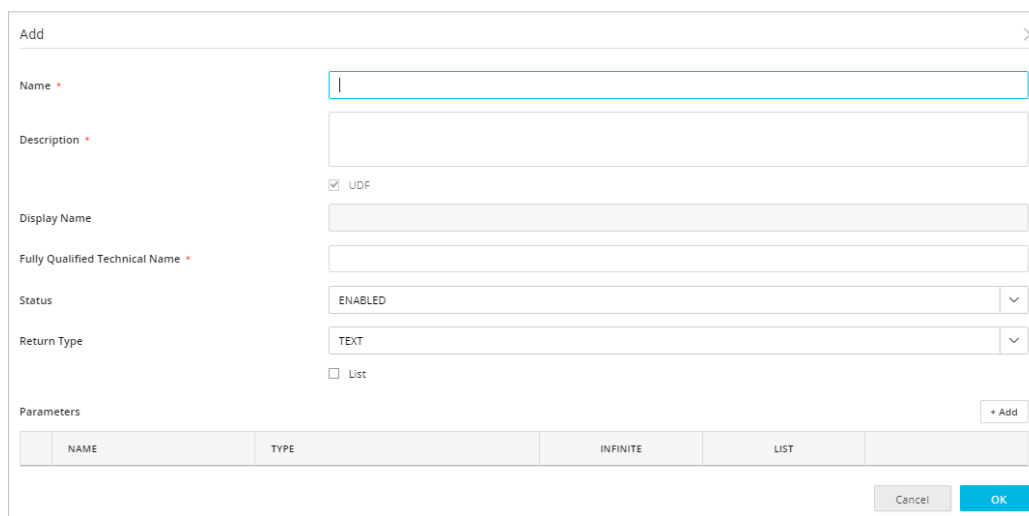
4. Click +Add.



NAME	DESCRIPTION	STATUS
ABS (num);NUMERIC	Returns the absolute value of a number	ENABLED
AVERAGE (num1, num2, num3...);NUMERIC	Returns the average of the values within a giv...	ENABLED
CONCATENATE (text1, text2, text3...);TEXT	Returns the concatenation of the given list of ...	ENABLED
COUNT (num1, num2, num3...; Enter paramet...	Returns the count of the values within a given...	ENABLED
DATE (Year, Month, Day);DATE	Returns Date from given Year, Month and Day	ENABLED
DATE_ADD (date, days_to_add);DATE	Returns the date plus the number of days	DISABLED

Creating a customized function

The Add function dialog is displayed.



Add

Name *

Description *

☒ UDF

Display Name

Fully Qualified Technical Name *

Status

Return Type

☐ List

Parameters

NAME	TYPE	INFINITE	LIST

Cancel OK

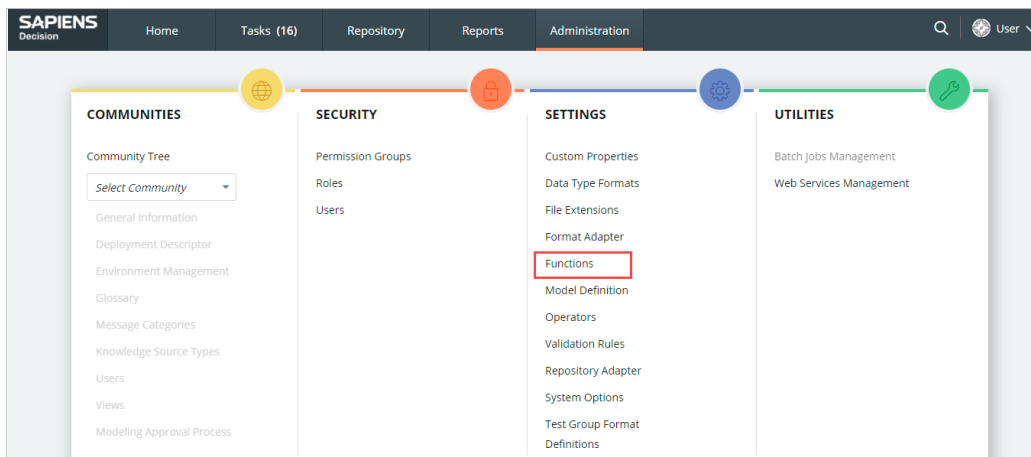
Add function dialog

For details on how to create, edit or migrate new Functions, refer to the Functions section in the first step in adding a custom function is to define the function prototype in Decision. Doing so will allow modelers to use the new function in the function wizard when writing Rule Family View formulas in Decision.

This step is only required if you are defining a new custom function or modifying an existing one (as opposed to overriding one that has the exact same prototype). Skip this section if you want to override a core function that receives the same parameters as the function you want to create.

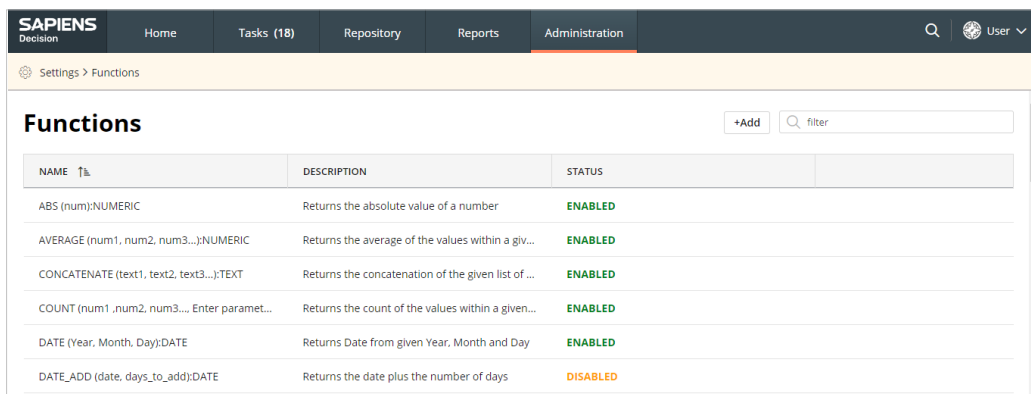
To define a custom function prototype in Decision:

1. Log in to Decision with administrator privileges.
2. Click Administration.
3. Click Functions under Settings.



Functions option

4. Click +Add.



Creating a customized function

The Add function dialog is displayed.

The 'Add' function dialog box is shown. It includes the following fields and options:

- Name ***: Text input field.
- Description ***: Text input field.
- ☒ **UDF**: Checkmark indicating User Defined Function.
- Display Name**: Text input field.
- Fully Qualified Technical Name ***: Text input field.
- Status**: Dropdown menu with 'ENABLED' selected.
- Return Type**: Dropdown menu with 'TEXT' selected.
- ☐ **List**: Checkmark indicating List return type.
- Parameters**: Table with columns NAME, TYPE, INFINITE, and LIST. A '+ Add' button is next to the table.

Buttons at the bottom: Cancel, OK.

Add function dialog

For details on creating, editing, or migrating new Functions, refer to the Functions section in the *Decision Manager (DM) Admin Guide*.

8.3.2 Adding a Custom Implementation in Decision

The implementation of custom functions in Decision should be written in Java.

To add a custom Java implementation of your custom function:

1. Write your Java implementation of the function and compile it into a jar file.
2. Deploy or copy the jar file to the DM and DE testing applications in the correct location, according to your Decision deployment type, by following the “Custom Functions Deployment” instructions below.
3. Once tested and finalized, deploy the jar file(s) to any DE or other execution environment. If using DE, follow the same instructions detailed below for a DE instance.
4. Restart the relevant DE and DM servers.

8.3.2.1 Custom Functions Jars Deployment

Any jar containing custom functions, along with any dependency jars, must be deployed to both the DM and the DE instances used for testing and/or production, according to your deployment type.

Containerized Deployment

For containerized environments, refer to **the Enterprise Chart README.md** file for detailed instructions on mounting UDFs.

All-in-one Deployment

DM Application:

- To use User Defined Functions (UDF) jar files in DM, place the jar files in the following directory:

`<Tomcat folder>/gen-classpath/udfs`

DE Application:

- To use UDF jar files in DE, mount them to Tomcat’s library by modifying the `server.xml` file:

```
<Context docBase="bdes" path="/bdes">
  <Resources allowLinking="true">
    <PostResources
      base="/udf"
      className="org.apache.catalina.webresources.DirResourceSet"
      webAppMount="/WEB-INF/lib" />
    </Resources>
  </Context>
```

- In this example, the UDF jar files are placed in the `/udf` directory and are automatically mounted to `/WEB-INF/lib` in Tomcat.

Standalone Deployment

DM Application:

- To use UDF jar files in DM, place the jar files in the following directory:

<DM jar location>/gen-classpath/udfs

DE Application:

- If you need to use additional UDF jar files in DE, start DE using the following command:

```
java -Xmx<max memory> -cp de-7.5.2.jar -Dloader.path=<path_to_udf_
additional_jars> \
  org.springframework.boot.loader.launch.PropertiesLauncher \
  --add-opens=java.base/sun.reflect.annotation=ALL-UNNAMED \
  --add-modules=java.se \
  --add-exports=java.base/jdk.internal.ref=ALL-UNNAMED \
  --add-opens=java.base/sun.nio.ch=ALL-UNNAMED \
  --add-opens=java.base/java.lang=ALL-UNNAMED \
  --add-opens=java.management/sun.management=ALL-UNNAMED \
  --add-opens=jdk.management/com.sun.management.internal=ALL-UNNAMED
```

- Replace:
 - Replace <max memory> with the desired memory allocation for DM, as specified in the [Prerequisites](#) section.
 - <path_to_udf_additional_jars> with the directory path where the UDF JAR files are stored.

8.3.3 Appendix: Java Code Examples

Example of a Java implementation of a 'Frac' function:

```
package com.sapiens.bdms.function;

public class Frac {
    public static double frac(int num) {
        return 0.0;
    }

    public static double frac(double num) {
        num = num - (long) num;
        return num = Math.round(num * 1e9) / 1e9;
    }
}
```

Example of a JavaScript function that calls the Java code above:

```
function frac(num) {  
    return com.sapiens.bdms.function.Frac.frac(num);  
}
```

8.4 Customizing External Rule IDs

This document provides information about the customization of External Rule IDs for Rule Family View Rows in Decision.

8.4.1 External Rule IDs

Many Decision assets include a built-in mechanism that calculates the asset hash ID. For Rule Family View Row, the system supports an internal hash ID calculation and an external hash ID generator (also named 'External Rule ID'). The calculation is performed each time the Rule Family or Decision is validated (clicking "Validate") and it is based on the Rule Family View structure and cell logic content.

The client has the option to replace the built-in external rule ID calculation with a customizable External Sign that will remain constant with the Rule Family View Row to provide enhanced support for publishing and tracking of business logic.

With the external rule ID feature, rule IDs are generated for a Rule Family View Row based on a user-defined prefix and sequential number. Users can reset and copy external rule IDs to support synchronization of the same logic to the same external rule ID.

8.4.2 Configuring the External Rule ID

The properties used to configure the external rule ID should be placed in the DM's `application.properties` file.

In that file, edit and add the following properties:

- `enable.external.rule.id=true`

This property enables the External Sign generation to generate a rule ID for the external sign.

- `external.rule.id.prefix={YOUR_PREFIX}{YOUR_SEPARATOR}`

This property defines the system-wide prefix for an external rule ID.

For example, `external.rule.id.prefix=DA-`

This will cause the generated IDs to appear as follows:

"DA-1", "DA-2", "DA-3" ...

The default is "DEC:" which would cause the generated IDs to appear as follows:

"DEC:1", "DEC:2", "DEC:3"...

Note: The prefix also contains the separator (":" by default).

- `external.rule.id.numeric.fixed.size={YOUR_FIXED_SIZE}`

This property sets the size to which the sequential number generated for the external rule ID is padded. The default is 0, which indicates no padding.

For example, `external.rule.id.numeric.fixed.size=8`

Which would cause the generated IDs to appear as follows:

`"DA-00000001"`, `"DA-00000002"`, `"DA-00000003"` ...

Note: If the next sequence contains more digits than the fixed size, it will generate a longer ID (which, in the example above, would result in `"DA-100000000"`, which has 9 digits, and not 8 as set). In addition, if the size is exceeded, the `errors.log` file will record a warning.

8.4.3 Resetting the External Rule IDs for Existing Logic

The Installation Guide includes instructions to reset the external rule ID for logic already entered in the Decision database. Several web services are provided to regenerate external rule IDs and reset a Rule Family View's sign to support rule ID version changes for a Rule Family View.

Note: Running the external rule ID generation web service will reset the external rule IDs for all rows, changing the External Sign associated with each Rule Family View Row. Prior values for the External Sign will be replaced with new values.

To reset the external rule IDs for existing logic:

1. To include conclusion Fact Type names in the logical sign calculation, add the following property to DM's `application.properties` file:

- `sign.logical.conclusion.include = true`

2. Restart the server.
3. Navigate to the application and log in as administrator.
4. Make sure users are out of the system and run the following web services in the same order as presented:

`{DECISION-URL}/ws/rs/administration/signs/recalculate/logical/true`

This will recalculate Rule Family View Row logic. The process should take a short time (a few minutes) to complete and users should not use the system during that time.

`{DECISION_URL}/ws/rs/administration/populate/external/sign/by/logicalSign`

This will generate an external rule ID for each row based on its logic. The process should take a short time (a few minutes) to complete and users should not use the system during that time.

A report presenting which row hash was assigned to which generated external rule ID can be viewed in the `administration.log` file.

`{DECISION_URL}/ws/rs/administration/signs/recalculate/internal`

This process may take several hours (depending on the size of your repository) and users should not use the system during that time.

Check the administration.log file for a message stating that the operation started.

A message should also appear in the file when the process is completed.

8.4.4 Managing External Rule IDs in Decision

The initial external rule IDs will be generated by the external rule ID generation web service with a user-friendly identifier for the External Sign.

The generated external rule ID will be based on the selected Prefix and Padding, as defined in DM's `application.properties` file and discussed above.

Rule Family View Rows that are created and saved will automatically be assigned new and unique external rule IDs.

The external rule ID assigned to a Rule Family View Row will retain its value when copying the Rule Family View to a whiteboard, cloning the Rule Family View or changing the logic or messages of the Rule Family View Row. This means that the external rule ID will be a fixed identifier for the Rule Family View Row until a user explicitly changes the external rule ID.

The external rule ID is set and can be changed only when the Rule Family View is in the whiteboard (Draft status).

The external rule ID of a single Rule Family View Row can be copied to another Rule Family View by clicking the right mouse button on the Sign column cell in the Rule Family View to obtain a context menu that includes 'Copy' and 'Paste'.

To view the Sign, rule ID column and the Generate New Sign option in a Rule Family View, right-click the top left cell and select Show Sign.

	Insured Location Construction	Insured Location Occupied Square Footage	Insured Location Zip-5	Insured Major Location Change
1	Is Not : Previous Construction	-	-	Is : Yes
2	Set Operator Display Mode To Symbol	Is Not : Previous Occupied Square Footage	-	Is : Yes
3	-	-	Is Not : Previous Zip-5	Is : Yes
4	Is : Previous Construction	Is : Previous Occupied Square Footage	Is : Previous Zip-5	Is : No

The Sign column will appear in the table.

	Sign	Insured Location Construction	Insured Location Occupied Square Footage	Insured Location Zip-5	Insured Major Location Change
1	DA-158	Is Not : Previous Construction	-	-	Is : Yes
2	DA-254	-	Is Not : Previous Occupied Square Footage	-	Is : Yes
3	DA-245	-	-	Is Not : Previous Zip-5	Is : Yes
4	DA-19	Is : Previous Construction	Is : Previous Occupied Square Footage	Is : Previous Zip-5	Is : No

Copy the external rule ID from the Rule Family View Row cell that contains the desired external rule ID. Highlight the Sign cell and press <ctrl> + <c> to copy the cell.

SAPIENS Decision						
Home Tasks (15) Repository Reports Administration						
Loan Approval Process > Policy Renewal Method (View: Base) [V1.1] > Insured Major Location Change (View: Base) [V1]						
DRAFT						
Sign	Insured Location Construction		Insured Location Occupied Square Footage		Insured Location Zip-5	
1	DA-158	Is Not : Previous Construction	-		-	
2	DA-254	-	Is Not : Previous Occupied Square Footage		-	
3	DA-245	-	-		Is Not : Previous Zip-5	
4	DA-19	Is : Previous Construction	Is : Previous Occupied Square Footage		Is : Previous Zip-5	
					Is : No	

Then paste the external rule ID in the Sign column cell of the Rule Family View Row that should have the same external rule ID. Highlight the Sign cell to be updated and press <ctrl> + <v> to past the copied cell rule ID into the cell.

SAPIENS Decision						
Home Tasks (15) Repository Reports Administration						
Loan Approval Process > Policy Renewal Method (View: Base) [V1.1] > Insured Major Location Change (View: Base) [V1]						
DRAFT						
Sign	Insured Location Construction		Insured Location Occupied Square Footage		Insured Location Zip-5	
1	DA-158	Is Not : Previous Construction	Is Not : Previous Construction		-	
2	DA-254	-	Is Not : Previous Occupied Square Footage		-	
3	DA-245	Is : Previous Construction	Is : Previous Construction		Is Not : Previous Zip-5	
4	DA-19	Is : Previous Construction	Is : Previous Occupied Square Footage		Is : Previous Zip-5	
					Is : No	

The Rule Family View Row is updated with the copied external rule ID from the other Rule Family Row.

SAPIENS Decision						
Home Tasks (15) Repository Reports Administration						
Loan Approval Process > Policy Renewal Method (View: Base) [V1.1] > Insured Major Location Change (View: Base) [V1]						
DRAFT						
Sign	Insured Location Construction		Insured Location Occupied Square Footage		Insured Location Zip-5	
1	DA-158	Is Not : Previous Construction	Is Not : Previous Construction		-	
2	DA-254	-	Is Not : Previous Occupied Square Footage		-	
3	DA-245	Is : Previous Construction	Is : Previous Construction		Is Not : Previous Zip-5	
4	DA-19	Is : Previous Construction	Is : Previous Occupied Square Footage		Is : Previous Zip-5	
					Is : No	

If the logic included in the Rule Family Row was changed significantly, you may need to generate a new external rule ID for the row to ensure that it is not associated with other Rule Family Rows with the same external rule ID, since it may now represent a different rule.

To generate a new Rule Family View Row external rule ID, right-click the row and select the Generate New Sign option.

The screenshot shows the SAPIENS Decision Manager interface. The top navigation bar includes 'Home', 'Tasks (15)', 'Repository', 'Reports', and 'Administration'. Below the navigation bar, there is a breadcrumb trail: 'Loan Approval Process > Policy Renewal Method (View: Base) [V1.1] > Insured Major Location Change (View: Base) [V1]'. A yellow 'DRAFT' button is visible. The main area contains a table with the following columns: 'Sign', 'Insured Location Construction', 'Insured Location Occupied Square Footage', 'Insured Location Zip-5', and 'Insured Major Location Change'. A context menu is open over the first row, with options: 'Insert', 'Remove', 'Create Diagonal', and 'Generate New Sign' (highlighted in red). The table data is as follows:

Sign	Insured Location Construction	Insured Location Occupied Square Footage	Insured Location Zip-5	Insured Major Location Change
1	Is Not : Previous Construction	Is Not : Previous Construction	-	Is : Yes
2	-	Is Not : Previous Occupied Square Footage	-	Is : Yes
3	Is : Previous Construction	Is : Previous Construction	Is Not : Previous Zip-5	Is : Yes
4	Is : Previous Construction	Is : Previous Occupied Square Footage	Is : Previous Zip-5	Is : No

After you select that option, click OK to confirm the regeneration of sign in the Generate new sign for 'row number' dialog.

The dialog box is titled 'Generate new sign for row 4'. It contains the text 'Are you sure you want to re-generate the sign for this row?'. At the bottom right, there are two buttons: 'Cancel' and 'OK'.

The row will be assigned a new unique external rule ID.

The screenshot shows the SAPIENS Decision Manager interface after the sign has been regenerated for row 4. The table data is as follows:

Sign	Insured Location Construction	Insured Location Occupied Square Footage	Insured Location Zip-5	Insured Major Location Change
1	Is Not : Previous Construction	Is Not : Previous Construction	-	Is : Yes
2	-	Is Not : Previous Occupied Square Footage	-	Is : Yes
3	Is : Previous Construction	Is : Previous Construction	Is Not : Previous Zip-5	Is : Yes
4	Is : Previous Construction	Is : Previous Occupied Square Footage	Is : Previous Zip-5	Is : No

The option can also be used for multiple rows to generate an external rule ID for each selected row. To do so, select the rows you want, right-click in any of the rows you selected and select Generate New Sign.

The screenshot shows the SAPIENS Decision Manager interface with rows 2, 3, and 4 selected. A context menu is open over row 2, with options: 'Insert', 'Remove', 'Create Diagonal', and 'Generate New Sign' (highlighted in red). The table data is as follows:

Sign	Insured Location Construction	Insured Location Occupied Square Footage	Insured Location Zip-5	Insured Major Location Change
1	Is Not : Previous Construction	Is Not : Previous Construction	-	Is : Yes
2	-	Is Not : Previous Occupied Square Footage	-	Is : Yes
3	Is : Previous Construction	Is : Previous Construction	Is Not : Previous Zip-5	Is : Yes
4	Is : Previous Construction	Is : Previous Occupied Square Footage	Is : Previous Zip-5	Is : No

After you select that option, click OK to confirm the regeneration of signs for multiple rows in the Generate new sign for multiple rows dialog.

Generate new sign for multiple rows

Are you sure you want to re-generate the signs for row(s): 2,3,4?

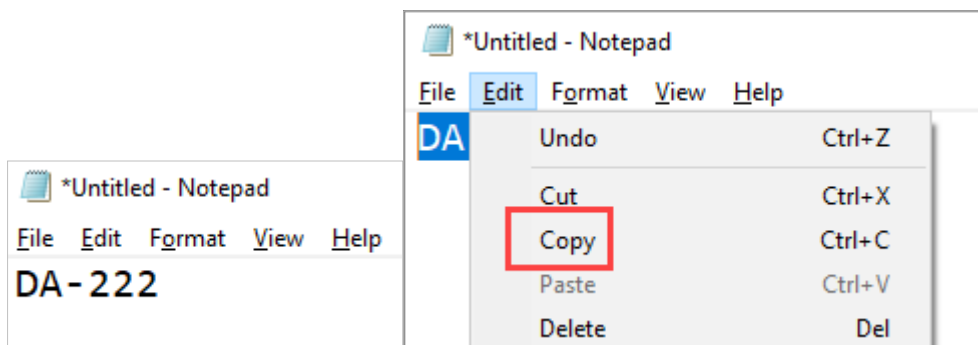
Cancel
OK

Each row that you selected will be given a new external rule ID.

SAPIENS Decision						
Home Tasks (15) Repository Reports Administration						
Loan Approval Process > Policy Renewal Method (View: Base) [V1.1] > Insured Major Location Change (View: Base) [V1]						
DRAFT						
	Sign	Insured Location Construction	Insured Location Occupied Square Footage	Insured Location Zip-5	Insured Major Location Change	
1	DA-256	Is Not : Previous Construction	Is Not : Previous Construction	-	Is : Yes	
2	DA-260	-	Is Not : Previous Occupied Square Footage	-	Is : Yes	
3	DA-261	Is : Previous Construction	Is : Previous Construction	Is Not : Previous Zip-5	Is : Yes	
4	DA-262	Is : Previous Construction	Is : Previous Occupied Square Footage	Is : Previous Zip-5	Is : No	

Values can be entered in the Sign column only by using the Paste option. To manually change the external rule ID, open a notepad application (or other editing tool), write the values you want to enter, copy the values, and paste them in the Rule Family's Sign column.

Copy:



Paste:

SAPIENS Decision						
Home Tasks (15) Repository Reports Administration						
Loan Approval Process > Policy Renewal Method (View: Base) [V1.1] > Insured Major Location Change (View: Base) [V1]						
DRAFT						
	Sign	Insured Location Construction	Insured Location Occupied Square Footage	Insured Location Zip-5	Insured Major Location Change	
1	DA-256	Is Not : Previous Construction	Is Not : Previous Construction	-	Is : Yes	
2	DA-260	-	Is Not : Previous Occupied Square Footage	-	Is : Yes	
3	DA-261	Is : Previous Construction	Is : Previous Construction	Is Not : Previous Zip-5	Is : Yes	
4	DA-262	Is : Previous Construction	Is : Previous Occupied Square Footage	Is : Previous Zip-5	Is : No	

Result:

SAPIENS Decision					
Home		Tasks (15)	Repository	Reports	Administration
Loan Approval Process > Policy Renewal Method (View: Base) [V1.1] > Insured Major Location Change (View: Base) [V1]					
DRAFT					
Sign	Insured Location Construction	Insured Location Occupied Square Footage	Insured Location Zip-5	Insured Major Location Change	
1 DA-256	Is Not : Previous Construction	Is Not : Previous Construction	-	Is : Yes	
2 DA-260	-	Is Not : Previous Occupied Square Footage	-	Is : Yes	
3 DA-261	Is : Previous Construction	Is : Previous Construction	Is Not : Previous Zip-5	Is : Yes	
4 DA-262	Is : Previous Construction	Is : Previous Occupied Square Footage	Is : Previous Zip-5	Is : No	

8.4.5 Validating the External Rule ID

During Rule Family View validation, warnings may appear indicating that logic in a Rule Family View Row is identical to the logic in a row in another Rule Family View, although the two rows have different external rule IDs. If this happens, you can evaluate the logic to determine if the two rows should have the same external rule ID and take appropriate action to synchronize the external rule IDs for the rows.

Examples of warning messages:

- P A row with the same logic has a different External Sign in Rule Family View "{0}", Row Id "{1}", and External Sign "{2}"
- P A row with the same logic has a different External Sign in the unapproved Rule Family View "{0}", Row Id "{1}", External Sign "{2}", Whiteboard "{3}", and User "{4}"

SAPIENS Decision					
Home		Tasks (3)	Repository	Reports	Administration
My Task 1 > Policy Renewal Method (View: Base) [V1.0] > Insured Major Location Change (View: Base) [V1.0]					
DRAFT					
Row ID	Sign	Insured Location Construction	Insured Location Occupied Square Footage	Insured Location Zip-5	Insured Major Location Change
1 1056022	DA-353	Is : Previous Construction	Is : Previous Occupied Square Footage	Is : Previous Zip-5	Is : No
2 1056138	DA-462	Is Not : Previous Construction	-	-	Is : Yes
3 1056032	DA-5555	-	Is Not : Previous Occupied Square Footage	-	Is : Yes
4 1056036	DA-320	-	-	Is Not : Previous Zip-5	Is : Yes

SAPIENS

Decision

Home

Tasks (3)

Repository

Reports

Administration

...

>

Policy Renewal Method (View: FL) [V1.0]

>

Insured Major Location Change (View: FL) [V1.0]

✓

	Row ID	Sign	Hurricane Zoning Change	Insured Location Construction	Insured Location Occupied Square Footage	Insured Location Zip-5	Insured Major Location Change
1	1014804	DA-326	Is : No	Is : Previous Construction	Is : Previous Occupied Square Footage	Is : Previous Zip-5	Is : No
2	1014813	DA-450	Is : Yes	-	-	-	Is : Yes
3	1014816	DA-382	-	Is Not : Previous Construction	-	-	Is : Yes
4	1014819	DA-370	-	-	Is Not : Previous Occupied Square Footage	-	Is : Yes
5	1014810	DA-320	-	-	-	Is Not : Previous Zip-5	Is : Yes

To find all rows with identical external rule IDs, but different logic, use the following SQL command:

```

SELECT DISTINCT
    rr2.id AS ROW_ID,
    rr2.external_sign AS RULE_ID,
    rr2.logical_sign AS LOGICAL_SIGN,
    ftd.latest_friendly_name || ' ' || vn.name || ' ' ||
ast.version_label AS RFV
FROM
    rule_family_row rr2
JOIN rule_family_view rfv
ON rr2.rule_family_view_id = rfv.id
JOIN asset ast
ON ast.id = rfv.id
JOIN fact_type_definition ftd
ON ftd.id = rfv.conclusion_ft_def_id
JOIN view_name vn
ON vn.id = rfv.view_id
WHERE
    rr2.external_sign IN (
        SELECT
            tmp.external_sign
        FROM (
            SELECT
                rr.external_sign AS external_sign,
                rr.logical_sign AS logical_sign
            FROM
                rule_family_row rr
            GROUP BY
                rr.external_sign,
                rr.logical_sign
            ) tmp
        GROUP BY
            external_sign
        HAVING
            COUNT(tmp.external_sign) > 1
        )
ORDER BY
    rr2.external_sign DESC;

```

Input: None

Output:

- Row ID
- Rule Family incl. View and Version
- External ID
- Rule Logic Hash

8.4.6 Locating Assets by their External Rule IDs

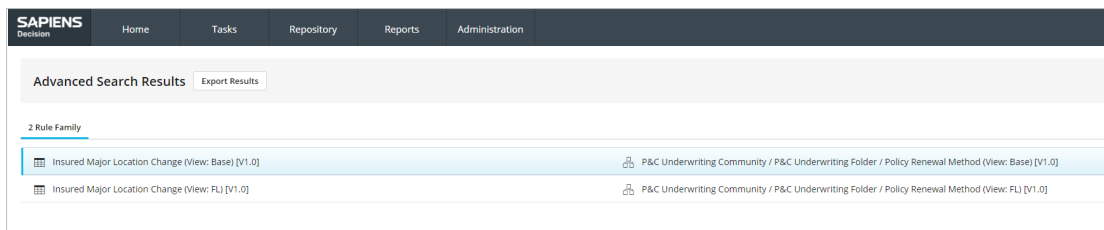
You can use the Advanced Query to locate Rule Families that contain specific external rule IDs.

The screenshot shows the SAPIENS Decision Manager interface. At the top, there's a navigation bar with 'SAPIENS Decision' and tabs for 'Home', 'Tasks', 'Repository', 'Reports', and 'Administration'. A search icon and a user profile icon are on the right. Below the navigation bar, there's a 'Quick Search' section with a text input 'Search asset by name', a 'Search' button, and a checkbox 'Search all Versions'. The 'Advanced Search' section is below, with a toggle switch set to 'ON'. It contains a table with the following structure:

Find	Attributes	Operator	Value
Rule Family	Row Hash	IS	DA-382

At the bottom right of the table, there's a checkbox 'Search all Versions', a 'Clear' button, and an 'Advanced Search' button.

Result:



8.5 Clustering Configuration

8.5.1 Configuring a DM Cluster

DM clustering is an optional configuration. Typically, a load balancer is used to intercept all user requests to DM and distribute them among DM cluster members, enhancing performance through load distribution. The Hazelcast library enables clustering in DM, allowing cluster members to communicate and share data over a high-speed TCP connection bound to specific ports, as defined in your DM configuration.

8.5.1.1 Standard Environment Configuration

To activate and use the DM clustering feature, follow these steps:

1. Enable Clustering in DM Configuration

- Edit the `application.properties` file of the DM application.
- Set the `spring.profiles.active` property to include the value `clustering`.
 - If this is the first time setting the property:
 - `spring.profiles.active=clustering`
 - If the property already has other values:
 - `spring.profiles.active={other values},clustering`

2. Configure Hazelcast Clustering Settings

- In the `application.properties` file of each DM instance (across all servers in the cluster), add the following properties:
 - `hazelcast.instance.name={INSTANCE_NAME}`
 - `hazelcast.cluster.name={CLUSTER_NAME}`
- Set one of the following for port configuration:
 - `hazelcast.port={INSTANCE_PORT}`
 - OR
 - `hazelcast.auto.port.range={PORT_RANGE}`
- Additional cluster settings:
 - `hazelcast.cluster.members.csv={MEMBERS_CSV}`
 - `hazelcast.instance.eviction.policy={EVICTION_POLICY}`

- `hazelcast.instance.max.size.policy={MAX_SIZE_POLICY}`
- `hazelcast.instance.max.size={MAX_SIZE}`

3. Parameter Details:

- `{INSTANCE_NAME}`:
 - A unique identifier for each server in the cluster.
 - If not set, Hazelcast automatically generates a unique UUID for each instance.
 - This name helps identify each DM instance within the cluster, aiding monitoring and troubleshooting.
- `{CLUSTER_NAME}` (Mandatory):
 - The name of your cluster, acting as a unique identifier.
 - Prevents other Hazelcast clusters (from other apps or environments) from accidentally joining your DM cluster.
 - Choose a unique name that will not conflict with other clusters in your network.
- `{INSTANCE_PORT}` or `{PORT_RANGE}`:
 - `INSTANCE_PORT`:
 - It is the specific TCP port that each DM instance will use for cluster communication.
 - Ensure each instance in the cluster uses a unique port to avoid conflicts.
 - `PORT_RANGE`:
 - A range of TCP ports from which Hazelcast can auto-select an available port. (e.g., 7878-8000).
 - If neither is set, Hazelcast defaults to ports 5701-5800.
- `{MEMBERS_CSV}`:
 - A comma-delimited list of hostnames and/or IP addresses, with or without ports, specifying all servers to be included in the cluster. The list must include the host/IP of the DM instance itself. Wildcards such as "*" or "-" are supported for the last digit of the IP address. Any specified host or IP can also include a port number to restrict communication to that specific port.
Example value for `hazelcast.cluster.members.csv`:
 - `hazelcast.cluster.members.csv=machine1,machine2,localhost:5799,192.168.1.0-7,192.168.2.*`
 - An equivalent `hazelcast.cluster.members.csv` property value must be defined in the `application.properties` file of each DM instance in the cluster.

Example Configuration:

If there are two DM instances in the cluster:

- For the DM instance on `machine2`, the property value can be:
`hazelcast.cluster.members.csv=machine1,localhost:5799`
- For the DM instance on `machine1`, the property value can be:
`hazelcast.cluster.members.csv=localhost,machine2:5799`
- Alternatively, both DM instances can use the same value:
`hazelcast.cluster.members.csv=machine1,machine2:5799`
- If a specific port is set for any of the hosts, the corresponding DM instance must also set the `hazelcast.port` property in its `application.properties` file with the same value. If no specific port is specified for a host in the list, Hazelcast will automatically attempt to use ports 5701, 5702, and so on to find a member on that host. In this case, the DM instance must set the `hazelcast.port` property to a value between 5701-5800 or configure the `hazelcast.auto.port.range` property with a range that includes 5701-5800.
- If all DM instances in the cluster are deployed on the same machine, you can choose not to set the `hazelcast.cluster.members.csv` property. In this scenario, any configured `hazelcast.auto.port.range` or `hazelcast.port` settings will be ignored, and Hazelcast will automatically attempt to discover other cluster members on localhost within the default port range of 5701-5800.
- `{EVICTION_POLICY}`:
 - This setting defines the policy used to determine how entries are evicted from Hazelcast's in-memory shared data maps when memory limits are reached. You can choose from several eviction policies depending on your memory management needs.

Value	Description
None	If set, no items will be evicted, and the property max-size will be ignored.
LRU	Least Recently Used.
LFU	Least Frequently Used
- `{MAX_SIZE_POLICY}` and `{MAX_SIZE}`
 - `MAX_SIZE` defines the eviction threshold for the maps. When this threshold is reached according to the `MAX_SIZE_POLICY`, entries in the map are evicted based on the defined eviction policy.
 - For example, if `MAX_SIZE_POLICY` is set to `FREE_HEAP_PERCENTAGE` and `MAX_SIZE` is set to 20, map entries will be evicted when the free heap memory falls below 20%.

- Valid values for MAX_SIZE are integers between 0 and Integer.MAX_VALUE.

You can choose from the available policies based on your memory requirements.

Value	Description	
MAX VALUE	Default value is 0. For maximum size, eviction-policy property must be set to a value other than NONE. Following are the attributes and its descriptions.	
	PER_NODE	Maximum number of map entries in each JVM. This is the default policy.
	PER_PARTITION	Maximum number of map entries within each partition. Storage size depends on the partition count in a JVM. So, this attribute may not be used often. If the cluster is small it will be hosting more partitions and therefore map entries, than that of a larger cluster.
	USED_HEAP_SIZE	Maximum used heap size in megabytes for each JVM.
	USED_HEAP_PERCENTAGE	Maximum used heap size percentage for each JVM. If, for example, JVM is configured to have 1000 MB and this value is 10, then the map entries will be evicted when used heap size exceeds 100 MB.
Note: Following are the default settings for Eviction used by DM: {EVICTON_POLICY}=LRU {MAX_SIZE_POLICY}=FREE_HEAP_PERCENTAGE {MAX_SIZE}=20		

4. Example Configurations:

For example, suppose we have a cluster with two servers:

SERVER1, with IP address 10.245.31.181

SERVER2, with IP address 10.245.31.182

The following is an example of how the file may appear in each server:

In server 1:

hazelcast.instance.name=SERVER1

hazelcast.cluster.members.csv=10.245.31.181:6393, 10.245.31.182: 6395

hazelcast.port=6393

hazelcast.instance.eviction.policy=LRU

```
hazelcast.instance.max.size.policy=FREE_HEAP_PERCENTAGE
hazelcast.instance.max.size=20
```

In server 2:

```
hazelcast.instance.name= SERVER2
hazelcast.cluster.members.csv=10.245.31.181:6393, 10.245.31.182: 6395
hazelcast.port=6395
hazelcast.instance.eviction.policy=LRU
hazelcast.instance.max.size.policy=FREE_HEAP_PERCENTAGE
hazelcast.instance.max.size=20
```

After completing the configuration above, during startup, the bdms.log file should appear as shown below:

```
SERVER1 localhost-startStop-1 29 Oct 2017 18:39:32,667 INFO
Hazelcast DM properties:
hazelcast.instance.name=SERVER1
hazelcast.port=6393
hazelcast.cluster.members.csv=10.245.31.181:6393, 10.245.31.182: 6395
hazelcast.instance.eviction.policy=LRU
hazelcast.instance.max.size.policy=FREE_HEAP_PERCENTAGE
hazelcast.instance.max.size=20
Members [1] {
  Member [10.245.31.181]:6393 - ee0d4294-b85e-4e24-b2ec-7632252e1321
}
```

5. Additional configuration

Generally, any endpoint can reach any server when requesting a service from the load balancer. However, it is possible to force the load balancer to address all of a client's requests using only a single server in the cluster. This is called a 'sticky session'.

To implement a sticky session, you must first define `jvmRoute` in the Tomcat configuration file, which is located in `tomcat/conf/server.xml`, with the same `jvmRoute` properties configured in the load balancer, as follows:

Open the `server.xml` file and replace the following line:

```
<Engine name="Catalina" defaultHost="localhost">
```

with the line:

```
<Engine name="Catalina" defaultHost="localhost" jvmRoute="{JVM_ROUTE}">
```

where:

`{JVM_ROUTE}` is the unique name of the server in the cluster.

For example, using the case of the two servers mentioned above, the configuration may appear as follows:

In server 1:

```
<Engine name="Catalina" defaultHost="localhost" jvmRoute="server1">
```

In server 2:

```
<Engine name="Catalina" defaultHost="localhost" jvmRoute="server2">
```

8.5.1.2 AWS Environment Configuration

This section is applicable only when DM is hosted on AWS.

To activate and use the DM clustering feature in AWS environment, do the following:

1. In DM's `application.properties` file, add the value "clustering" to the `spring.profiles.active` property. This property accepts multiple values, so you may need to append the value "clustering" to it, or create the property if it does not exist.
 - For example, if the `spring.profiles.active` property is being set for the first time:
 - `spring.profiles.active=clustering`
 - If the property already contains other values:
 - `spring.profiles.active={other values},clustering`
2. Add the following mandatory properties in the DM's `application.properties` file
 - `hazelcast.aws.access.key` = AWS account user
 - `hazelcast.aws.access.secret` = AWS account password
 - `hazelcast.aws.region` = AWS account region
 - `hazelcast.aws.eth0.ip` = Ethernet IP address
3. To reduce the members in the cloud down to only Hazelcast members, set the following parameters as you specified in the EC2 console. The following properties are optional.
 - `hazelcast.aws.tag.key` = AWS tag key
 - `hazelcast.aws.tag.value` = AWS tag value
 - `hazelcast.aws.host.header` = The URL that is the entry point for a web service.

8.5.1.3 Hazelcast Configuration Properties for Clustering

The following table provides a comprehensive overview of the key Hazelcast configuration properties used in a DM. These properties are configured in the DM's `application.properties` file or a YAML file (`application.yml`),

Name	Valid Values	Default Value	Description
<code>hazelcast.auto.port.range</code>	Numbers range (dash separated)		Defines a range of ports that Hazelcast can use to automatically identify available ports for binding, and for discovering other Hazelcast members in the cluster. Example: "5701-5800".
<code>hazelcast.aws.access.key</code>	Any value		Specifies the AWS access key for authentication when connecting to AWS services. Relevant for Hazelcast cluster in AWS platform.

Name	Valid Values	Default Value	Description
hazelcast.aws.access.secret	Any value		Specifies the AWS secret access key for authentication when connecting to AWS services. Relevant for Hazelcast cluster in AWS platform.
hazelcast.aws.eth0.ip	Valid IP value		Specifies the Ethernet (eth0) IP address for the Hazelcast cluster member. Relevant for Hazelcast cluster in AWS platform.
hazelcast.aws.host.header	Valid URL		Specifies the URL used as the entry point for a web service within AWS. Relevant for Hazelcast cluster in AWS platform.
hazelcast.aws.legacy.discovery	true, false	false	Enables or disables the use of legacy cluster member discovery in AWS instead of the discovery SPI. Relevant for Hazelcast cluster in AWS platform.
hazelcast.aws.region	Any value		Specifies the AWS region where the Hazelcast cluster is deployed. Relevant for Hazelcast cluster in AWS platform.
hazelcast.aws.tag.key	Comma-separated string values		Specifies the EC2 instance tag keys to filter for when identifying Hazelcast cluster members. Relevant for Hazelcast cluster in AWS platform.
hazelcast.aws.tag.value	Comma-separated string values		Specifies the EC2 instance tag values to filter for, corresponding to the keys in "hazelcast.aws.tag.key". Relevant for Hazelcast cluster in AWS platform.

Name	Valid Values	Default Value	Description
hazelcast.azure.client.id	Any value		Specifies the Azure Active Directory Service Principal client ID. Relevant for Hazelcast cluster in Azure platform.
hazelcast.azure.client.secret	Any value		Specifies the Azure Active Directory Service Principal client secret. Relevant for Hazelcast cluster in Azure platform.
hazelcast.azure.instance.metadata.available	true, false	false	Indicates if instance metadata can be fetched and used instead of manually defined Azure properties. Relevant for Hazelcast cluster in Azure platform.
hazelcast.azure.resource.group	Any value		Specifies the Azure resource group where the Hazelcast instance is running. Relevant for Hazelcast cluster in Azure platform.
hazelcast.azure.scale.set	Any value		Specifies the Azure VM scale set name, restricting cluster members to instances within this scale set. Relevant for Hazelcast cluster in Azure platform.
hazelcast.azure.subscription.id	Any value		Specifies the Azure subscription ID associated with the Hazelcast instance. Relevant for Hazelcast cluster in Azure platform.
hazelcast.azure.tag	Any value=Any value		Specifies a key-value pair tag on Azure network interfaces (NIC). Only instances with this tag can join the cluster. Relevant for Hazelcast cluster in Azure platform.

Name	Valid Values	Default Value	Description
hazelcast.azure.tenant.id	Any value		Specifies the Azure Active Directory tenant ID. Relevant for Hazelcast cluster in Azure platform.
hazelcast.cluster.members.csv	Comma-separated host values		Specifies a comma-separated list of hostnames or IP addresses of Hazelcast members. Wildcards ("*" or "-") can be used for IP ranges. Ports can be specified for each host. Example: machine1,machine2,machine3:5799,192.168.1.0-7,192.168.2.*
hazelcast.cluster.name	Any value		Specifies a unique name for the Hazelcast cluster to which this application instance belongs, ensuring proper identification in multi-cluster environments.
hazelcast.instance.eviction.policy	LRU, LFU, None	LRU	Defines the eviction strategy for Hazelcast memory cache: LRU (Least Recently Used), LFU (Least Frequently Used), or None (no eviction).
hazelcast.instance.max.size.policy	FREE_HEAP_PERCENTAGE, PER_NODE	FREE_HEAP_PERCENTAGE	Determines the eviction threshold policy for Hazelcast: FREE_HEAP_PERCENTAGE specifies a minimum percentage of free heap space, while PER_NODE limits the maximum number of map entries per cluster member.
hazelcast.instance.max.size.value	Integer	20	Specifies the threshold value for eviction. For example, with FREE_HEAP_PERCENTAGE set to 20, map entries will be evicted if the free heap size falls below 20%.
hazelcast.instance.name	Any value	Random generated UUID	Specifies a human-readable name for this Hazelcast instance. If not defined, a randomly generated UUID is assigned.

Name	Valid Values	Default Value	Description
hazelcast.kubernetes.api.retries	Integer		Specifies the number of retry attempts if there are connection issues with the Kubernetes API. Relevant for Kubernetes API mode.
hazelcast.kubernetes.api.token	Any value		Specifies the API token used to authenticate with the Kubernetes API. Relevant for Kubernetes API mode.
hazelcast.kubernetes.ca.certificate	Any value		Specifies the CA certificate for secure communication with the Kubernetes API. Relevant for Kubernetes API mode.
hazelcast.kubernetes.master	Any valid URL		Specifies the URL of the Kubernetes Master server used for cluster management. Relevant for Hazelcast cluster in Kubernetes platform when working in API mode.
hazelcast.kubernetes.namespace	Any value		Specifies the Kubernetes namespace where the Hazelcast cluster is deployed. This is relevant when using the Kubernetes API mode for Hazelcast cluster discovery.
hazelcast.kubernetes.pod.label.name	Comma-separated string values		Specifies the label names used to identify Pods that form the Hazelcast cluster. Multiple values are allowed, separated by commas. Must have the same count as "hazelcast.kubernetes.pod.label.value". Relevant for Kubernetes API mode.

Name	Valid Values	Default Value	Description
hazelcast.kubernetes.pod.label.value	Comma-separated string values		Defines the label values used to tag Pods that form the Hazelcast cluster. Multiple values are allowed, separated by commas. Must match the number of elements in "hazelcast.kubernetes.pod.label.name". Relevant for Kubernetes API mode.
hazelcast.kubernetes.resolve-not-ready-addresses	true, false	false	If set to true , includes addresses of Pods that are not in a ready state while discovering the Hazelcast cluster. Relevant for Kubernetes API mode.
hazelcast.kubernetes.service.dns.timeout	Integer		Sets the timeout duration (in seconds) for DNS lookup while discovering cluster members. Relevant for Hazelcast cluster in Kubernetes platform when working in DNS mode.
hazelcast.kubernetes.service.dns.value	Any value		Specifies the DNS name of the service used to identify cluster members by their IPs. Relevant for Hazelcast cluster in Kubernetes platform when working in DNS mode.
hazelcast.kubernetes.service.label.name	Comma-separated string values		Specifies the service label names used to identify services forming the Hazelcast cluster. Multiple values are allowed, separated by commas. This is relevant for Kubernetes API mode.
hazelcast.kubernetes.service.label.value	Comma-separated string values		Specifies the values corresponding to the service label names defined by "hazelcast.kubernetes.service.label.name". Multiple values are allowed, separated by commas. Relevant for Kubernetes API mode.

Name	Valid Values	Default Value	Description
hazelcast.kubernetes.service.name	Any value		Defines the name of the Kubernetes service used to filter the Pods that should join the Hazelcast cluster. If not specified, all Pods in the namespace are considered. Relevant for Kubernetes API mode.
hazelcast.kubernetes.service.port	Valid port number		Specifies the port number used by Hazelcast members for communication. Relevant for Hazelcast cluster in Kubernetes platform when working in DNS mode.
hazelcast.partition.group.type	HOST_AWARE, CUSTOM, PER_MEMBER, ZONE_AWARE, NODE_AWARE, PLACEMENT_AWARE, SPI		Defines the partition group strategy used to minimize data loss by controlling partition distribution across cluster nodes. Relevant for Hazelcast cluster in Kubernetes platform when working in API mode.
hazelcast.port	Valid port number		Specifies the port number on which the application should bind for Hazelcast communication.
hazelcast.sticky-session	true, false	false	Enables sticky-session mode, ensuring all requests with the same session ID are routed to the same instance in a load-balanced environment. Set this to true if the load balancer is configured for sticky sessions.

8.6 JNDI Configuration (Tomcat Data Source)

Prerequisites to configure JNDI (Oracle Datasource)

- Remove old version of ojdbc8-~~ojdbc8~~.jar from the tomcat/lib/ directory.
- Copy ojdbc8-21.7.0.0.jar from dm/WEB-INF/lib to tomcat/lib/ directory.

8.6.1 JNDI Configuration – DM

The following instructions are for embedded Tomcat data source configuration in **All-in-One** installations.

1. Define the data source in tomcat's context.xml file.

Add a `<Resource>` tag in the context.xml file. Note that each data source has its own properties, and the attributes specified in the `<Resource>` tag should be made according to the specific data source.

For example, the standard data source (`javax.sql.DataSource`) can be configured according to the following specifications, while the data source used in Decision `com.zaxxer.hikari.HikariDataSource` should be configured according to [the specifications](#) provided in the appendix.

2. Add the following lines to DM's application.properties file:

- `spring.datasource.jndi-name = jdbc/TestDB`
- `datasource.class.name = javax.sql.DataSource`

The `jdbc/TestDB` parameter should be the JNDI name for the datasource as configured in the tomcat.

The `datasource.class.name` parameter defines the class that will be used as a data source. If this property does not exist, the default `com.zaxxer.hikari.HikariDataSource` will be used.

8.6.2 JNDI Configuration – DT

Important Note: The JNDI configuration for DT is only possible in Oracle.

To configure JNDI support for the DT components, add the following:

1. *Configure Tomcat's context.xml with the values given below:*

```
<Resource name="MyLocalDB"
auth="Container"
user="<db UserName>"
password="<db Password>"
url="jdbc:oracle:thin:@//<host>:<port>/<db>"
type="oracle.jdbc.pool.OracleDataSource"
driverClassName="oracle.jdbc.OracleDriver"
factory="oracle.jdbc.pool.OracleDataSourceFactory"
maxActive="100"
maxIdle="20"
minIdle="5"
```

```
maxWait="10000"/>
```

2. Verify if `ojdbc.jar` is available in Tomcat's lib folder. If not, copy it from the <decision runtime package>/app/tomcat/lib/ directory.
3. Add the property "repo.datasource.jndi.name" in DT Repository application.properties file. The value of this property will be same as "name" attribute of JNDI resource. In this case the value will be "MyLocalDB".
4. Note that there are no changes in the following properties:
 - `type="oracle.jdbc.pool.OracleDataSource"`
 - `driverClassName="oracle.jdbc.OracleDriver"`
 - `factory="oracle.jdbc.pool.OracleDataSourceFactory"`

8.6.3 Configuring Internal Thread Pool Mechanism

This section details how to configure the internal thread pool mechanism in DM.

By default, DM will be configured to have a pool with a max size of 1000 threads. The default value is a suggestion for standard deployment and can be set based on the requirements according to machine memory resources.

To change the default, modify the size of the below-mentioned property in DM's application.properties file:

- `bdms.thread.pool.size`

For example, `bdms.thread.pool.size=5000`

8.7. Authorization and Authentication

The following sections provide authorization and authentication instructions that can be performed after the installation is complete.

8.7.1 Configuring LDAP for DM

As described on the LDAP (Lightweight Directory Access Protocol) web page (<https://www.ldap.com/getting-started-with-ldap>): "LDAP is an open, standards-based, mature, efficient, extensible, and popular means of interacting with the data contained in directory servers. It is a binary (i.e., not text-based) protocol that uses TCP/IP for transport."

DM supports LDAP for both authorization and authentication.

By default, authorization management is provided by an embedded control, which allows the assignment of roles to users, and then a second level of control that provides assignment of a given user with a given role to hierarchical directories (Communities, ViewGroups).

Replacing authorization control with LDAP authorization

The authorization control can be replaced by LDAP authorization, and to do so, the following steps need to be executed.

1. For authorization, an LDAP group needs to be defined for every combination of Community-Role.

The name of the group should match a concatenation of the community name and role name, separated by a configured delimiter, plus one for each 'global' role (a role that holds global permissions, such as administration access reports, etc., and is not restricted by a concept of ViewGroups).

2. Open DM's `application.properties` for editing:

add the value "ldap-authentication" to "spring.active.profiles" property value ("spring.active.profiles" is a multi-value property) and make sure no other authentication type is part of the value list:

`spring.profiles.active=ldap-authentication,< ...possibly other values...>`

3. LDAP password can be plain or encrypted.

- a. For plain text set the property in DM's `application.properties`:

`decision.dm.ldap.password={ldap_plain_password}`

- b. For encrypted password:

Generate encrypted password for LDAP using the decision encryptor. The `decision-encrypt-1.0.jar` can be found in `Decision-Encryptor-1.0.tar.gz`

Run `decision-encrypt-1.0.jar` in the CMD line with two parameters:

- i. The algorithm for encryption, use "RSA".
- ii. The password to encrypt.

Example for Windows: `java -jar decision-encrypt-1.0.jar RSA password`

Example for Linux: `java -jar decision-encrypt-1.0.jar 'RSA' 'password'`

The Jar output should be:

Entered Text - Validate that is equal to the password entered.

Encryption Public Key- Only for reference.

Encrypted text.

- iii. Set the encrypted password in DM's `application.properties`:

`decision.dm.ldap.password=ENC:{encrypted_ldap_password}`

4. Enter the following properties and settings in either `externalSecurity.properties` or `application.properties` file of DM:

- **decision.dm.ldap.url**

Your LDAP URL.

For example, `decision.dm.ldap.url=ldap://sapiens.int:636/dc=sapiens,dc=int`

- **decision.dm.ldap.userName**

Username to be used when querying LDAP.

For example, `decision.dm.ldap.userName=sapiens\john.doe`

Or: full user DN(`dc=sapiens,dc=int,uid=john.doe`)

Note: This user must also exist in DM for the authentication to pass.

- **decision.dm.ldap.group.search.dn**
The group search hierarchy to filter by.
For example, decision.dm.ldap.group.search.dn=OU=BDMS_test,OU=Pcsupport,OU=Playground
- **decision.dm.ldap.user.search.query={sAMAccountName={0}}**
DM user search query DN.
- **idp.failOnError=true**
If true, LDAP will fail if an error occurs.
- **idp.group.splitterSign**
A delimiter between the community name and the role name concatenated in the group name.
- **idp.group.prefix**
A prefix used to separate DM groups from others.
For example, idp.group.prefix=DECISION_
- **idp.group.suffix**
A suffix used to separate DM groups from others.
For example, idp.group.suffix=_ANY_SUFFIX
- **idp.group.none.viewgroup.name**
A name for global roles defined in DM (roles that are not defined in any community). The name must not match any ViewGroup name defined in DM.
For example, idp.group.none.viewgroup.name=global_non_vg
- **idp.group.role.none.name**
Set this property to 'none' for the LDAP role name to represent no name when assigning a user to a Community without a role.
For example, idp.group.role.none.name=none
- **idp.group.role.inherit.name**
Set this property to 'inherit' or 'allow' for the role name to indicate that all roles for the user from the next upper-level Community will be inherited when assigning the user to the Community.
For example, idp.group.role.inherit.name=allow
- **idp.group.dependency.group.role.delimiter**
Set this property to '#' for the delimiter to indicate that the role cannot be granted unless the user was authorized for the Community.
For example, idp.group.dependency.group.role.delimiter=#
- **decision.dm.ldap.case.ignore.vg.role**
When set to true, a match for the LDAP group would not have to be case sensitive.
For example, decision.dm.ldap.case.ignore.vg.role=true

- **decision.dm.ldap.user.email.attribute**

Used to configure which attribute on LDAP stores the user's email address, so the mail can be synchronized with DM.

For example, `decision.dm.ldap.user.email.attribute=email`

- **decision.dm.ldap.user.display.name.attribute**

Used to configure which attribute on LDAP stores the user's display name, so the name can be synchronized with DM.

- **decision.dm.ldap.must.have.role.for.vg.role**

When set to true, in order for a user to have a role defined in a certain community, the role would first need to be defined in the corresponding global group.

For example, `decision.dm.ldap.must.have.role.for.vg.role=false`

8.7.2 LDAP enhancements for private Communities

LDAP (Lightweight Directory Access Protocol), which works with DM to coordinate security operations and control access to the system, was enhanced to synchronize with and support DM's Community-level security.

To support the private Communities option, changes were made to enhance the LDAP-DM interface. Locally defined Communities enable DM administrators to create private Communities with specific users, and changes in the interface allow the LDAP server to make use of the new Community administration enhancements.

The latest LDAP-DM interface changes include the following:

delimiter

The # Delimiter dependency is designed to prevent users from being assigned roles in a Community for which they do not have access permission. Since Community access and Community role assignment are generally separate procedures, the # dependency will block a user from receiving a role in a Community if the user did not gain access to the Community from a separate group.

Example:

If 'Private' is a locally defined Community, `BDMP_Private&BDMS Administrator#_gg` is a group assigning users with the 'BDMS Administrator' role, only if the users were granted access to the 'Private' Community separately via another group, such as `BDMP_Private&BCD Manager_gg`.

If 'User1' is defined in `BDMP_Private&BDMS Administrator#_gg` and in no other group, that user will be blocked from accessing the Community and from being assigned the role. In addition, the synchronization process will generate a warning message to inform the DM administrator that the user was not granted the role.

'none' keyword for role parameter

The new option 'none' can be set for a group to block role assignment. Adding users to a group with the 'none' option allows access to the Community, but with no role assigned to the user.

Example:

The group BDMP_WORLD&none_gg will add users to the WORLD Community without assigning them roles.

‘allow’ keyword for role parameter

The new option ‘allow’ can be set for a group to inherit roles from the parent of a locally-defined Community. Adding users to a group with the ‘allow’ option inherits all assigned roles from the parent Community.

Example:

‘User2’ is defined in the WORLD Community with the Decision Analyst, BDMS Administrator and BCD Manager roles. ‘Private’ is a direct child of the World Community. If ‘User2’ is added to the BDMP_Private_allow_gg group, the user will be assigned all roles inherited from the parent Community.

This will avoid the need to define a separate group for each role in a locally-defined Community.

8.7.3 SAML Single sign-on (SSO) Authentication

Single sign-on (SSO) is an authentication method where one set of credentials (For example, ID and password) is used to access multiple applications, including Decision, by passing the authentication token seamlessly to configured applications.

The SSO authentication can be accomplished using one or more Identity Providers (IDP) such as Active Directory Federated Services (ADFS), PingFederate, Auth0, Okta and so on.

Notes

- You can define more than one IDP. In this case, the user will be prompted to select the relevant IDP when a login is required.
- When implementing SSO with SAML, different identity managers might have different implementation concepts. Follow all the procedures as described in the Configuration Guide and plan appropriate time and testing into any implementation project. In case of any issues or queries on SAML implementation, contact Sapiens Decision team.
- When you are authenticating via an Identity Provider (IDP), the IDP must support Security Assertion Markup Language (SAML) 2.0.

Tip: The Security Assertion Markup Language (SAML) standard defines how providers can offer both authentication and authorization services.

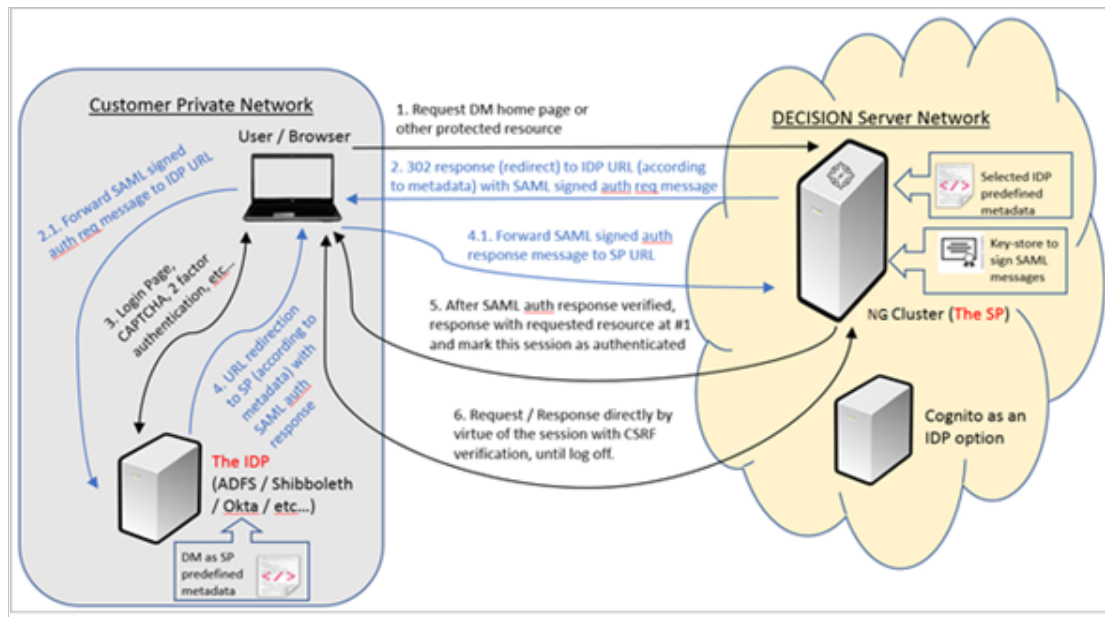
For more details on the SSO terms and definitions, refer to the Glossary section.

Advantages of implementing SSO in DM

- Mitigate risk for access to 3rd-party sites (user passwords not stored or managed externally).
- Reduce password fatigue from different username and password combinations.
- Reduce time spent re-entering passwords for the same identity.
- Automatically follow your corporation’s identity and password standards as implemented in your identity manager.

8.7.3.1 Configuring DM for SSO authentication

DM accepts the SSO authentication from an IDP via SAML 2.0 by using “Spring Security SAML Extension”. For more information on the authentication process, refer to the [Spring Security SAML documentation](#).



Architecture of authentication process

DM Configuration

Prerequisite: When the selected IDP is ADFS, DM must be configured to use HTTPS.

For more details on how to enable HTTPS, refer to HTTPS Setup for bundle Installation.

You can configure the application to retrieve the SAML metadata either from a file or dynamically from a URL. To do this, set either the **metaDataFileName** or **metaDataUrl** property. If you set the **metaDataUrl** property, the application retrieves the identity provider (IDP) certificate from the specified URL. The server then uses the certificate to authenticate users.

You can obtain the correct **metaDataUrl** value from your identity provider (IDP) server.

Note: If both **metaDataUrl** and **metadataFileName** are configured, the URL (**metaDataUrl**) takes precedence.

To accommodate certificate expiration, use the **metaDataUrl** to eliminate the need for manual updates to file contents.

Configuring Basic SSO

Open DM's `application.properties` file and add the following properties:

- Add the value `saml-authentication` to the `spring.profiles.active` property (a multi-value property) and ensure that no other authentication type is included in the value list.
`spring.profiles.active=saml-authentication,< ...possibly other values...>`
- `decision.saml2.idp.registration.info` should be used to configure Single and Multi IDP by given format `{ "entityId" : "idp entity id", "metaDataFileName": "saml2-`

metadata","type":"PRIVATE or PUBLIC","displayName":"multiple IDP selection page display name.", "metaDataUrl": "<Metadata URL from your IDP server>"]}

Setting Single IDP with IDP Metadata file

- Copy the metadata file and paste it as <filename> in the DM classpath location.
- Open DM's application.properties file and add the following property:

```
decision.saml2.idp.registration.info=[{\
    "entityId": "<IDP Entity ID configured>",\
    "metaDataFileName": "<IDP Metadata file Name>",\
    "type": "<PUBLIC,PRIVATE>",\
    "displayName": "<your Display name>",\
    "registrationId": "<Service provider id configured>"\
}]
```

For example,

```
decision.saml2.idp.registration.info=[{\
    "entityId" : "jumpcloud-dev-local-cluster",\
    "metaDataFileName": "jump-cloud-saml2-metadata-local.xml",\
    "type": "PUBLIC",\
    "displayName": "Jump Cloud",\
    "registrationId": "jumpcloud-dev"\
}]
```

Note: It is mandatory to configure the type as “PUBLIC” if you are configuring a single IDP.

Setting Single IDP with Metadata URL

In DM's application.properties file, add the metadata URL as a value to the following property:

```
decision.saml2.idp.registration.info=[{\
    "entityId": "<IDP Entity ID configured-1>",\
    "metaDataUrl": "<Metadata URL from your IDP server>"\
    "type": "<PUBLIC,PRIVATE>",\
    "displayName": "<your Display name-1>",\
    "registrationId": "<Service provider id configured-2>"\
}]
```

For example,

```
decision.saml2.idp.registration.info=[{\
    "entityId" : "jumpcloud-dev-local-cluster",\
    "metaDataUrl":\
    "https://sso.jumpcloud.com/saml2/metadata/652c3099d5be9f9200929316",\
    "type": "PUBLIC",\
    "displayName": "Jump Cloud",\
    "registrationId": "jumpcloud-dev"\
}]
```

Setting Multiple IDPs - IDP Metadata

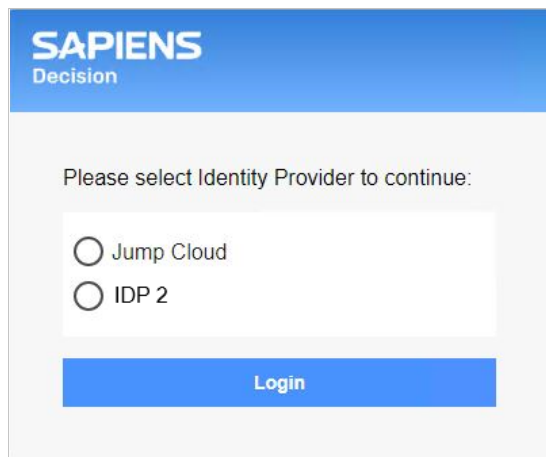
- Place metadata file(s) in a directory that is in the DM classpath.
- Separate metadata per IDP is mandatory; it won't function if all metadata files are combined and generate a single metadata file, like in previous versions.
- In DM's `application.properties` file, add the metadata file name as a value to the following property:

```
decision.saml2.idp.registration.info=[{\
    "entityId": "<IDP Entity ID configured-1>",\
    "metaDataFileName": "<IDP Metadata file Name-1>",\
    "type": "<PUBLIC,PRIVATE>",\
    "displayName": "<your Display name-1>",\
    "registrationId": "<Service provider id configured-2>"\
},{\
    "entityId": "<IDP Entity ID configured-2>",\
    "metaDataFileName": "<IDP Metadata file Name-2>",\
    "type": "<PUBLIC,PRIVATE>",\
    "displayName": "<your Display name-2>",\
    "registrationId": "<Service provider id configured-2>"\
},{\
    "entityId": "<IDP Entity ID configured-n>",\
    "metaDataFileName": "<IDP Metadata file Name-n>",\
    "type": "<PUBLIC,PRIVATE>",\
    "displayName": "<your Display name-n>",\
    "registrationId": "<Service provider id configured-n>"\
}]
```

Example,

```
decision.saml2.idp.registration.info=[{\
    "entityId" : "jumpcloud-dev-local-cluster",\
    "metaDataFileName": "jump-cloud-saml2-metadata-local.xml",\
    "type": "PUBLIC",\
    "displayName": "Jump Cloud",\
    "registrationId": "jumpcloud-dev"\
},{\
    "entityId" : " idp.2.com ",\
    "metaDataFileName": " idp2com -metadata.xml",\
    "type": "PUBLIC",\
    "displayName": " IDP 2",\
    "registrationId": " idp-dev"\
}]
```

when the property is set as in the example the following landing page is displayed:



Multiple IDPs - IDP Selection Prompt Page

If more than one of the IDPs are set as “PUBLIC”, user is automatically presented with an IDP selection prompt page. Each “PUBLIC” IDP option is displayed with the `idp-display` string as mentioned in the previous configuration section.

To access the application home page, when an IDP is defined as “PRIVATE”, it can be used only as an embedded parameter in the URL. the relevant IDP `entityId` should be passed as a query parameter called `idp`, use the following:

`http://<DM host>:<port>/dm/enterprise?idp=<entityId>`

For example, `http://127.0.0.1:9080/dm/enterprise?idp=jumpcloud`

To login to application using configured Display name and entity ID, the URL can be invoked in below format.

`http://<DM host>:<port>/dm/idpSelection.jsp?idp_registrationId=jumpcloud-sp-local`

`http://<DM host>:<port>/dm/idpSelection.jsp?idp_name=Jump%20Cloud`

DM SAML KeyStore configuration

KeyStore Generation

This section is not applicable if the selected IDP is ADFS.

Note: If DM is configured with HTTPS or the IDP is ADFS, you can skip the KeyStore generation as you already have Tomcat's KeyStore. The same KeyStore with the same key name and password used in Tomcat's “server.xml” can be used.

A Java KeyStore (JKS) needs to be generated and configured for DM to digitally sign the SAML messages. The Java “keytool” command line application is used to generate the JKS. For more details, refer to the [Java keytool documentation](#).

The default command to use with keytool:

```
keytool -genkeypair -alias <key name here> -keypass <key password here> -keystore <file path to save key store here>
```

This command generates a KeyStore with the specified key name and key password using DSA 1024 bit as the encryption algorithm.

To change to another algorithm, add the following parameter “-keyalg” with supported algorithms. For example. “-keyalg RSA”.

Note: When using a DM clustered environment, the KeyStore should be generated once and the same KeyStore file must be used in each DM instance.

Warning: You should not create KeyStore for each instance separately. If created separately, it might not work as intended.

KeyStore DM Configuration

- Copy the KeyStore file and paste it in the DM classpath location,

Note: In case of Tomcat HTTPS, copy the Tomcat KeyStore file used for Tomcat HTTPS and paste it in DM classpath.

- Open DM's `application.properties` file and add the following properties:

- **`decision.dm.saml2.sp.keystore.filename=<keystore file name>`**

where `<filename>` is the generated KeyStore file name without folder path.
For example: `"saml_keystore.jks"`.

- **`decision.dm.saml2.sp.keystore.password=<keystore password>`**

`decision.dm.saml2.sp.keystore.alias=<keystore alias name>`

where `<key password>` and `<key name>` are the key name and password that were set during KeyStore generation.

OR

- For enhanced security, set the `*encrypted*` key name and password values that were encrypted using the "decision-encrypt" tool with the RSA algorithm. Once encrypted, configure the same properties with the `ENC:` prefix:

`decision.dm.saml2.sp.keystore.password=ENC:<encrypted keystore password>`

`decision.dm.saml2.sp.keystore.alias=ENC:<encrypted keystore alias name>`

8.7.3.2 Customize DM SSO configurations

You can customize the SSO configuration setup based on your security requirements. It is not recommended to modify the default configuration which might result in non-standard requirement of the IDP or an unusual security requirement.

To customize the SSO configuration, perform the following:

Open DM's `application.properties` file and modify the following properties:

Property to be added	Description
decision.saml2.idp.signing.algorithm=<https://www.w3.org/2000/09/xmldsig#rsa-sha1>	- Set as http://www.w3.org/2000/09/xmldsig#rsa-sha1 to configure DM to use SHA1 algorithm for signature on sent requests and reception of response. - Set as http://www.w3.org/2001/04/xmldsig-more#rsa-sha256 to configure DM to use SHA256 algorithm for signature on sent requests and reception of response. Note: By default, the configuration value is set as https://www.w3.org/2000/09/xmldsig#rsa-sha1
decision.saml2.idp.authn.requests.signed=<false>	- Set as true if you want the signed authentication requests to be sent to IDP. - Set as false if you do not want the signed authentication requests to be sent to IDP. Note: By default, the configuration value is set as true
decision.saml2.idp.response.time.validation.slew=<300>	It Except value in integer and represent in seconds. Note: By default, the configuration value is set as 300 sec
decision.saml2.idp.authn.requests.signed=false	- Set as true to enable SSO with signed SAML requests. - Set as false if you do not want the metadata of the IDP to be digitally signed. Note: By default, the configuration value is set as false.

Property to be added	Description
decision.saml2.sp.authn.requests.signed=false	- Set as true to enable SSO with signed SAML requests. - Set as false if you do not want the metadata of the IDP to be digitally signed. Note: By default, the configuration value is set as false.

8.7.3.3 IDP Configuration

To establish SAML communication, you must add the DM SAML metadata for each IDP. For more details on how to add the DM SAML metadata for the IDP, refer to [Setting IDP Metadata](#).

An example of how to add the DM SAML metadata for IDPs of types ADFS or Okta is shown below. Any SAML2 protocol compliant IDP should work.

Once SSO configuration of DM is completed, start the DM server.

ADFS

To configure ADFS on DM, perform the following:

1. Ensure DM is up and running configured to use HTTPS.
2. Navigate to "{DECISION-URL}/saml2/service-provider-metadata/{registrationId}" and download DM metadata and save the file as "DM-meta.xml".
3. Open ADFS 2.0 Management Console and select "Add Relying Party Trust".
4. Select "Import data about the relying party from a file" option and select "DM-meta.xml" which was created in step 3 and click Next.
A pop-up window appears with the following error message "The content of metadata is not supported". Ignore the error and proceed further.
5. Click on "Ready to Add Trust" and ensure the tab endpoints contains multiple endpoint values.
Note: If the endpoints do not have multiple endpoint values, verify the DM configuration, and ensure the DM configuration is set with HTTPS and "DM-meta.xml" is generated with HTTPS protocol URLs.
6. Select the "Open the Edit Claim Rules dialog" check box and click Finish.
7. Select "Add Rule"> "Send LDAP Attributes as Claims" and click Next.
8. In the Add NameID field, enter "Claim rule name".
9. In the Attribute store field, select "Active Directory".
10. In the LDAP Attribute field, select "SAM-Account-Name".
11. In the Name ID field, select "Outgoing claim type".
Note: If you are using ADFS 3.0 version, you must configure the Name ID as a Pass Through claim.
Open the provider and select Advanced tab and change "Secure hash algorithm" to SHA-1

12. In the claim rules window, click Finish.
The IDP type for SSO configuration of ADFS is configured.

Okta

To set the Okta as the IDP type for SSO configuration, perform the following:

1. Login to Okta as an administrator.
2. In the Home page, select Admin > Applications > Create New App.
3. Choose SAML 2.0 from the list of supported protocols and click Create.
4. Enter the application name. For example, Decision Manager
5. Select the application image. This step is optional.
6. Click Next.
7. Configure SAML with the following settings:

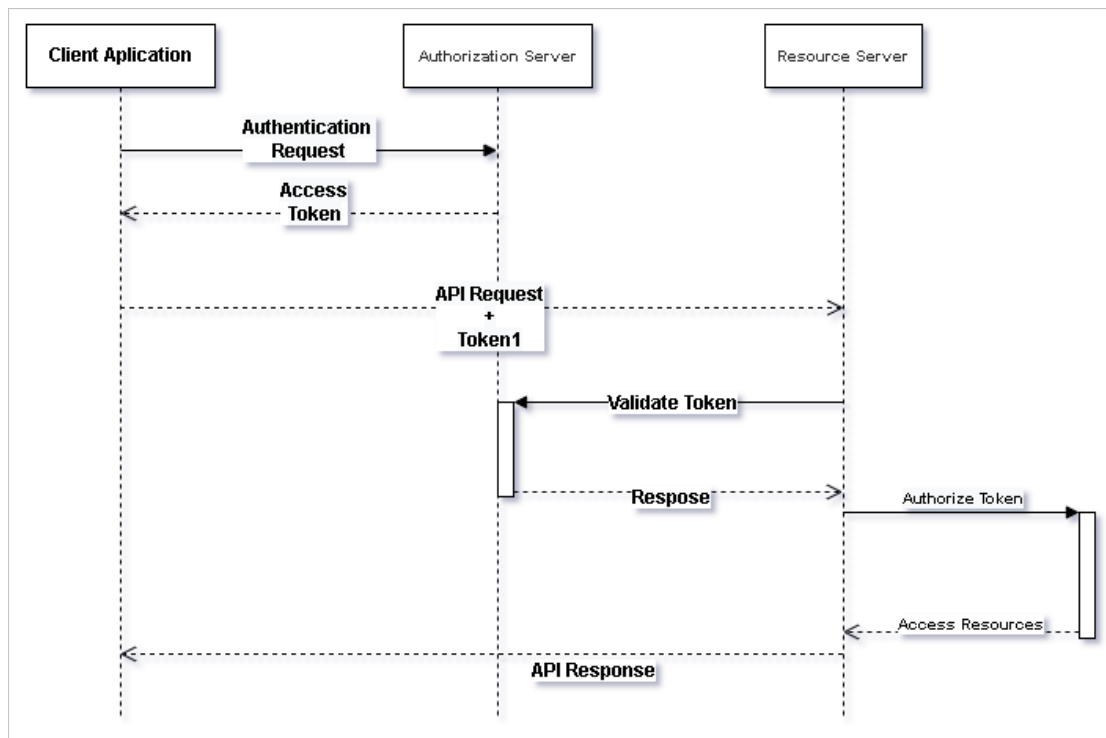
Field	Recommended Settings
Single Sign on URL	{DECISION-URL}/login/saml2/sso/{registrationId}
Audience URI (SP Entity ID)	{DECISION-URL}/saml2/service-provider-metadata/ {registrationId}
Default RelayState	Leave this setting blank unless you require Okta to provide a value to DM
Name ID format	Select any of the available options based on your requirements
Application username	Select any of the available options based on your requirements
Advanced Settings	
Response	signed
Assertion	signed
Authentication context class	Select any of the available options based on your requirements
Request compression	Uncompressed

8. Define the required attributes which need to be sent to Spring SAML after single sign-on and click Next. This step is optional.
9. In the Feedback page, select "This is an internal application that we created" option and click Finish.
The IDP type for SSO configuration of Okta is configured.

8.7.4 Integrating OAuth with DM

OAuth is a standard designed for online authorization that allows a website or application to access resources hosted by other web apps on behalf of a user.

To integrate OAuth 2.0 with DM, you must do the following:



8.7.4.1 OAuth 2.0 Configuration

There are three common scenarios when using OAuth:

1. OAuth IDP is accessible by DM: The issuer URI is accessible by DM and provides an accessible JWKS URI in the OpenID Connect discovery endpoint.
2. OAuth IDP does not provide an accessible JWKS URI in the OpenID Connect discovery endpoint. Still, there is a different JWKS URI that it is accessible to DM and it can be provided as a property to DM.
3. OAuth IDP is not accessible by DM.

OAuth 2.0 Authentication – OAuth IDP is accessible by DM

When OAuth IDP is accessible by DM, add the following properties in DM's `application.properties` file:

- `spring.security.oauth2.resourceserver.jwt.issuer-uri=<issuer URL>`
- `spring.security.oauth2.resourceserver.jwt.audiences=<audience-1>`

For example, using Auth0.com:

- `spring.security.oauth2.resourceserver.jwt.issuer-uri=https://dev-abc.auth0.com/`
- `spring.security.oauth2.resourceserver.jwt.audiences=Decision-Auth0`

Where,

Issuer: The issuer URL is also a point for resolving important metadata about the server, such as its endpoints and capabilities.

Audience: (Optional parameter) Identifies the audience (resource URI or server) that this access token is intended for.

It is possible to add more than one:

- `spring.security.oauth2.resourceserver.jwt.audiences[0]=<audience-1>`
- `spring.security.oauth2.resourceserver.jwt.audiences[1]=<audience-2>`

If 'audience' is provided, the authentication will validate the audience matches the configuration value with the authentication token value.

OAuth 2.0 Authentication - OAuth IDP JWKS URI

If the OAuth IDP does not provide an accessible JWKS URI in the OpenID Connect discovery endpoint but there is a different JWKS URI that is accessible to DM and it can be provided as a property of DM, add the following property in DM's `application.properties` file:

- `spring.security.oauth2.resourceserver.jwt.issuer-uri= <issuer URL>`
- `spring.security.oauth2.resourceserver.jwt.jwk-set-uri= <jwk set URL>`
- `spring.security.oauth2.resourceserver.jwt.audiences=<audience-1>`

For example,

- `spring.security.oauth2.resourceserver.jwt.issuer-uri=https://xxx.auth0.com/`
- `spring.security.oauth2.resourceserver.jwt.jwk-set-uri=https://xxx.auth0.com/.well-known/jwks.json`
- `spring.security.oauth2.resourceserver.jwt.audiences=Decision-Auth0`

Where,

Issuer: The issuer URL is also a point for resolving important metadata about the server, such as its endpoints and capabilities.

JWKSetURI: Specifies the JWK Set URI for DM to fetch the JWK Set.

Audience: (Optional parameter) Identifies the audience (resource URI or server) that this access token is intended for.

It is possible to add more than one:

- `spring.security.oauth2.resourceserver.jwt.audiences[0]=<audience-1>`
- `spring.security.oauth2.resourceserver.jwt.audiences[1]=<audience-2>`

If 'audience' is provided, the authentication will validate the audience matches the configuration value with the authentication token value.

OAuth 2.0 Authentication – OAuth IDP is not accessible by DM

For OAuth 2.0 standalone (no access to external issuer), add the following property in DM's `application.properties` file:

- `spring.security.oauth2.resourceserver.jwt.issuer-uri=<issuer URL>`
- `spring.security.oauth2.resourceserver.jwt.public-key-location=< Certificate>`
- `spring.security.oauth2.resourceserver.jwt.audiences=<audience-1>`

For example,

- `spring.security.oauth2.resourceserver.jwt.issuer-uri=https://dev-abc.auth0.com/`
- `spring.security.oauth2.resourceserver.jwt.public-key-location=oauthClientSigningCertificate.txt`
- `spring.security.oauth2.resourceserver.jwt.audiences=Decision-Auth0`

Where,

Issuer: The issuer URL is also a point for resolving important metadata about the server, such as its endpoints and capabilities.

Certificate: To avoid resource servers triggering request and validate token which can be avoided by configuring public certificate from the authentication application/api.

Audience: (Optional parameter) Identifies the audience (resource URI or server) that this access token is intended for.

It is possible to add more than one:

- `spring.security.oauth2.resourceserver.jwt.audiences[0]=<audience-1>`
- `spring.security.oauth2.resourceserver.jwt.audiences[1]=<audience-2>`

If 'audience' is provided, the authentication will validate the audience matches the configuration value with the authentication token value.

OAuth 2.0 Authentication – Scope Validation

When DE is configured with OAuth authentication, and DM deploys a release to DE, DM pre-authenticates the OAuth credentials for the environment using the default OAuth scope (DE_ADMIN). However, this default behavior can pose a limitation if the client's Identity Provider (IDP) does not support OAuth scopes.

You can disable the pre-authentication of OAuth credentials to address this limitation and allow DM deployments to DE without scope validation. Even with this property disabled, DE will still authenticate the request and validate permissions for the deployment service when it receives the deployment request.

You can configure whether OAuth client scopes should be validated when obtaining a token from the client IDP by adding the following property to the configuration:

- `decision.security.oauth2.client.de.request-default-scopes.enabled`

Options:

- **True (Default):** Enables scope validation.
- **False:** Disables scope validation.

When set to False, scope validation is bypassed during pre-authentication, allowing for compatibility with IDPs that do not support OAuth scopes.

8.7.4.2 Auto Synchronizing User Display name and email (in OAuth environment)

The `userId`, `username`, and `email` will be updated only once for the `ON_CREATE` and `ALWAYS` mode. You cannot synchronize them again after the user is created. To synchronize the user display name and email, add the following properties to the DM's `application.properties` file:

- `spring.security.oauth2.client.provider.sapidp.issuer-uri=<Issuer URL>`
- `spring.security.oauth2.client.provider.sapidp.token-uri=<Token URL>`
- `spring.security.oauth2.client.provider.sapidp.jwk-set-uri=<jwk set URL>`
- `spring.security.oauth2.client.provider.sapidp.user-name-attribute=<Attribute name in the token which includes the username>`
default value: name
- `spring.security.oauth2.client.provider.sapidp.user-info-uri=<userInfo URL>`

Example:

- `spring.security.oauth2.client.provider.sapidp.issuer-uri= https://xxxx.auth0.com/`
- `spring.security.oauth2.client.provider.sapidp.token-uri=https://xxxx.auth0.com/oauth/token`
- `spring.security.oauth2.client.provider.sapidp.jwk-set-uri=https://xxxx.auth0.com/.well-known/jwks.json`
- `spring.security.oauth2.client.provider.sapidp.user-name-attribute=name`
- `spring.security.oauth2.client.provider.sapidp.user-info-uri=https://xxxx.auth0.com/userinfo`

Note: If there is an error while calling the userinfo endpoint, no error will be displayed for the user. Instead, it will use the user id as the username in DM with an empty email field.

8.7.4.3 Configuring DM User Identifier from OAuth Token

The user id for DM is resolved from the JWT based on the following property:

- `decision.oauth2.resourceserver.jwt.authorized.party.keys=sub,azp,client_id,cid`

The property specifies the claims in the JWT that contains the user identifier. The user identifier will be resolved in a specific order of the values of the property. The first search will be done on 'sub' claim. If it doesn't exist, then the search will be done on 'azp' claim and so on. The following are the default values and the order:

- sub
- azp
- client_id
- cid

8.7.5 Activate Users Auto-Creation

DM can be configured to automatically create users after authentication against the IDP though the users do not exist in the DM users database. To activate auto creation of users, add the following properties to DM's `application.properties` file:

- `bdms.auto.create.missing.authenticated.user=true`
Default value: false
- `bdms.user.default.role.name=<desired role name here>`

The role name that the auto-created user will be assigned. The name must be an existing DM role name.

- `bdms.user.default.view.groups.csv=<comma separated community names>`

CSV name values of communities to which the auto-created users will be added with the role specified above. Each community name in the list must be unique in the system.

8.7.6 Configuring auto synchronization of Roles with external IDP

8.7.6.1 Using SAML

The auto sync feature enables the user and user community role synchronization via extensions to the SAML authorization token in the IDP to include additional data structures indicating a user's Community and Roles. When a user logs into Sapiens Decision Manager, it will authenticate the user with the client's SAML-compliant IDP and retrieve the user authentication token with the extension for the permitted community and roles and synch the current settings with the DM user community and roles.

A SAML attribute named `DecisionCommunityRole` should be added as custom metadata to the IDP user authentication token.

This default name can be overridden with a different name using the following property:

- `bdms.saml.roles.attribute.name`

The `DecisionCommunityRole` custom attribute can be added to the user authentication token using either of the defined patterns below:

1. Single SAML attribute with multiple attribute values.

For example,

```
<saml:Attribute name="DecisionCommunityRole">
  <saml:attributeValue>WORLD&SUPER USER</saml:attributeValue>
  <saml:attributeValue>SampleCommunity&Decision
  Analyst</saml:attributeValue>
</saml:attribute>
```

2. Multiple SAML attributes with single attribute values. For example,

```
<saml:Attribute name="DecisionCommunityRole">
  <saml:attributeValue>WORLD&SUPER USER</saml:attributeValue>
</saml:attribute>
<saml:Attribute name="DecisionCommunityRole">
  <saml:attributeValue>SampleCommunity&Decision
  Analyst</saml:attributeValue>
</saml:attribute>
```

Once the IDP authentication token has been granted, DM will automatically synch this information in the user authentication token with the DM RBAC security module based on the following configuration property:

- `bdms.user.sync-mode`: Setting the property values defined below for these parameters will set how the Decision Manager's internal security data is synced for both SAML and OAuth.
- `bdms.user.sync-mode.saml`: Setting the property values defined below for these parameters will set how the Decision Manager's internal security data is synced for OAuth only. This parameter should be used if you want a different sync behavior for SAML and OAuth.

Refer to [Property Valid Values](#) section to know about the detailed valid values.

8.7.6.2 Using OAuth

A custom JWT claim named `https://decision.sapiens.com/communityRoles` should be added as a custom claim that will contain the user communities and roles information in json format.

This default name can be overridden with a different name using the following property:

- `decision.security.community-roles-claim`

The behavior for defining community and roles will have the same configurations as currently available in the LDAP synch. That is, a user will be granted permission by specifying a `CommunityName&Role`, `CommunityName&none`, and `CommunityName&allow`.

Example of the custom claim that contains a json value describing the communities and roles for the user:

- `{ "permissionUpdateId": 2, "communityRoles": [ "Sample Community&SUPER USER", "Comm1&Decision Analyst" ] }`

The `permissionUpdateId` field in the JSON decides whether the user roles should be synced to DM, as will be described later.

Once the IDP authentication token has been granted, DM will automatically synch this information in the user authentication token with the DM RBAC security module based on the configuration properties:

- `bdms.user.sync-mode`: Setting the property values defined below for these parameters will set how the Decision Manager's internal security data is synced for both SAML and OAuth.
- `bdms.user.sync-mode.oauth`: Setting the property values defined below for these parameters will set how the Decision Manager's internal security data is synced for OAuth only. This parameter should be used if you want a different sync behavior for SAML and OAuth.

Refer to [Property Valid Values](#) section to know about the detailed valid values.

8.7.6.3 Property Valid Values

The following are the valid values for `bdms.user.sync-mode`, `bdms.user.sync-mode.oauth`, and `bdms.user.sync-mode.saml` properties:

- **NONE**: No data will be synchronized. This is the system default. This option should be used if auto synchronization with a client IDP is not desired, and users and roles are preferred to be managed in the DM Security Administration.

- **ON_CREATE:** Synchronization with the user authentication token will occur only once on the first login and creation of the user in DM. This option should be used if setting up a user and their security is desired only for the first login of the user to DM. Then afterward the preference is to manage users and roles in the DM Security Administration.
- **ALWAYS:** Synchronization will occur on every user login to DM. This option should be used if the preference is to only manage user permissions for DM in the client IDP and automatically update the user permissions in DM every time a user logs into DM.

8.7.6.4 Additional Properties to read community Roles from IDP

Additional properties can optionally be configured to designate in the IDP Authentication token how to represent a user's community and role definitions. The properties are as follows:

- **idp.failOnError**
If true, the authentication will fail.
Default: true
Optionality: Mandatory
- **idp.group.splitterSign**
A delimiter between the community name and the role name is concatenated in the group name.
Default: &
Optionality: Mandatory
- **idp.group.prefix**
A prefix is used to separate DM groups from others.
Default: No default
Optionality: Optional
For example, idp.group.prefix=DECISION_
- **idp.group.suffix**
A suffix is used to separate DM groups from others.
Default: No default
Optionality: Optional
For example, idp.group.prefix=_ANY_SUFFIX
- **idp.group.none.viewgroup.name**
A name for global roles defined in DM (roles that are not defined in any community). The name must not match any ViewGroup name defined in DM.
For example, idp.group.none.viewgroup.name=global_non_vg
Default: No default
Optionality: Optional

- **idp.group.role.none.name**

Set this property to 'none' for the LDAP role name to represent no name when assigning a user to a community without a role.

For example, idp.group.role.none.name=none

Default: none

Optionality: Mandatory

- **idp.group.role.inherit.name**

Set this property to 'inherit' or 'allow' for the role name to indicate that all roles for the user from the next upper-level Community will be inherited when assigning the user to the Community.

For example, idp.group.role.inherit.name=allow

Default: allow

Optionality: Mandatory

- **idp.group.dependency.group.role.delimiter**

Set this property to '#' for the delimiter to indicate that the role cannot be granted unless the user was authorized for the Community.

For example, idp.group.dependency.group.role.delimiter=#

Default: #

Optionality: Mandatory

- **case.ignore.vg.role**

When set to true, a match for the IDP group would not have to be case sensitive.

For example, case.ignore.vg.role=true

Default: true

Optionality: Mandatory

- **must.have.role.for.vg.role**

When set to true, in order for a user to have a role defined in a certain community, the role would first need to be defined in the corresponding global group.

For example, must.have.role.for.vg.role=false

Default: false

Optionality: Mandatory

delimiter

The # Delimiter dependency is designed to prevent users from being assigned roles in a Community for which they do not have access permission. Since Community access and Community role assignment are generally separate procedures, the # dependency will block a user from receiving a role in a Community if the user did not gain access to the Community from a separate group.

Example:

If 'Private' is a locally defined Community, BDMP_Private&BDMS Administrator#_gg is a group assigning users with the 'BDMS Administrator' role, only if the users were granted access to

the 'Private' Community separately via another group, such as BDMP_Private&BCD Manager_gg.

If 'User1' is defined in BDMP_Private&BDMS Administrator#_gg and in no other group, that user will be blocked from accessing the Community and from being assigned the role. In addition, the synchronization process will generate a warning message to inform the DM administrator that the user was not granted the role.

'none' keyword for role parameter

The option 'none' can be set for a group to block role assignment. Adding users to a group with the 'none' option allows access to the Community, but with no role assigned to the user.

Example:

The group BDMP_WORLD&none_gg will add users to the WORLD Community without assigning them roles.

'allow' keyword for role parameter

The option 'allow' can be set for a group to inherit roles from the parent of a locally defined Community. Adding users to a group with the 'allow' option inherits all assigned roles from the parent Community.

Example:

'User2' is defined in the WORLD Community with the Decision Analyst, BDMS Administrator and BCD Manager roles. 'Private' is a direct child of the World Community. If 'User2' is added to the BDMP_Private_allow_gg group, the user will be assigned all roles inherited from the parent Community.

This will avoid the need to define a separate group for each role in a locally defined Community.

8.8 Configure CAPTCHA support

DM allows reCAPTCHA support for failed login.

To enable this functionality:

1. Generate keys from Google site: <https://www.google.com/recaptcha/admin#list>, make sure to add the domain's name, the server where DM is installed
2. Add the following properties in DM's `application.properties` file:
 - `bdms.enable.captcha=true`
 - `google.recaptcha.key.site=%google_site_key%`
 - `google.recaptcha.key.secret=%google_secret_key%`
3. Add the following to JVM: **-Dcom.sun.net.ssl.enableECC=false**, this will enable access to google site.
 - a. For all-in-one or any external tomcat installation:

Edit the DM's Java command line by modifying the `JAVA_OPTS` variable in the `setenv.sh` script (located in Tomcat's bin folder) to set `com.sun.net.ssl.enableECC` to false (if it is currently set to true).

- b. For a standalone installation, set it to `false` in the Java command line used to start the DM application.
 - c. For a containerized installation, set it in the global Java `configMap`.
4. Captcha will be required after 3 failed attempts. This may be configured by adding the following to DM's `application.properties`
- `google.recaptcha.show.after.failed.attempts.num=%number_of_failed_attempts%`

Note: The CAPTCHA enabled mode is not supported in the DM Desktop application.

8.9 Configure Parameter Management Link on DM Header

This feature adds a Parameter Management link to the Decision Manager (DM) header. To enable this feature, configure the following properties in `application.properties` file:

```
decision.dm.pam.environment.info.environments[0].name=DE Development
decision.dm.pam.environment.info.environments[0].type=DEVELOPMENT
decision.dm.pam.environment.info.environments
[0].url=http://localhost:8080/pam-ui

decision.dm.pam.environment.info.environments[1].name=DE Production
decision.dm.pam.environment.info.environments[1].type=PRODUCTION
decision.dm.pam.environment.info.environments
[1].url=http://localhost:9090/pam-ui
```

where,

`decision.dm.pam.environment.info.environments[0].name=` Environment name

`decision.dm.pam.environment.info.environments[0].type=` Environment type

`decision.dm.pam.environment.info.environments[0].url=` Environment URL

Note: To configure additional environments, increment the environment index ([0], [1], etc.) and add corresponding name, type, and url properties for each environment.

8.10 Test with a Java Double Implementation

The section deals with the property required to continue running the Test Facility with a Java Double implementation.

Java 'BigDecimal' support for numeric data types and calculations was implemented in DM. Java 'BigDecimal' supports larger numbers and high levels of calculation precision for financial calculations.

The Java 'BigDecimal' implementation is supported in the DM Test Facility, Decision Execution (DE) and the Java Adapter. Support for 'BigDecimal' implementation will be provided in additional Deployment Adapters (DD) in future releases.

The default implementation for DM will be for the Test Facility to execute with a `BigDecimal` implementation. If a configuration of the Test Facility with a Java 'Double' implementation, as was the case with prior versions of DM, is desired, the following property must be configured in the DM's `application.properties` file:

```
bdes.execution.data-type.decimal=double
```

Note: If your current deployments are using a Java 'Double' implementation, the above property should be set to 'double' to ensure the internal testing in DM matches your deployment. If you want to change to a Java 'BigDecimal' implementation, remove the property from the DM's `application.properties` file or set it to:

```
bdes.execution.data-type.decimal=bigdecimal
```

Note: Decision Execution (DE) has a similar property for changing the DE implementation from Java 'BigDecimal' to Java 'Double'. The new default implementation for DE is also Java 'BigDecimal'. If a Java 'Double' implementation is desired for DE, the following property and value should be set in the DE's `application.properties` file:

```
decision.de.execution.data-type.decimal=double
```

If the default setting of Java 'BigDecimal' is desired, the property can be omitted from DE `application.properties` file or set with the value "bigdecimal".

8.11 Test with Java Date or Timestamp

The `bdes.export.data-type.temporal` property is used to designate the approach to handling temporal data types ('DATE', 'DATE_TIME') defined for Fact Types in Sapiens Decision at execution time. This will control the Java data type used in interfaces to User Defined Functions (UDFs) and deployed code interfaces provided by the Deployment Adapters for the temporal data types.

If DATE and DATE_TIME are to be implemented as 'java.util.Long' (e.g. timestamp) the property shall be set in the DM's `application.properties` file as:

```
bdes.export.data-type.temporal=timestamp
```

If DATE and DATE_TIME are to be implemented as 'java.util.Date' the property shall be set in the DM's `application.properties` file as:

```
bdes.export.data-type.temporal=date
```

Note: If no value is set for the property, the implementation will default to 'java.util.Date' as if the property was set as:

```
bdes.export.data-type.temporal=date
```

8.12 Appendix

8.12.1 Configuration Files

8.12.1.1 DM `application.properties` file

The `dm-config/application.properties` file allows users to override default property values in Decision.

All the properties in the file are listed below in alphabetical order.

Name	Valid Values	Default value	Description
authentication.hsts.on	true, false	true	Defines the relevant authentication process to turn off HTTP Strict-Transport-Security (HSTS).
bdes.deployment.connection Timeout	Any integer (in milliseconds)	6000	Defines the underlying URLConnection's (the REST services of the DE in this case) connection timeout (in milliseconds). A timeout value of 0 specifies an infinite timeout. Connection timeout indicates that the connection to DE was not completed within the specified timeout.

Name	Valid Values	Default value	Description
bdes.deployment.receiveTimeout	Any integer (in milliseconds)	7200000	Defines the underlying URLConnection's (the REST services of the DE in this case) read timeout (in milliseconds). A timeout value of 0 specifies an infinite timeout. Read timeout occurs when the socket is open, connection to the host server is established, the request is sent, but the response from the server is not received on time and cannot be read.

Name	Valid Values	Default value	Description
bdms.db.exec.time.warn.ms.threshold	Integer	120000	A diagnostic ability is added to DM that any data that access longer than a predefined threshold will be written to a new log. The Information that is saved in the log is the server method that invokes it with its parameters. The file is called "db_warnings.sql". This functionality will have no effect on the overall server performance and will identify slow queries that can be improved for better server performance on each new server version. The property values are specified in milliseconds.

Name	Valid Values	Default value	Description
bdms.env.type.change.check.exported	true/false	true	The value 'true' for this property means that after an initial export using the environment, changes to the Environment Type will not be permitted, unless the change is from null to another option. If set to 'false', changes to the Environment Type will be permitted even after the first export.
bdms.environment.type	ProductionYY, Prod=Yes;DevelopmentYY, prod=no;UATYY, Prod=No	bdms.environment.type=Production, Prod=Yes;Development, prod=no;UAT, Prod=No	In the export release user interface, the options for the Production Type include Production, Development and UAT. If you use 'bdms.environment.type=Something, Prod=Yes', only "Something" will appear in the Production Type drop down list.

Name	Valid Values	Default value	Description
bdms.excel.file.format	XLSX, XLS	XLS	The format in which Excel Workbook files will be created during export. The value XLSX is used for Excel 2007 and later, and XLS is used for versions prior to Excel 2007. If not specified, the value will be XLSX.
bdms.external.datetime.format	Valid datetime Java format pattern	MM/dd/yyyy HH:mm:ss	Globally sets the format that DM expects for and produces date/date-time values
bdms.hibernate.slower.query.time	Integer	120000 milliseconds i.e., 2 minutes	A diagnostic ability in addition to the bdms.db.exec.time.warn.ms.threshold property is added to DM that any data query that runs for longer than a predefined threshold will be logged. The property values are specified in milliseconds.
bdms.premigration.search.template	Valid regex expression for files	classpath*:preMigration/*.sql	Specifies the path to the SQL script files to be executed before the usual version migration.

Name	Valid Values	Default value	Description
bdms.revision.associated.facts.search.status.filter	DRAFT,CANDIDATE,APPROVED,NONE	NONE	The Mapped Entities tab of Revision will display the associated Fact Types filtered by mapped status based on the property value. Ideally you would want to see all Fact Type mappings. However, if you have a lot of Fact Type Mappings you can change this property to only see draft and candidate which would be the fact Types that could block the deployments. This would reduce the data on the page to make it easier to find the blocking Fact Types.
bdms.run.premigration	true, false	false	Enables the execution of SQL scripts before the usual version migration.
bdms.superuser	Any value	admin	Sets the Super User definition to admin.

Name	Valid Values	Default value	Description
bdms.user.sync-mode	NONE, ON_CREATE, ALWAYS	ON_CREATE	This property globally defines whether internal security permissions in Decision Manager should be synchronized with SAML and/or OAuth authentication tokens during user login. If not set, the default behavior will apply.
bdms.user.sync-mode.oauth	NONE, ON_CREATE, ALWAYS	“bdms.user.sync-mode” value if set, otherwise NONE	This property defines whether internal security permissions in Decision Manager should be synchronized with OAuth authentication tokens during a transaction request. If not set, the default behavior will apply.
bdms.user.sync-mode.saml	NONE, ON_CREATE, ALWAYS	“bdms.user.sync-mode” value if set, otherwise NONE	This property defines whether internal security permissions in Decision Manager should be synchronized with SAML authentication tokens during a user login request. If not set, the default behavior will apply.

Name	Valid Values	Default value	Description
datasource.connection.idle.timeout	Integer	600000	Specifies the maximum time (in milliseconds) that a connection can remain idle in the pool before being retired. The actual idle timeout may vary by up to +30 seconds, with an average variation of +15 seconds. A connection will never be retired as idle before this timeout. A value of 0 means that idle connections are never removed from the pool.
datasource.connection.max.lifetime	Integer	1800000	Specifies the maximum lifetime of a connection in the pool. Once this timeout is reached, the connection will be retired from the pool, regardless of recent usage. However, an active (in-use) connection will not be retired; it will only be removed when it becomes idle.
datasource.connection.test.query	SQL query string		Specifies the SQL query used to test the validity of connections in the pool. If not set, connectivity is not verified using an SQL query.

Name	Valid Values	Default value	Description
datasource.connection.timeout	Integer	30000	Specifies the maximum duration (in milliseconds) that a client will wait for a connection from the pool before timing out. If this duration is exceeded without a connection becoming available, a SQLException will be thrown.
datasource.max.pool.size	Integer	30	Defines the maximum number of connections a pool will maintain at any given time. Must not be set lower than 3.
datasource.min.pool.size	Integer	3	Defines the minimum number of connections a pool will maintain at any given time. Must not be set lower than 3.
dd.export.file.name	Any valid file name	DeploymentDescriptors	Default value of the file name present in the exported package.

Name	Valid Values	Default value	Description
dd_empty_dates_range	no-date-range, full-date-range	full-date-range	If set to 'no-date-range', non-populated effective and expiration dates in a Deployment Descriptor row are considered 'null' and ignored when checking for overlaps or gaps. The default 'full-date-range' sets null effective dates to "1970-01-01" and null expiry dates to "9999-12-31".
decision.base.url	<Scheme>://<DomainName> where DomainName = dnsname or host:port	http://localhost:9080/	This is used in SAML authentication type as the URL service location of the DM application to be used by the Identity Provider (IDP).
decision.dm.expire.anonymous.authenticated.request	true, false	true	Controls the expiration of anonymous authenticated sessions. If disabled, the default configured Tomcat session timeout is used.
decision.dt.exec.base-url	<valid host and port URL>/dt-exec	http://localhost:\${server.port}/dt-exec	The URL of the Decision Testing Exec application. This is not required for All-in-One deployment.

Name	Valid Values	Default value	Description
decision.dt.generate.base-url	<valid host and port URL>/dt-generate	http://localhost:\${server.port}/dt-generate	The URL of the Decision Testing Generate application. This is not required for All-in-One deployment.
decision.dt.orch.base-url	<valid host and port URL>/dt-orchestrator	http://localhost:\${server.port}/dt-orchestrator	The URL of the Decision Testing Orchestrator application. This is not required for All-in-One deployment.
decision.dt.repair.base-url	<valid host and port URL>/dt-repair	http://localhost:\${server.port}/dt-repair	The URL of the Decision Testing Repair application. This is not required for All-in-One deployment.
decision.dt.repo.base-url	<valid host and port URL>/dt-repo	http://localhost:\${server.port}/dt-repo	The URL of the Decision Testing Repo application. This is not required for All-in-One deployment.
decision.enable.webservices. download-deploymentpackage-revisionhistory	true, false	true	Turns off/on the listDeploymentRequests and downloadDeploymentPackage webservice.

Name	Valid Values	Default value	Description
decision.healthcheck.interval	Any number (milliseconds)	999999999	Defines the interval for the internal scheduler to run the scheduled health check. If the value is ≤ 0 , the scheduler is disabled. When the result is 'DOWN', health information is logged as an error.
decision.healthcheck.threadresources.active.threads.prefixes	Comma-separated list (e.g., tcp,c3p0,hz,reactor-tcp,reactor-http)	http-nio	Reviews information on various active thread prefixes provided in the health check service response.
decision.session.event.lock.timeout	30s	30s	Specifies the time-to-live (TTL) for the lock used to prevent duplicate publishing of session events.

Name	Valid Values	Default value	Description
decision.session.redis.config-action.enabled	true, false	false	Controls whether setup commands are sent to Redis. This setting is relevant for users running Redis locally. - Set to true to enable sending setup commands (required for local Redis). - Set to false (or leave unset) to skip sending setup commands—this is required for environments such as Azure or AWS, where such commands may be blocked.
deployment.descriptor.datetime.format	Date format	Property value of "bdms.external.datetime.format"	Sets the format for the Deployment Descriptor approval date.
hibernate.dialect	The required dialect is specified in the Hibernate documentation.		Defines which DB dialect is in use in the environment. Examples: org.hibernate.dialect.Oracle10gDialect, org.hibernate.dialect.HSQLDialect

Name	Valid Values	Default value	Description
hibernate.hbm2ddl.auto	validate, update, create, create-drop	validate	Automatically validates, updates, or creates the application schema in the database when the SessionFactory is created (i.e., when the server starts).
jdbc.driverClassName	The required driver, depending on the database type, is specified in the installation guide.		The JDBC driver for the database connection
jdbc.password	Password string		Sets the password for the database connection (can be set encrypted like any other property value).
jdbc.url	The required value, depending on the database type, is specified in the installation guide.		Sets the connection URL for the database.
jdbc.username	Username string		Sets the username for the database connection.
property-groups.core-prop-groups.bean-conf.beanName.propertyName			

Name	Valid Values	Default value	Description
server.port	Valid port number	9080	The application server port. For All-in-One deployment, this must be set to the same port number defined in the Tomcat server.xml configuration. For other deployment types, it defines the actual port number that will be used for the application.
server.servlet.context-path	path string	/dm	The context path of the application access URL.
testing.de.password	Any value	bdms	The configured password in DE (If DE is working in secured mode)
testing.de.url	Valid full URL	http://localhost:8090/bdes	The full HTTP/S URL to access DE testing instance
testing.de.username	Any value	admin	The configured username in DE (If DE is working in secured mode).

Name	Valid Values	Default value	Description
validate.valuemapping.datatype.compatibility	true, false	true	If this property is set to true, upon export, enforces value mapping Data Type compatibility with Fact Type's data type. The check is done on ExternalEngineValidator and is performed upon export using Java exporter and DE. For example, if a Fact Type's data type is Numeric and the mapped values are "A, B, and C" and this property is set to "true", then an exception will occur during deployment and indicate the Fact Types in the release that have incompatible value mappings, rather than allowing the deployment and catching this exception during execution.

Name	Valid Values	Default value	Description
validation.functions.ignoreList	Any function name	None	Defines the comma separated list of names of functions to be ignored during validation. This would be used if you override any internal decision functions that change their behavior.
valueMapping.sync.default.value	true, false	false	If set to 'true', empty valid value mappings will be filled in with values identical to the valid values. If set to 'false', empty mappings will remain empty. Example: If valid values are 'a', 'b' and 'c', and this property is set to 'true', the mapped values (if empty) will also be set to 'a', 'b' and 'c'. If this property is set to 'false', the mapped values will remain empty.

Name	Valid Values	Default value	Description
valueMapping.sync.delete.unused	true, false	false	If set to 'true', value mapping for all deleted valid values will be deleted. If set to 'false', the value mapping for deleted valid values will not be deleted. Example: If valid values are 'a', 'b', and 'c', and this property is set to 'false', deleting valid value 'a' will not delete the value and its mapping.

8.12.1.2 DT apps application.properties files

The following properties are common across all DT applications (DT Orchestrator, DT Repository, DT Executor, DT Generate, and DT Repair). These can be configured with custom values in each application's `application.properties` file. Depending on the deployment type, some properties may not be applicable to all DT applications.

Name	Default Value	Example Value	Description
async.support.thread.pool.size.core	20	Any number	Defines the core size of the thread pool for asynchronous HTTP requests. Should only be changed if a performance issue is identified in the DT micro-app async requests.
async.support.thread.pool.size.max	80	Any number	Defines the maximum size of the thread pool for asynchronous HTTP requests. Should only be changed if a performance issue is identified in the DT micro-app async requests.

Name	Default Value	Example Value	Description
decision.health.heartbeat.timeout	3000	Any number	Specifies the timeout (in milliseconds) for each service heartbeat response during the health check.
decision.healthcheck.interval	0	Any number	Defines the interval (in milliseconds) for the DT micro-app to perform self-health checks. A value of 0 disables the health check.
decision.healthcheck.threadresources.active.threads.prefixes	http-nio	Comma-separated list (e.g., tcp,c3p0,hz,reactor-tcp,reactor-http)	Specifies the comma-separated thread prefixes to be included in the active thread details for the DT micro-app health check.
dt.additionally.allowed.access.csv.hosts		1.1.1.1, server-x	A comma-delimited list of additional IPs or hostnames permitted to access DT applications. If not specified, access is limited to the hosts deployed with DM or any DT app.
dt.hazelcast.instance.eviction.policy	LRU	LRU, LFU, None	Specifies the Hazelcast memory cache eviction strategy: LRU (default) - Least Recently Used. LFU - Least Frequently Used. None - No items will be evicted.
dt.hazelcast.instance.io.threads.max.queue.size	100	Any number	Specifies the maximum number of threads queue size for Hazelcast communication. Adjust only if performance issues occur.

Name	Default Value	Example Value	Description
dt.hazelcast.instance.io.threads.max.size	50	Any number	Specifies the maximum number of threads for Hazelcast. Adjust this value only if performance issues are identified in the DT micro-app Hazelcast communication.
dt.hazelcast.instance.max.size	20	Integer	Defines the eviction threshold value, used with "dt.hazelcast.instance.max.size.policy". Example: If set to "FREE_HEAP_PERCENTAGE" and max size is "20", eviction will occur when free heap size falls below 20%.
dt.hazelcast.instance.max.size.policy	FREE_HEAP_PERCENTAGE	FREE_HEAP_PERCENTAGE, PER_NODE	Specifies the Hazelcast memory cache eviction threshold type: FREE_HEAP_PERCENTAGE - Minimum free heap size percentage for each JVM. PER_NODE - Maximum number of map entries in each cluster member.
dt.hazelcast.instance.name	Random generated UUID	Any value	Specifies a name for this Hazelcast instance within the cluster. If not set, a random UUID is assigned.
dt.http.timeout.default	20000	Any number	Specifies the default HTTP request timeout duration (in milliseconds) for DT micro-apps.

Name	Default Value	Example Value	Description
dt.http.timeout.long.running.query	120000	Any number	Defines the timeout duration (in milliseconds) for long-running HTTP requests between DT micro-apps and the DT Repository. If you increase this property value, the <code>"spring.mvc.async.request-timeout"</code> property for the orchestrator and repository micro-apps should also be adjusted by the same amount to maintain synchronization.
dt.hz.aws.access.key		Any value	Specifies the AWS account user key. This setting is relevant for Hazelcast clusters in the AWS environment.
dt.hz.aws.access.secret		Any value	Specifies the AWS account user secret. This setting is relevant for Hazelcast clusters in the AWS environment.
dt.hz.aws.connection.retries	3	Any number	Specifies the number of retries for calls to the AWS API in case of failure. Relevant for Hazelcast clusters deployed on the AWS platform.
dt.hz.aws.connection.timeout.sec	10	Any number	Specifies the connection timeout in seconds for API calls made to the AWS platform. Relevant for Hazelcast clusters deployed in AWS.
dt.hz.aws.iam.role		Any value	Specifies the AWS IAM role associated with the account. This setting is relevant for Hazelcast clusters in the AWS environment.

Name	Default Value	Example Value	Description
dt.hz.aws.legacy.discovery	false	true, false	Indicates whether to use the legacy cluster member discovery mode in AWS, as opposed to the discovery SPI. This setting is relevant for Hazelcast clusters in the AWS environment.
dt.hz.aws.read.timeout.sec	10	Any number	Defines the read timeout in seconds when making a call to the AWS API. Relevant for Hazelcast clusters deployed on the AWS platform.
dt.hz.aws.region		Any value	The AWS region associated with the AWS account. This is relevant for Hazelcast clusters deployed on the AWS platform.
dt.hz.aws.security.group.name		Any value	An optional filter that limits the search to EC2 instances associated with a specific AWS security group. Relevant for Hazelcast clusters deployed in AWS.
dt.hz.aws.tag.key		Comma separated string values	Specifies the tag key(s) used to filter and search for EC2 instances within the AWS platform. Relevant for Hazelcast clusters deployed in AWS.
dt.hz.aws.tag.value		Comma separated string values	Specifies the tag value(s) associated with the tag key (s) from "hazelcast.aws.tag.key". This filter is used to search for EC2 instances with the matching tag values. Relevant for Hazelcast clusters deployed in AWS.

Name	Default Value	Example Value	Description
dt.hz.azure.client.id		Any value	The Azure Active Directory Service Principal client ID used for authentication. Relevant for Hazelcast clusters deployed on the Azure platform.
dt.hz.azure.client.secret		Any value	The Azure Active Directory Service Principal client secret used for authentication. Relevant for Hazelcast clusters deployed on the Azure platform.
dt.hz.azure.instance.metadata.available	false	true, false	Set this to true if instance metadata can be fetched and used automatically, instead of manually defining the "tenant.id", "subscription.id", "resource.group", and "scale.set" properties. Relevant for Hazelcast clusters deployed on the Azure platform.
dt.hz.azure.resource.group		Any value	The name of the Azure resource group in which the Hazelcast instance is running. Relevant for Hazelcast clusters deployed on the Azure platform.
dt.hz.azure.scale.set		Any value	The name of the Azure VM scale set. If configured, only instances within this scale set can join the Hazelcast cluster. Relevant for Hazelcast clusters deployed on the Azure platform.
dt.hz.azure.subscription.id		Any value	The Azure subscription ID used for authentication. Relevant for Hazelcast clusters deployed on the Azure platform.

Name	Default Value	Example Value	Description
dt.hz.azure.tag		Any value=Any value	Defines the key-value pair for the tag on Azure network interfaces (NIC). The format should be key=value. If configured, only instances with matching tags on their network interfaces can join the Hazelcast cluster. Relevant for Hazelcast clusters deployed on the Azure platform.
dt.hz.azure.tenant.id		Any value	The Azure Active Directory tenant ID used for authentication. Relevant for Hazelcast clusters deployed on the Azure platform.
dt.hz.cluster.name		Any value	Specifies a unique name for the Hazelcast cluster this application instance belongs to. Required in networks with multiple clustered environments.
dt.hz.discovery.type		aws, azure, kubernetes, host-ip	Sets the type of Hazelcast discovery mechanism explicitly: aws - AWS-based discovery. azure - Azure-based discovery. kubernetes - Kubernetes-based discovery. host-ip - Host IP-based discovery.

Name	Default Value	Example Value	Description
dt.hz.hosts.lookup.csv		Comma-separated host values	A comma-separated list of hosts/IPs to search for Hazelcast members. Supports wildcards "*" or "-" for the last IP segment. Specific IPs can be specified with a port number. Example: hazelcast.cluster.members.csv = machine1,machine2,machine3:5799,192.168.1.0-7,192.168.2.*
dt.hz.interfaces.csv		Comma-separated valid IP addresses.	Defines the IP interfaces Hazelcast should bind to. Supports wildcards like "*" or "-" in the last digit. Example: 10.3.16.*,10.3.10.4-18,192.168.1.3
dt.hz.kubernetes.api.retries		Integer	Specifies the number of retries in case of issues while connecting to the Kubernetes API. Relevant for Hazelcast clusters in Kubernetes platform when working in API mode.
dt.hz.kubernetes.api.token		Any value	API Token for connecting to the Kubernetes API. Relevant for Hazelcast clusters in Kubernetes platform when working in API mode.
dt.hz.kubernetes.ca.certificate		Any value	CA Certificate for connecting to the Kubernetes API securely. Relevant for Hazelcast clusters in Kubernetes platform when working in API mode.

Name	Default Value	Example Value	Description
dt.hz.kubernetes.master		Any valid URL	URL of the Kubernetes Master for API access. Relevant for Hazelcast clusters in Kubernetes platform when working in API mode.
dt.hz.kubernetes.namespace		Any value	Specifies the Kubernetes namespace where Hazelcast is running. Relevant for Hazelcast cluster in Kubernetes platform when operating in API mode.
dt.hz.kubernetes.pod.label.name		Comma-separated string values	Defines the label names for Pods forming the Hazelcast cluster in Kubernetes. Must be used with dt.hz.kubernetes.pod.label.value . Example: " label-1,label-2 ". Ensure both properties have the same number of values.
dt.hz.kubernetes.pod.label.value		Comma-separated string values	Specifies the label values for Pods forming the Hazelcast cluster in Kubernetes. Must be used with dt.hz.kubernetes.pod.label.name . Example: " label-1,label-2 ". Ensure both properties have the same number of values.
dt.hz.kubernetes.resolve-not-ready-addresses	false	true, false	When set to true , also checks the addresses of Pods that are not ready. Relevant for Hazelcast clusters in Kubernetes platform when working in API mode.

Name	Default Value	Example Value	Description
dt.hz.kubernetes.service.dns.timeout		Integer	Specifies the timeout duration for DNS lookup searches. This setting is applicable when using a Hazelcast cluster in a Kubernetes environment operating in DNS mode.
dt.hz.kubernetes.service.dns.value		Any value	Service DNS name defining the cluster members' IPs. Relevant for Hazelcast clusters in Kubernetes platform when working in DNS mode.
dt.hz.kubernetes.service.label.name		Comma-separated string values	Defines the labels for services forming the Hazelcast cluster in Kubernetes. Must be used with "dt.hz.kubernetes.service.label.value". Example: " label-1,label-2 ". Ensure both properties have the same number of values.
dt.hz.kubernetes.service.label.value		Comma-separated string values	Specifies the label values for services forming the Hazelcast cluster in Kubernetes. Must be used with dt.hz.kubernetes.service.label.name . Example: " label-1,label-2 ". Ensure both properties have the same number of values.
dt.hz.kubernetes.service.name		Any value	Specifies the Kubernetes service name for filtering Pods that should join the cluster. If not specified, all Pods in the namespace are considered.

Name	Default Value	Example Value	Description
dt.hz.kubernetes.service.port		Valid port number	Defines the port number for Hazelcast member communication. This setting is relevant for Hazelcast clusters in a Kubernetes environment operating in DNS mode.
dt.hz.partition.group.type		HOST_AWARE, CUSTOM, PER_MEMBER, ZONE_AWARE, NODE_AWARE, PLACEMENT_AWARE, SPI	Defines the type of partitioning to prevent data loss in Hazelcast cluster. Relevant for Hazelcast clusters in Kubernetes platform when working in API mode.
dt.hz.port		Valid port number	Specifies the specific port number for the application to bind to for Hazelcast.
dt.hz.port.range	5701-5800	Numbers range (dash-separated)	Defines the range of ports for Hazelcast to search for the next available port. It also looks for other Hazelcast members within this range to join the cluster.
dt.repo.interface	rsocket	rsocket, http	Specifies the interface type (RSocket or HTTP) for communication between DT micro-apps and the DT Repository.
dt.rsocket.client.connection.acquire.timeout	10000	Any number	Specifies the timeout duration (in milliseconds) for acquiring a connection in RSocket communication.
dt.rsocket.client.connection.retry.interval	4000	Any number	Specifies the retry interval (in milliseconds) for RSocket connection attempts.

Name	Default Value	Example Value	Description
dt.rsocket.client.connection.retry.max	unlimited	Any number	Defines the maximum number of retry attempts for RSocket connections.
dt.rsocket.client.connection.single	false	true, false	Determines whether to use a single connection for RSocket communication when "dt.repo.interface" is set to "rsocket".
spring.servlet.multipart.max-file-size	10	Any number	Specifies the maximum allowed file size (in MB) for requests to DT applications.
spring.servlet.multipart.max-request-size	100	Any number	Defines the maximum allowed size (in MB) for multipart requests to DT applications.
testing.exec.host	localhost	Any valid hostname	Specifies the hostname of the DT Executor component.
testing.generate.host	localhost	Any valid hostname	Specifies the hostname of the DT Generate component.
testing.orchestrator.host	localhost	Any valid hostname	Specifies the hostname of the DT Orchestrator component.
testing.repair.host	localhost	Any valid hostname	Specifies the hostname of the DT Repair component.
testing.repo.context.path	dt-repo	Any value	Specifies the context path of the Decision Testing Repository application.
testing.repo.host	localhost	Any valid hostname	Specifies the hostname of the DT Repository component.
testing.repo.port	8082	Any valid port number	Specifies the port number on which the Decision Testing Repo application listens.
testing.repo.protocol	http	http, https	Specifies the communication protocol (HTTP or HTTPS) for the Decision Testing Repo application.

Name	Default Value	Example Value	Description
testing.repo.rsocket.password		Any value	Specifies the password for RSocket communication when "dt.repo.interface" is "rsocket" without OAuth.
testing.repo.rsocket.server.mapping-path	/rsocket	Any value	Specifies the URL context path for RSocket communication when "dt.repo.interface" is set to "rsocket".
testing.repo.rsocket.server.port	7001	Any valid port number	Specifies the port number for RSocket communication when "dt.repo.interface" is set to "rsocket".
testing.repo.rsocket.user	internal User	Any value	Specifies the username for RSocket communication when "dt.repo.interface" is "rsocket" without OAuth.
tg.events.sample-period-msec	500	Any number	Sets the time interval (in milliseconds) between progress bar updates for batch operations. Smaller values provide real-time updates but are more resource-intensive.

* Hazelcast - 3rd party component used for DT micro-apps data and communication sync.

8.12.1.3 DT Repository application.properties

Name	Default Value	Example Value	Description
db.connection.pool.max.idletime.seconds	180	any number	Defines the time (in seconds) a connection can remain pooled but unused before being discarded.

Name	Default Value	Example Value	Description
db.connection.pool.size.initial	10	any number	Defines the number of connections a pool will try to acquire upon start-up.
db.connection.pool.size.max	50	any number	Defines the maximum number of connections a pool will allow at any given time.
dm.context.path	dm		The context path of DM application.
dm.host	localhost		Hostname of DM application.
dm.port	9080		The port number which DM application listens to.
dm.protocol	http	http,https	The protocol to use for communication with DM application.
dt.repo.delete.orphan.tg.run.duration	120	any number	To control run duration for deletion of orphan Test Groups. This property accepts value in minutes.

Name	Default Value	Example Value	Description
dt.testgroup.export.download.zip	true	false	When set to "true", exported Test Group will be downloaded in a zip format. When set to "false", exported Test Group will be downloaded and will not be zipped unless the export contains more than one file. In this case it will be downloaded in a zip format.
ks.purge.logging.file.path		<no default>	Sets the log path where the log file will be generated after the purge.
ks.purge.timeStampPattern	dd-MM-yyyy_ HH.mm.ss.SSS	any number	The user can provide preferred timestamp format to generate purge log file name. If not, then the default purge log file name timestamp format (dd-MM-yyyy_ HH.mm.ss.SSS) will be taken.

Name	Default Value	Example Value	Description
logging.file.name	log/dt_repo.log	/dt-home/logs/dt_repo.log	Path and file name for log file. Recommended to specify full path from file system root.
purge.logging.file.path		<no default>	Sets the log path where the log file will be generated after the purge.

Name	Default Value	Example Value	Description
r2dbc.jdbc.threads.max.queue.size	100	any number	Relevant only if Oracle is the underlying DT database. Max number of threads to wait in the queue for database interactions. Should not be changed unless performance issue identified in the DT queries specifically. Smaller size – Risking bottleneck, but less load on the database and less memory consumption. Bigger size – Risking database or memory overload, but faster results assuming enough resources of memory, CPU and network. If you are using Oracle Version 20 or lower, it is advised to increase the default value to 200.

Name	Default Value	Example Value	Description
r2dbc.jdbc.threads.max.size	50	any number	Relevant only if Oracle is the underlying DT database. Max number of threads to use for database interactions. Should not be changed unless performance issue identified in the DT queries specifically. Smaller size – Risking bottleneck, but less load on the database and less memory consumption. Bigger size – Risking database or memory overload, but faster results assuming enough resources of memory, CPU and network. If you are using Oracle Version 20 or lower, it is advised to increase the default value to 100.

Name	Default Value	Example Value	Description
repo.datasource.connection.secured	false	true, false	If to make a secure connection with the database.
repo.datasource.database	dt	Any value	The database name to connect to
repo.datasource.host	localhost	Valid hostname	The host name of the database server
repo.datasource.password	dt_usr	Any value	The database password to connect with
repo.datasource.port	5432	Valid port number	The port number of the database server
repo.datasource.user	dt_usr	Any value	The database username to connect with
repo.datasource.vendor	postgres	postgres, oracle	The database type
server.port	8082	Any valid port number	The port number of DT Repository application.
server.servlet.context-path	dt-repo	Any value	The context path of DT Repository application.

Name	Default Value	Example Value	Description
spring.mvc.async.request-timeout	180000	any number	Duration in milliseconds to wait for an incoming asynchronous HTTP request between the DT micro-apps until it is considered a timeout. This property value must be at least 33% higher than any other asynchronous DT timeout property.
test.cases.purge.duration	1095	any number	Sets the maximum duration (in days) for the Test Cases to be retained when running the Test Case purge web service.

Name	Default Value	Example Value	Description
test.cases.purge.run.duration	120	any number	Web service execution is limited to execute the Test Cases purge for the configured duration (in minutes). This property can be overridden by explicitly passing the parameter in the web service.
test.cases.purge.timeStampPattern	dd-MM-yyyy_ HH.mm.ss.SSS	any number	The user can provide preferred timestamp format to generate purge log file name. If not, then the default purge log file name timestamp format (dd-MM-yyyy_ HH.mm.ss.SSS.) will be taken.

Name	Default Value	Example Value	Description
tg.progress.update.seconds	5	any number	The progress bar update frequency for DT Test Group operations-Generate, Execute, Repair, and Bulk Update-can be configured in seconds. If not configured, the system uses a default frequency.

8.12.1.4 DT Generate application.properties

Name	Default Value	Example Value	Description
decision.dt.bim.entity.attribute.max	10	any number	The maximum number of instances in a Test Case BIM level to allow the user to generate. A number too high might overload the database and/or system.
dt.testgroup.export-to-fs.enabled	false	true	When set to "true," Test Group will be exported to the filesystem in the following location: \${export.dir}/test-group-export-dir. This is an additional export location except for the default export to the database, which can later be downloaded through the application.

Name	Default Value	Example Value	Description
dt.testgroup.export-to-fs.threads.max.queue.size	10	Any number	Number of threads to wait for concurrent export requests
dt.testgroup.export-to-fs.threads.max.size	10	Any number	Max number of concurrent threads to perform export requests
dt.testgroup.export-to-fs.zip	false	true	When set to "true," Test Group will be exported to the filesystem in a zip format.
dt.testgroup.import.threads.max.queue.size	10	Any number	Number of threads to wait for concurrent import requests
dt.testgroup.import.zthreads.max.size	10	Any number	Max number of concurrent threads to perform import requests
export.dir	\${decision.home:./} /export	/export-folder	This property will be used as a parent folder of the folder where Test Groups will be exported to. Usually, when running DT as part of DM Tomcat, \${decision.home} will already be set. If not (when running DT as a standalone application), then the default will be ./export.

Name	Default Value	Example Value	Description
generate.tc-permutations.max	10000	any number	The maximum number of Test Cases to allow the user to generate out of test value combinations. A number too high might overload the database and/or system.
logging.file.name	logs/dt_generate.log	/dt-home/logs/dt_generate.log	Path and file name for log file. Recommended to specify full path from file system root.
server.port	8083	Any valid port number	The port number of DT Generate application.
server.servlet.context-path	/dt-generate	Any value	The context path of DT Generate application.
spring.mvc.async.request-timeout	1200000		This is a request timeout for async requests.
test-case.import.and.export.date-time.pattern	yyyy-MM-dd 'T'HH:mm:ss	dd/MM/yyyy HH:mm:ss	Date and time format for date or time Fact Types import or export. Only valid date/time expression standards are allowed. The millisecond section of a format is not supported.

Name	Default Value	Example Value	Description
tg.generate.exportfile.entry.size	1048576	any number	The size of each file (in bytes) when splitting an export of large test groups. The default file size is 1048576 bytes (1 MB).
tg.generate.result.buffer	10	any number	The amount of generated or imported Test Cases to send to DT Repository for saving in each request.
tg.generate.result.concurrency	3	any number	The concurrency level of HTTP requests to DT Repository for saving generated or imported Test Cases.
tg.progress.update.seconds	5	any number	The progress bar update frequency in seconds for DT Test Group export and Bulk Update operations.

8.12.1.5 DT Repair application.properties

Name	Default Value	Example Value	Description
logging.file.name	log/dt_repair.log	/dt-home/logs/dt_repair.log	Specifies the file path and name for the DT Repair application log file. It is recommended to provide a full absolute path for clarity and consistency.
server.port	8081	Any valid port number	Specifies the port number on which the DT Repair application listens for incoming requests.

Name	Default Value	Example Value	Description
server.servlet.context-path	dt-repair	Any value	Defines the context path under which the DT Repair application is accessed.
tg.repair.result.buffer	10	any number	Configures the number of repaired Test Cases to save in each transaction. If not explicitly set, a default value is applied.
tg.repair.result.concurrency	3	any number	Specifies the concurrency level for saving repaired Test Case transactions. If not configured, the system defaults to a predefined concurrency level.

8.12.1.6 DT Executor application.properties

Name	Default Value	Example Value	Description
de.context.path	bdes	Any value	Defines the context path for the DE application testing instance, specifying the base URL path for accessing its endpoints.
de.host	localhost		Specifies the hostname of the DE application testing instance for network communication.
de.password		Any value	Specifies the configured password for the DE testing instance, applicable only if the DE instance is operating in secured mode.

Name	Default Value	Example Value	Description
de.port	8090		Specifies the port number on which the DE application testing instance listens for incoming requests.
de.protocol	http	http, https	Specifies the communication protocol (HTTP or HTTPS) used for interacting with the DE application testing instance.
de.user		admin	Specifies the configured username for the DE testing instance, applicable only if the DE instance is operating in secured mode.
dt.http.timeout.de	20000	any number	Defines the duration in milliseconds to wait for an outgoing asynchronous HTTP request to the DE instance for executing a Test Case, after which the request is considered to have timed out.
logging.file.name	logs/dt_exec.log	/dt-home/logs/dt_exec.log	Specifies the path and file name for the log file of the DT Executor application. It is recommended to provide the full file path starting from the file system root for clarity and consistency.

Name	Default Value	Example Value	Description
server.port	8084	Any valid port number	Specifies the port number on which the DT Executor application listens for incoming HTTP requests.
server.servlet.context-path	/dt-exec	Any value	Specifies the context path under which the DT Executor application is hosted, defining the base URL for accessing application endpoints.
tg.execution.concurrency	30	any number	Specifies the maximum number of test cases that can be executed concurrently. A higher value can enhance execution speed but may increase the risk of overloading the database, DE, or underlying hardware resources.
tg.execution.result.buffer	10	any number	Defines the number of executed Test Case results to be sent to the DT Repository for storage in each HTTP request.
tg.execution.result.concurrency	3	any number	Specifies the concurrency level of HTTP requests to the DT Repository for saving executed Test Case results.

8.12.1.7 DT Orchestrator application.properties

Name	Default Value	Example Value	Description
decision.dt.revision.tgs.export.max.size.mb	250	Any number	Defines the maximum size (in MB) of the entire Release Test Groups export file uploaded to DM during the Release Test Groups export process.
decision.dt.tg.export.max.size.mb	70	Any number	Specifies the maximum size (in MB) of an individual Test Group export file generated in DT Generate during the Release Test Groups export process.
dm.context.path	dm	Any value	Specifies the context path for accessing the DM application.
dm.host	localhost	Valid host name	Specifies the hostname of the DM application.
dm.port	9080	Valid port number	Defines the port number on which the DM application listens.

Name	Default Value	Example Value	Description
dm.protocol	http	http,https	Specifies the communication protocol (HTTP/HTTPS) for the DM application.
dt.http.timeout.deploy-min	5	any number	Specifies the timeout duration (in minutes) for a synchronous HTTP request to DE when deploying an asset. Increase this value if large assets cause timeout errors.
dt.http.timeout.import.default	120000	Any number	Specifies the timeout duration (in milliseconds) for a synchronous HTTP request to DT Generate during the Test Group import process. Increase if large imports face timeout errors.

Name	Default Value	Example Value	Description
dt.http.timeout.test-value-generation	120000	any number	Specifies the timeout duration (in milliseconds) for a synchronous HTTP request to DM when generating test values for an asset. Adjust for larger assets if timeout errors occur.
dt.orchestrate.export-to-fs.threads.max.queue.size	10	Any number	Defines the maximum number of threads waiting in the queue for concurrent export requests to DT Generate.
dt.orchestrate.export-to-fs.threads.max.size	10	Any number	Specifies the maximum number of concurrent threads allowed for calling DT Generate during export requests.

Name	Default Value	Example Value	Description
dt.orchestrate.export.assets.concurrency	1	Any number	Sets the number of assets in a Release that can be processed concurrently for exporting their Test Groups.
dt.orchestrate.export.assets.tg.concurrency	2	Any number	Defines the number of Test Groups of a single asset in a Release that can be processed concurrently for export.
dt.timeout.export.release-tgs-min	5	any number	Defines the timeout duration (in minutes) for a synchronous HTTP request to DM when uploading a Release Test Groups export.
dt.timeout.export.test-group-min	20	any number	Specifies the timeout duration (in minutes) for a synchronous HTTP request to DT Generate for exporting each Test Group within Release Test Groups. Adjust if timeout errors occur.

Name	Default Value	Example Value	Description
logging.file.name	logs/dt_orch.log	/dt-home/logs/dt_orch.log	Specifies the path and file name for the log file. It is recommended to use the full path starting from the file system root for clarity.
orphan.test.groups.delete.cron.schedule.time	0 0 0 * * ?	For valid values, refer to Spring Cron Expressions Documentation .	Configures the cron schedule for the background process that identifies and deletes orphan Test Cases.
server.port	8085	Any valid port number	Defines the port number on which the DT Orchestrator application will listen.
server.servlet.context-path	/dt-orchestrator	Any value	Specifies the context path for accessing the DT Orchestrator application.

Name	Default Value	Example Value	Description
spring.mvc.async.request-timeout	1200000	Any number	Defines the timeout duration (in milliseconds) for asynchronous HTTP requests between DT micro-apps. It should be set at least 33% higher than any other asynchronous timeout property.
testing.exec.context.path	dt-exec	Any value	Specifies the context path for accessing the DT Executor application.
testing.exec.host	localhost	Valid host name	Specifies the hostname for the DT Executor application.
testing.exec.port	8084	Valid port number	Defines the port number on which the DT Executor application listens.
testing.exec.protocol	http	http,https	Specifies the communication protocol (HTTP/HTTPS) for the DT Executor application.

Name	Default Value	Example Value	Description
testing.generate.context.path	dt-generate	Any value	Specifies the context path for accessing the DT Generate application.
testing.generate.host	localhost	Valid host name	Specifies the hostname for the DT Generate application.
testing.generate.port	8083	Valid port number	Defines the port number on which the DT Generate application listens.
testing.generate.protocol	http	http,https	Specifies the communication protocol (HTTP/HTTPS) for the DT Generate application.
testing.repair.context.path	dt-repair	Any value	Specifies the context path for accessing the DT Repair application.
testing.repair.host	localhost	Valid host name	Specifies the hostname for the DT Repair application.
testing.repair.port	8081	Valid port number	Defines the port number on which the DT Repair application listens.

Name	Default Value	Example Value	Description
testing.repair.protocol	http	http,https	Specifies the communication protocol (HTTP/HTTPS) for the DT Repair application.
tg.events.heartbeat-period-sec	60	Any number	Specifies the heartbeat interval (in seconds) for the server to maintain the TCP socket connection with the client. A lower value ensures more frequent health checks.

8.12.1.8 Health Service application.properties

The health-service app is intended to perform a global health check of the environment. The “health-service-config/application.properties” configuration file is used to define which services will be included in the health-check. For each configured service, the health-check application will call the configured services healthcheck API to check its health and report it with the overall application health check.

By default, the property file contains the following required endpoints for component services required to run Decision Manager:

- health.service.service-info.0.name=dmService
- health.service.service-info.0.url=\${dm.protocol}://\${dm.host}:\${dm.port}/dm/ws/api/actuator/health
- health.service.service-info.1.name=de
- health.service.service-info.1.url=\${dm.protocol}://\${dm.host}:\${dm.port}/bdes/ws/rs/api/actuator/health
- health.service.service-info.2.name=dt-repair
- health.service.service-info.2.url=\${dm.protocol}://\${dm.host}:\${dm.port}/dt-repair/actuator/health
- health.service.service-info.3.name=dt-orchestrate
- health.service.service-info.3.url=\${dm.protocol}://\${dm.host}:\${dm.port}/dt-orchestrate/actuator/health
- health.service.service-info.4.name=dt-repo

- health.service.service-info.4.url=\${dm.protocol}://\${dm.host}:\${dm.port}/dt-repo/actuator/health
- health.service.service-info.5.name=dt-exec
- health.service.service-info.5.url=\${dm.protocol}://\${dm.host}:\${dm.port}/dt-exec/actuator/health
- health.service.service-info.6.name=dt-generate
- health.service.service-info.6.url=\${dm.protocol}://\${dm.host}:\${dm.port}/dt-generate/actuator/health

Each service that should be checked should have a name property and a url property.

Each service has also its own index in the properties list that affects the order of health check calls.

For example,

- health.service.service-info.0.name=dmService
- health.service.service-info.0.url=\${dm.protocol}://\${dm.host}:\${dm.port}/dm/ws/api/actuator/health

In the above example, \${dm.protocol}, \${dm.host} and \${dm.port} are variables that get resolved during runtime by preset System properties or Environment Variables. You can edit the file to remove/add/change an endpoint.

8.12.2 Advanced DT-related properties on DM

Name	Default Value	Example Value	Description
proxy.connections.route.max	500	any number	The max number of concurrent connections to keep in a route.
proxy.connections.total.max	2000	any number	The max number of total concurrent connections to keep.
proxy.timeout.socket	360000	any number	Duration in milliseconds to allow inactivity between two consecutive data packets on an HTTP socket from DM proxy to DT until it is considered a timeout. 0 = infinite This value must be higher than ANY dt timeout duration property defined in the other files.
proxy.timeout.connect	120000	any number	Duration in milliseconds to wait for a connection to be established for an HTTP request from DM proxy to DT until it is considered a timeout.

Name	Default Value	Example Value	Description
proxy.timeout.connection-request	120000	any number	Duration in milliseconds to wait for a connection request to be accepted for an HTTP request from DM proxy to DT until it is considered a timeout.
proxy.timeout.async-context.http	360000	any number	Duration in milliseconds to wait for an async HTTP request from DM proxy to DT until it is considered a timeout. This value must be higher than ANY dt timeout duration property defined in the other files.
proxy.timeout.async-context.sse	0	any number	Duration in milliseconds to wait for an async SSE HTTP request from DM proxy to DT until it is considered a timeout. 0 = infinite

8.12.3 Glossary

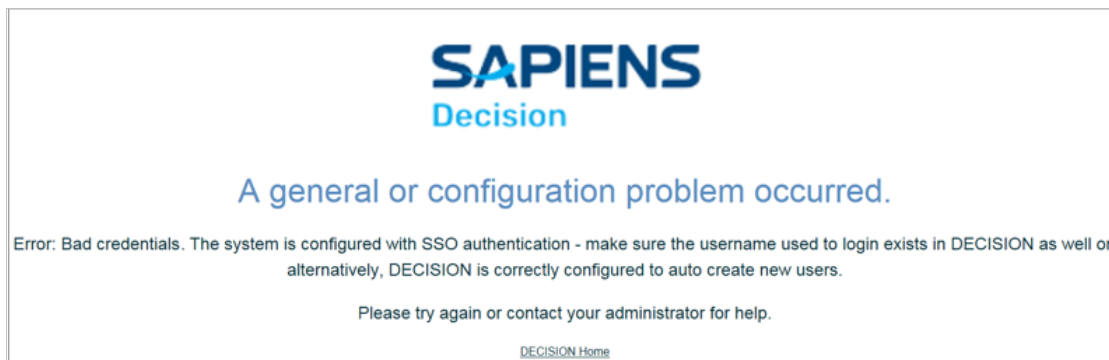
The following table lists terms and its definitions used throughout this document.

Term	Description
IDP	An Identity Provider (IDP) is the entity that is responsible for the user end-to-end authentication. It manages the users, passwords, and additional info of the users and provides the login page and any of a variety of authentication security elements (change of password, password strength, CAPTCHA, 2-factor authentication, and so on). To integrate with Decision SSO, the IDP must support Security Assertion Markup Language (SAML) 2.0. Examples of IDPs: • Shibboleth • ADFS • Okta • OpenAM • Oracle Identity Federation • PingFederate • WSO2 Identity Server
SSO	Single sign-on (SSO) is a session and user authentication service that permits a user to use one set of login credentials to access multiple applications. The service authenticates the end user for all the applications to which the user has been given rights and eliminates further prompts when the user switches applications during the same session.
SAML	Security Assertion Markup Language (SAML) is the protocol and standard used for the SSO web-based communication between the IDP and SP to achieve user authentication.
Service Provider (SP)	A Service Provider is the entity that provides the required application service for the user, DM in our scenario. It leaves authentication handling to the IDP and is responsible only for the authorization management and assignment which is assigning the relevant role to the logged in user.
SP / IDP Metadata	Both the SP and the IDP have a static predefined XML metadata document that describes SSO-related metadata that lists the URL which should be used to access it for each SSO operation such as sign in, sign out and cryptographic information for secured communication.

8.12.4 Troubleshooting

8.12.4.1 Align IDP users with DM users

Error: A general or configuration problem occurred.



Solution: DM application users must be aligned with the IDP users based on the username. If a user exists in the IDP, the same user must be able to authenticate in DM. When navigating to DM, if a user of the same name does not exist in DM, the above error message is displayed.

To solve the issue, perform the following:

Setup the Administrator User

Note: At least one administrator account in the IDP should exist as an administrator in DM.

- For a new DM Installation: By default, the DM database contains an administrator user called “admin”. If your IDP administrator user for DM is not created as “admin”, you have to directly update the DM database to modify “admin” to the desired administrator name:

Execute the following command on the DM database (e.g. for Oracle):

```
UPDATE APP_USER SET NAME='<desired username here>' WHERE  
name='admin'
```

- For an existing DM Installation moving to SSO authentication, you should know the name of your existing DM administrator. If your IDP administrator user for DM is not the same, you must directly update the DM database to modify it to the desired administrator name:

Assuming your existing DM admin is called “dm_admin”

Execute the following command on the DM database (e.g. for Oracle):

```
UPDATE APP_USER SET NAME='<desired username here>' WHERE name='dm_  
admin'
```

- Restart DM and log in with your IDP administrator username. You should now be able to access DM’s security administration and create or update users and roles to be aligned with the IDP users.

8.12.4.2 DM metadata generation failure

Error: 500 – Unexpected server error occurred during processing.



Solution: The issue occurs when DM fails to generate its SAML metadata for the following reasons.

- KeyStore file corrupted or generated in a different operating system rather than DM machine.
- KeyStore file name, key name or password set incorrectly in “KeyStore DM configuration”. For more information, refer to the KeyStore DM Configuration section.
- Custom definition mismatch in the “Advanced DM SSO configurations”. For more information, refer to the DM Configuration section.

SAPIENS

Partnering for Success

Contact Us

For more information, please visit or contact us at:

info.sapiens@sapiens.com

For Support related queries, contact us at:

Decision.Support@sapiens.com

For Documentation related queries, contact us at:

Decision_documentation@sapiens.com

© 2025 Copyright Sapiens International. All Rights Reserved.

www.sapiensdecision.com



Partnering for Success